

C++学习记录

C++三大特性

1. 封装：将方法与数据结合在一起，并通过访问权限隐藏内部实现。public：类的内外部都可以访问；protected：不能类外部访问，能被子类访问；private：既不能类外部访问，也不能被子类访问。
2. 继承：通过继承，子类可以获取父类的所有属性，并可以对属性进行扩展。
3. 多态：通过同一个接口调用不同类型对象，表现的行为不同。

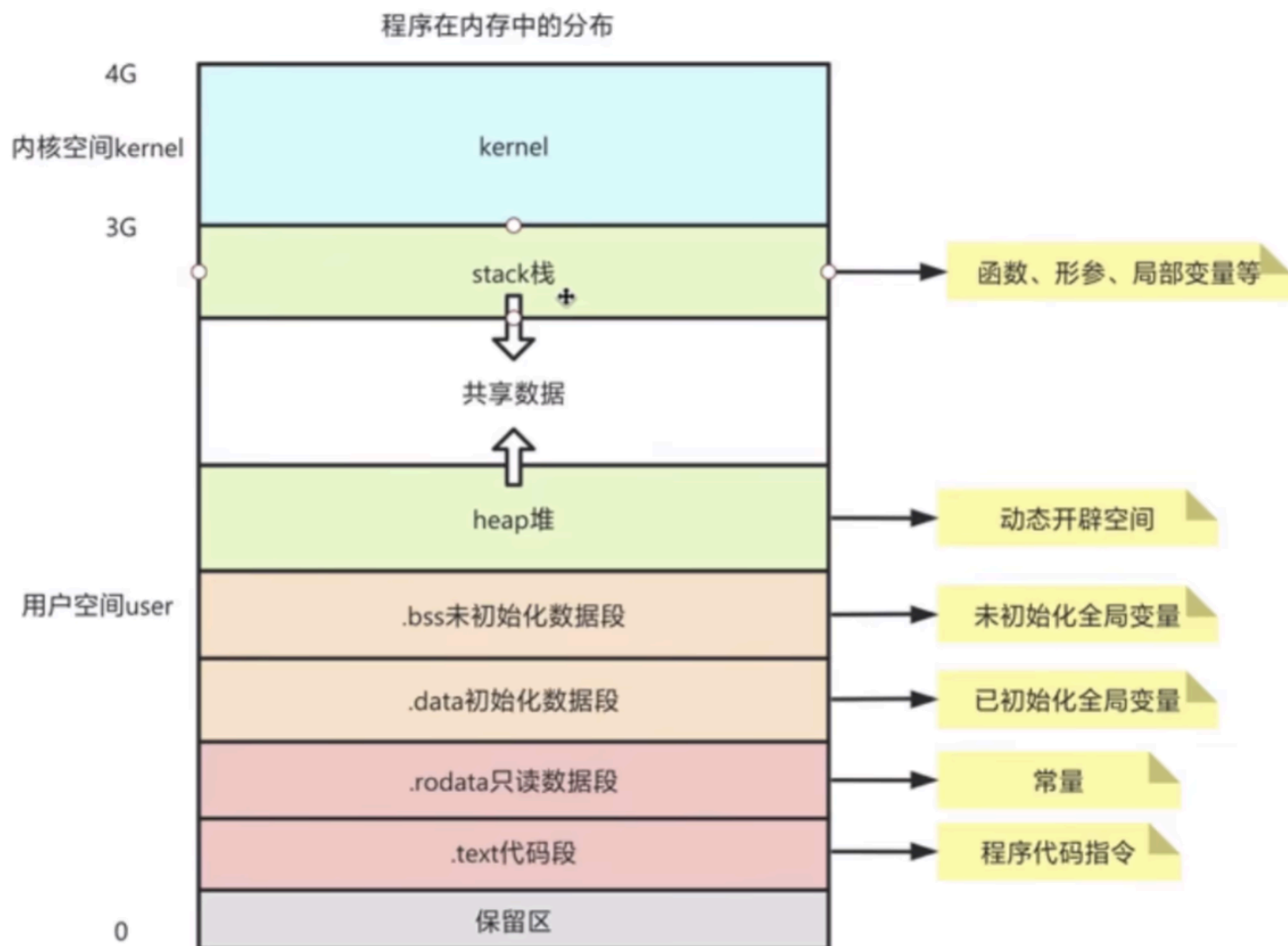
C++从源代码到运行通常包含以下几个步骤

1. 预处理：处理源代码中的预处理指令，例如#include、#define等
2. 编译：将预处理后的文件转换为汇编代码
3. 汇编：将汇编代码转换为机器代码（目标文件）
4. 链接：将一个或多个目标文件以及库文件链接成一个可执行文件
5. 运行

代码块

```
1  # 预处理 → 编译 → 汇编 → 链接 → 运行
2  g++ -E hello.cpp -o hello.i      # 1. 预处理（展开头文件、宏）
3  g++ -S hello.i -o hello.s        # 2. 编译（生成汇编代码）
4  g++ -c hello.s -o hello.o        # 3. 汇编（生成目标文件）
5  g++ hello.o -o hello             # 4. 链接（生成可执行文件）
6  # 也可以合并编译链接，如下：
7  g++ hello.cpp -o hello
8  ./hello                         # 5. 运行
```

内存划分



C++中字节对齐

字节对齐是指编译器在内存中为数据结构中的成员变量分配内存时，遵循特定的对齐规则，以提高访问效率和满足硬件要求。**空间换取时间**。

CPU 从内存读取数据时，通常以“字 (Word)”为单位（在64位系统下是8字节）。假设一个 `int`（4字节）的地址是奇数（如 `0x1001`），CPU 为了读取它，可能需要发起**两次**内存访问请求（先读 `0x1000`，再读 `0x1004`），然后丢弃无效字节并拼接数据。

- **不对齐的开销：**访问次数翻倍，效率极低。
- **对齐后：**如果 `int` 存储在 `0x1000` 或 `0x1004`（4的倍数），CPU 只需要一次总线周期即可读取。

代码块

- 1 三原则：
- 2 1. 第一个成员在结构体地址偏移为0的地址处
- 3 2. 其他成员变量要对齐到某个数字（对齐数）的整数倍的地址处
- 4 3. 结构体总大小为最大对齐数的整数倍
- 5

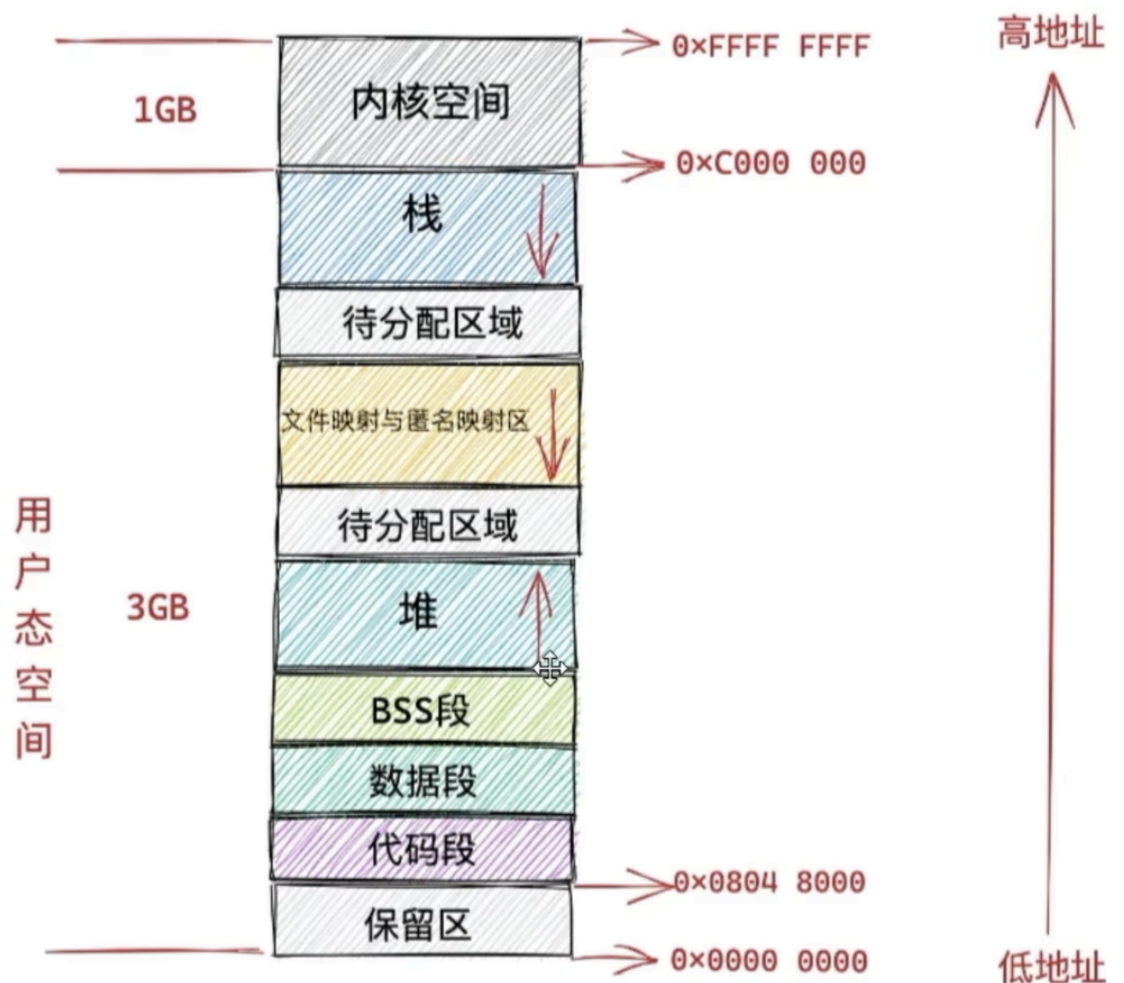
```

6  struct Mixed {
7      char a;        // 1字节, 偏移0
8      short b;       // 2字节, 偏移2 (2 % 2 = 0)
9      int c;         // 4字节, 偏移4 (4 % 4 = 0)
10     char d;        // 1字节, 偏移8
11     // 最大对齐值: 4
12     // 当前大小: 9字节, 需要填充到4的倍数
13     // 最终大小: 12字节 (9 + 3填充)
14 };

```

堆和栈

在 32 位机器上，进程虚拟内存空间分布如下（注：图片来源于网络）



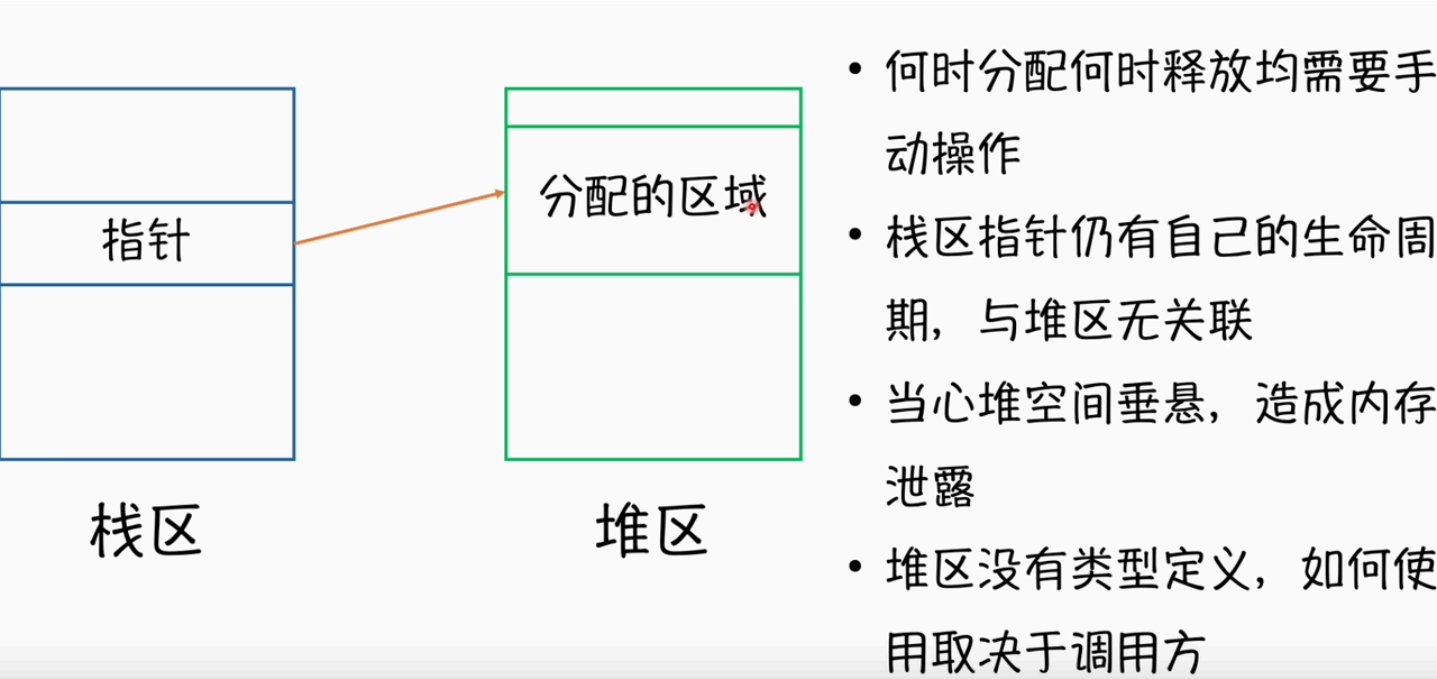
堆是存储动态分配的对象或数据结构，由程序员手动管理动态分配内存。**栈**由操作系统动态分配内存区域，用于存储函数调用和局部变量。

1. 为什么栈的分配速度比堆快？

栈只需要移动指针（rsp指针），堆需要查找空闲块、处理碎片和系统调用

2. 多线程程序中，栈和堆如何隔离？

栈是线程私有，堆是进程共享，如果多个线程去访问，需要加锁同步



堆内存和栈内存的区别

特性	栈内存	堆内存
分配方式	自动分配（由操作系统管理）	手动分配（由程序员管理）
生命周期	随着函数调用和返回自动管理	程序员显式管理，直到手动释放
存储内容	局部变量、函数调用信息	动态分配的数据、对象
访问速度	非常快	较慢（分配和释放较慢）
内存限制	通常较小（受限于操作系统）	较大（受限于系统内存）
线程安全性	每个线程有独立的栈	多线程共享堆，需要同步控制
管理难度	自动管理，无需手动干预	程序员必须手动管理

1. 栈内存由操作系统自动管理，通过栈指针进行分配和释放，堆内存是由程序员通过new/delet或者 malloc/free手动分配，手动释放
2. 栈内存存放局部变量、函数调用信息、形参；堆是存放动态分配的数据、对象

3. 栈的访问速度快，基于栈指针的简单递增或递减操作；堆的访问速度较慢，操作系统需要进行内存的分配和管理
4. 栈内存通常较小，堆内存较大
5. 每个线程有独立的栈，线程间不会相互影响；堆内存是跨线程共享的，需要同步控制

堆栈溢出是什么，会怎么样

堆溢出：比如不断的new 一个对象，一直创建新的对象，而不进行释放，最终**导致内存不足**。将会报错：OutOfMemory Error

栈溢出：一次函数调用中，栈中将被依次压入：参数，返回地址等，而方法如果**递归比较深或进去死循环**，就会导致栈溢出。将会报错：StackOverflow Error

new/delete和malloc/free的区别是什么

new与delete是运算符，new负责分配内存并调用对象的构造函数初始化对象，返回初始化对象类型的指针；delete释放内存并调用析构函数。malloc和free是函数，malloc只会分配内存不会调用构造函数，返回 `void*` 指针，需手动转换类型；free只释放内存，不会调用析构函数。new和delete在编译期间确定，malloc和free在运行时确定

new/delete的具体步骤

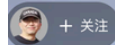
使用new操作符来分配对象内存时会经历三个步骤：

- 第一步：调用operator new 函数分配一块足够大的，原始的，未命名的内存空间以便存储特定类型的对象。
- 第二步：编译器运行相应的构造函数以构造对象，并为其传入初值。
- 第三步：对象构造完成后，返回一个指向该对象的指针。

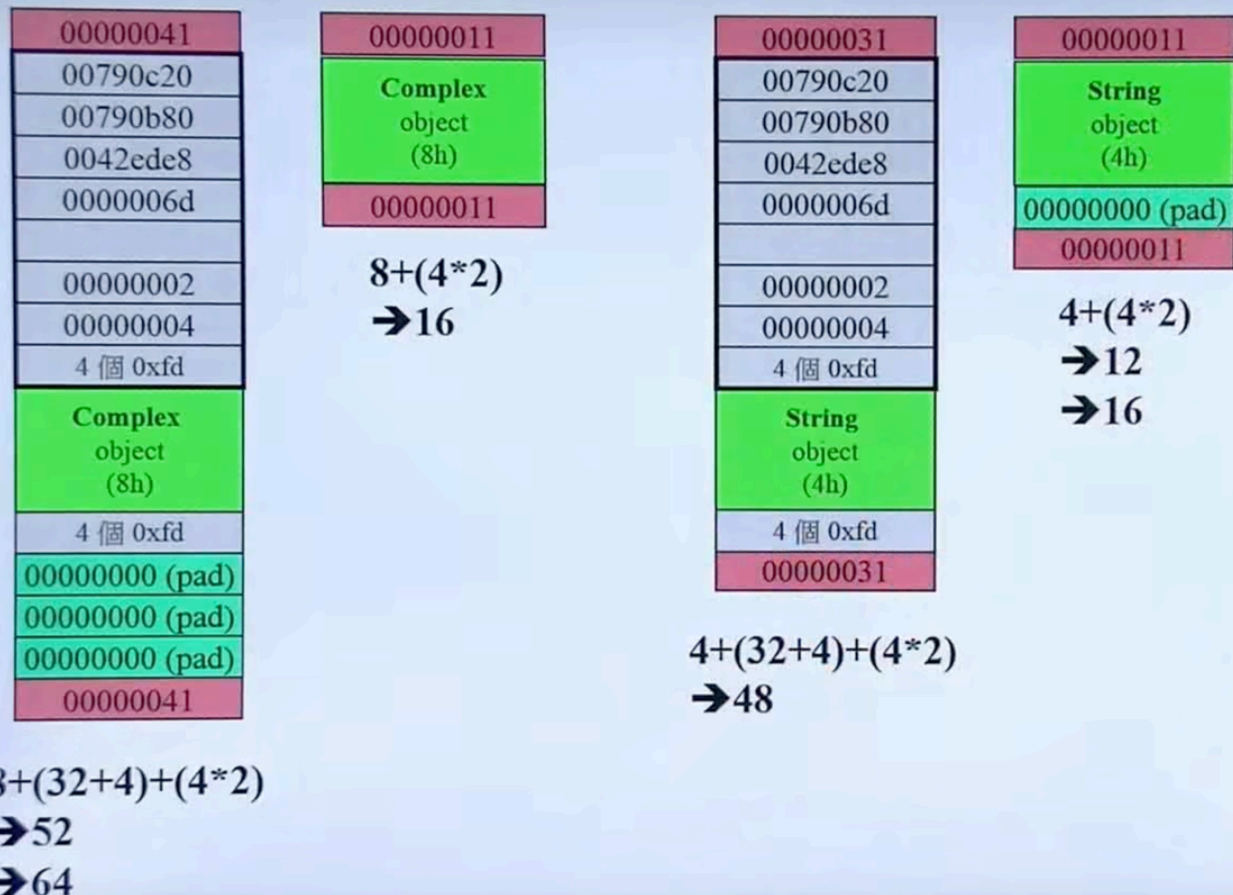
使用delete操作符来释放对象内存时会经历两个步骤：

- 第一步：调用对象的析构函数。
- 第二步：编译器调用operator delete函数释放内存空间。

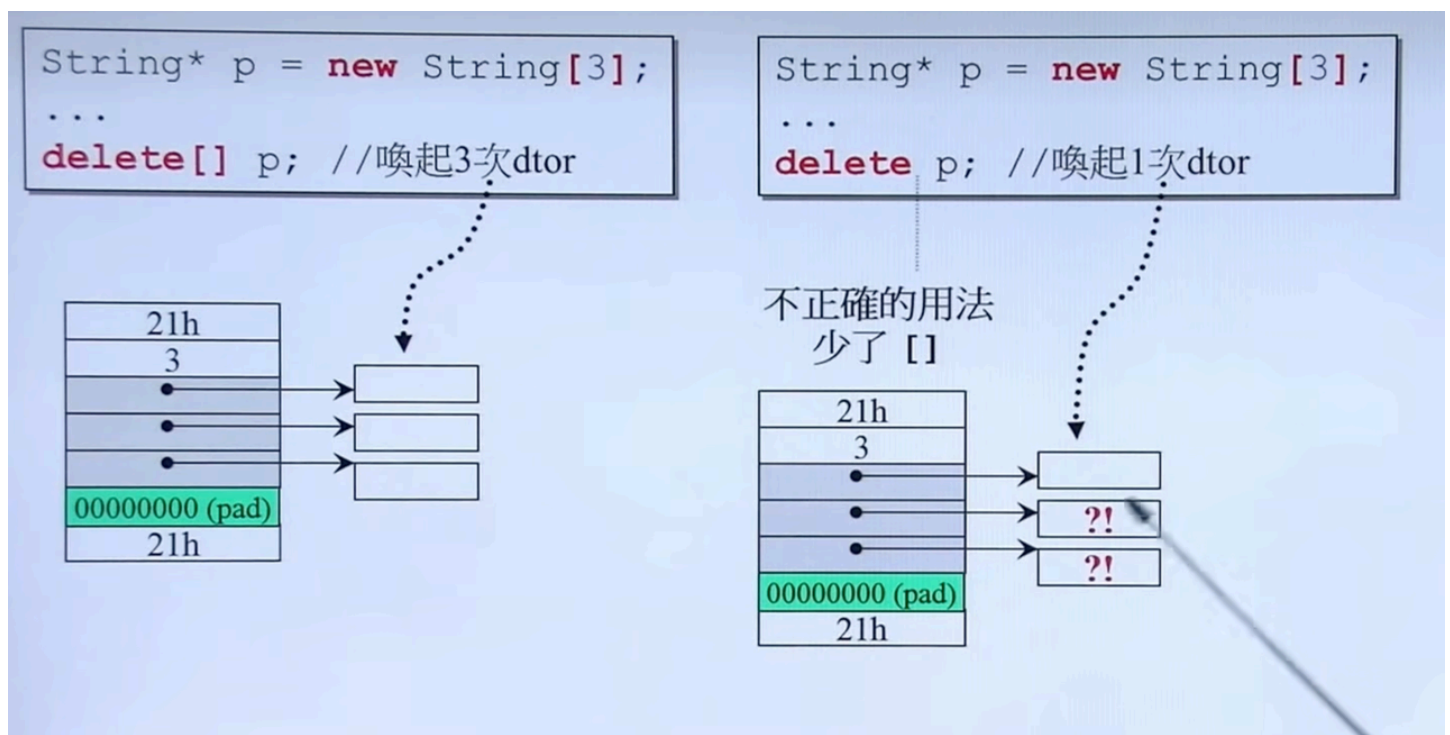
动态分配内存块大小：



+ 关注



new 一个数组，delete 也得加[]，原因如下：不加数组中括号只会调用一次析构函数，只能删除第一个指针指向的数组内存，剩余的不会删除。



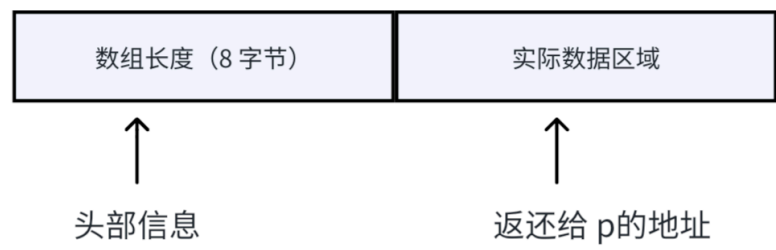
new [] 数组的时候，在 delete 时不需要指定删除的大小，只需要 delete []，原因是：在 new [] 的时候，编译器会在返回给你的内存地址前面（或另外的地方）额外存储一个头部信息，里

面就**包含数组的元素个数**。当你执行 `delete[]` 时，它会根据这个头部信息来知道应该调用多少次析构函数，以及释放多少内存。

代码块

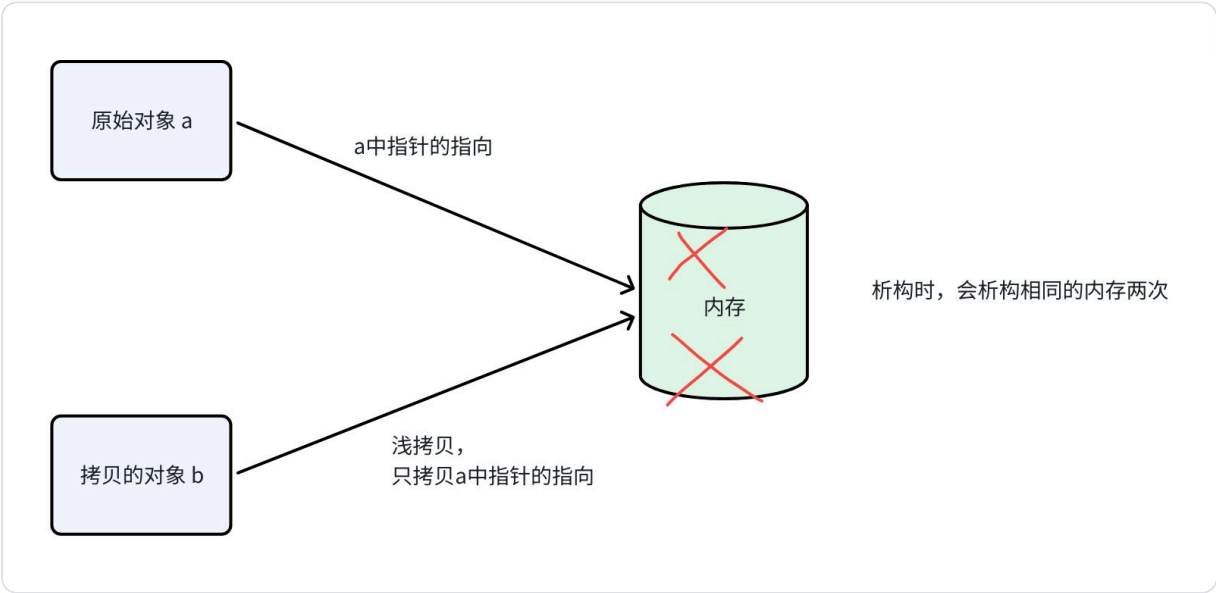
```
1 int *p =new int[10];
```

实际分配的内存布局可能像这样：

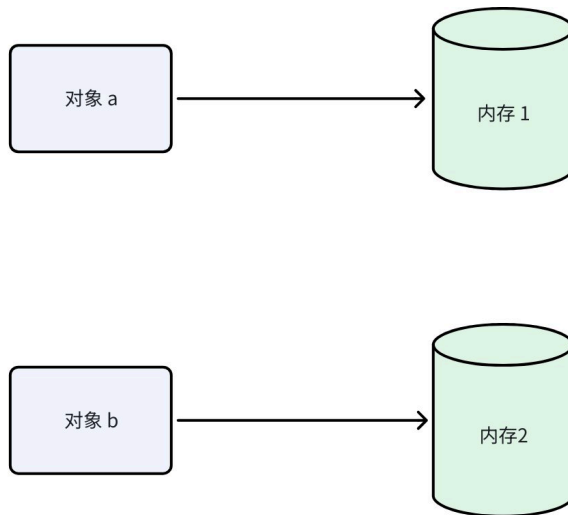


深拷贝与浅拷贝

- 1. **浅拷贝**：浅拷贝是指创建一个新对象，新对象的成员变量会直接复制原对象中成员的值，如果成员是指针类型，那么浅拷贝**只是复制了指针的地址**，而不复制指针指向的内容，导致两个对象共享同一块内存区域。这种共享可能导致**双重释放**或在一个对象销毁时另一个对象失效，以及修改一个对象的资源影响另一个对象，容易带来**悬空指针**和**资源管理混乱**的问题。



- 2. **深拷贝**：不仅复制对象的成员变量的值，**还会为指针成员分配新的内存，并复制指针指向的内容**。结果是每个对象拥有独立的内存空间。



3. 拷贝构造函数的一般形式：

代码块

```
1  ClassName(const ClassName& other);
```

- 拷贝构造函数是以**常量引用**方式接收源对象，**防止在拷贝过程中再次调用拷贝构造或改变源对象**。
- 在函数体内部，对每个动态资源成员：
 - a. 检查源对象的指针是否为空（如果指针可能为空）。
 - b. 根据源对象资源大小或约定，调用new为目标对象分配合适的内存。
 - c. 使用循环或std::copy将源对象的动态数据复制到目标对象的内存中。
- 对其他普通数据成员（如int、std::string等），可以直接在初始化列表中进行复制或调用其拷贝构造函数。

示例：

代码块

```
1  class MyBuffer {
2  public:
3      // 默认构造函数
4      MyBuffer(size_t n) : size(n), data(new int[n]) {
5          std::cout << "构造, 分配大小 " << size << std::endl;
6          for (size_t i = 0; i < size; ++i) {
7              data[i] = static_cast<int>(i); // 初始化示例
8          }
9      }
10
11      // 拷贝构造函数：实现深拷贝
```

```

12     MyBuffer(const MyBuffer& other)
13         : size(other.size), data(nullptr) // 先初始化非指针成员
14     {
15         std::cout << "拷贝构造, 进行深拷贝, 原大小 " << other.size << std::endl;
16         if (other.data) {
17             // 为目标对象分配新的内存
18             data = new int[size];
19             // 将源对象的数组内容逐项复制到新内存
20             std::copy(other.data, other.data + size, data);
21         }
22     }
23
24     // 拷贝赋值运算符: 也应实现深拷贝, 并处理自赋值
25     MyBuffer& operator=(const MyBuffer& other) {
26         if (this != &other) {
27             std::cout << "拷贝赋值, 进行深拷贝, 原大小 " << other.size <<
std::endl;
28             // 先释放当前对象已有资源
29             delete[] data;
30
31             // 复制 size, 并为新数据分配内存
32             size = other.size;
33             if (other.data) {
34                 data = new int[size];
35                 std::copy(other.data, other.data + size, data);
36             } else {
37                 data = nullptr;
38             }
39         }
40         return *this;
41     }
42
43     // 析构函数: 释放动态内存
44     ~MyBuffer() {
45         std::cout << "析构, 释放大小 " << size << std::endl;
46         delete[] data;
47     }
48
49     // 打印缓冲区内容, 辅助测试
50     void print() const {
51         std::cout << "内容:";
52         for (size_t i = 0; i < size; ++i) {
53             std::cout << " " << data[i];
54         }
55         std::cout << std::endl;
56     }
57

```

```

58     private:
59         size_t size; // 缓冲区长度
60         int* data;    // 动态数组
61     };
62
63     int main() {
64         MyBuffer buf1(5);          // 构造一个长度为 5 的缓冲区
65         buf1.print();
66
67         std::cout << "\n== 使用拷贝构造函数创建 buf2 ==\n";
68         MyBuffer buf2 = buf1;      // 深拷贝 buf1 到 buf2
69         buf2.print();
70
71         std::cout << "\n== 修改 buf1 数据 ==\n";
72         // 修改 buf1 的数据
73         // 由于 buf2 是深拷贝，它的数据与 buf1 独立
74         for (size_t i = 0; i < 5; ++i) {
75             // data 是私有成员，此处假设有 setter 或直接访问
76             // 为了示例，此处仅模拟内部修改效果
77             // 实际可提供接口修改，或 const_cast 访问 data
78         }
79
80         std::cout << "\n== 使用拷贝赋值将 buf1 赋值给 buf3 ==\n";
81         MyBuffer buf3(3);
82         buf3 = buf1;                // 深拷贝赋值
83         buf3.print();
84
85         return 0;
86     }
87

```

拷贝构造函数的参数是什么传递方式

拷贝构造的参数必须是引用传递。

如果拷贝构造函数中的参数不是一个引用，即形如 `CClass(const CClass c_class)`，那么就相当于采用了传值的方式(`pass-by-value`)，而传值的方式会调用该类的拷贝构造函数，从而造成无穷递归地调用拷贝构造函数。因此拷贝构造函数的参数必须是一个引用。

需要澄清的是，传指针其实也是传值，如果上面的拷贝构造函数写成 `CClass(const CClass* c_class)`，也是不行的。事实上，只有传引用不是传值外，其他所有的传递方式都是传值

C++中一个空类必定包含什么，这个空类的大小是多少

一个空类包含：

1. 默认构造函数（如果没声明其他构造函数）
2. 拷贝构造函数
3. 拷贝赋值运算符
4. 析构函数
5. 移动构造函数（C++11 起，如果没有用户声明的特殊成员函数）
6. 移动赋值运算符（C++11 起，如果没有用户声明的特殊成员函数）

空类的大小：**1 字节**（byte），这是为了确保每个对象都有唯一的地址，即使它本身不包含任何数据。这样做的原因是为了保证对象能够被区分和引用

static

1. 静态成员变量 -- 属性
2. 静态成员函数 -- 方法

与对象无关

代码块

```
1  struct Test{
2      static int sta_member; // 1
3      static void sta_method(); // 2
4      int member;
5      void method();
6  }
7  void Demo(){
8      Test a,b;
9      a.sta_member=6;
10     std::cout<<b.sta_member; // 6
11     std::cout<<Test::sta_member; // 6
12     a.sta_method(); b.sta_method(); Test::sta_method(); // 静态成员函数可以用类名
        调用
13 }
```

3. 静态全局变量
4. 静态普通函数

3 4 静态全局变量/函数



a.cpp

```
int glo_val = 1;
```



b.cpp

```
extern int glo_val;  
glo_val = 5;
```

```
static int sta_glo_val = 1;
```

```
static int sta_glo_val;  
sta_glo_val = 5;
```

static → inner

对于非静态全局变量/函数，在a文件中定义了一个全局变量，在b文件中定义了相同的全局变量，那么在链接时会链接为同一个变量。如果是静态全局变量/函数，在ab文件中是两个不同的变量，只会在本文件中生效不会跨文件链接。

5. 静态局部变量

与非静态局部变量的区别：生命周期为第一次初始化时创建，程序结束时释放

指针

1. 野指针

指针变量指向非法的内存空间

野指针: 指针变量指向非法的内存空间

示例2: 野指针

```
1 int main() {
2
3     //指针变量p指向内存地址编号为0x1100的空间
4     int * p = (int *)0x1100;
5
6     //访问野指针报错
7     cout << *p << endl;
8
9     system("pause");
10
11     return 0;
12 }
```

C++

总结: 空指针和野指针都不是我们申请的空间, 因此不要访问。

2. const

`const` 关键字在C/C++中并不是代表真正的常量, 而是应当理解为 `read-only`, 也就是只读。用 `const` 修饰的类型不可被修改, 只能读取。



解类型: 对指针而言, 解类型是去掉*后的类型。如: `int *p`的解类型是`int`, `int **p`的解类型是`int *`

主要区分`const`修饰的是指针类型本身还是指针的默认解类型中的类型:

代码块

```
1 int *p1; // 指针本身可变, 默认解类型是int
2 const int *p2; // 指针本身可变, 默认解类型是const int
3 int *const p3; // 指针本身不可变, 默认解类型是int
4 const int *const p4; // 指针本身不可变, 默认解类型是const int
```

C++中提供了一个模板工具 `std::remove_const`, 用于去掉类型的 `const` 修饰, 这里要注意的是, 它去掉的是类型本身的 `const`, 而跟解类型是完全没有关系的, 会原样保留, 比如说:

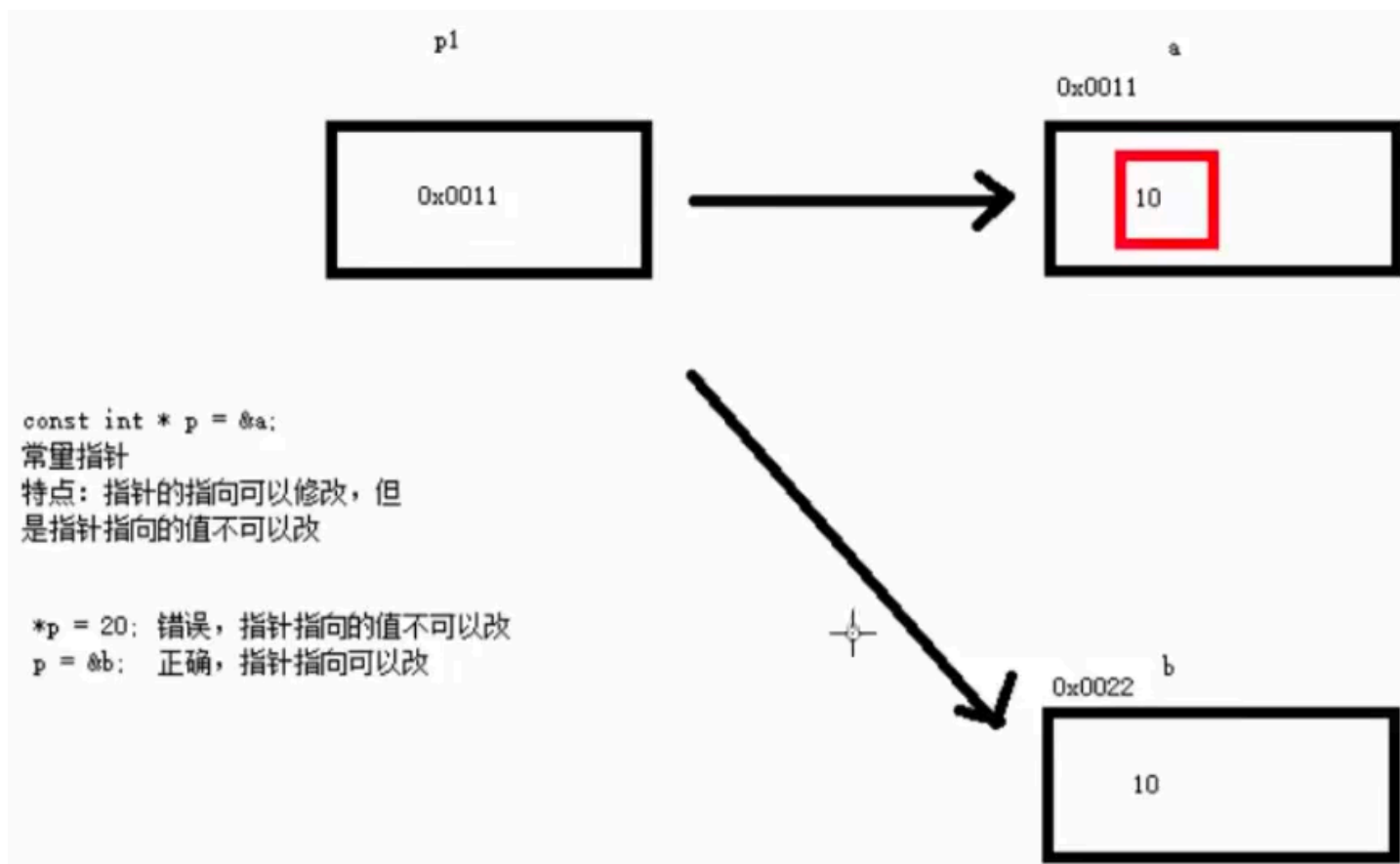
代码块

```
1 std::remove_const_t<const int *>; // const int *
2 std::remove_const_t<int *const>; // int *
```

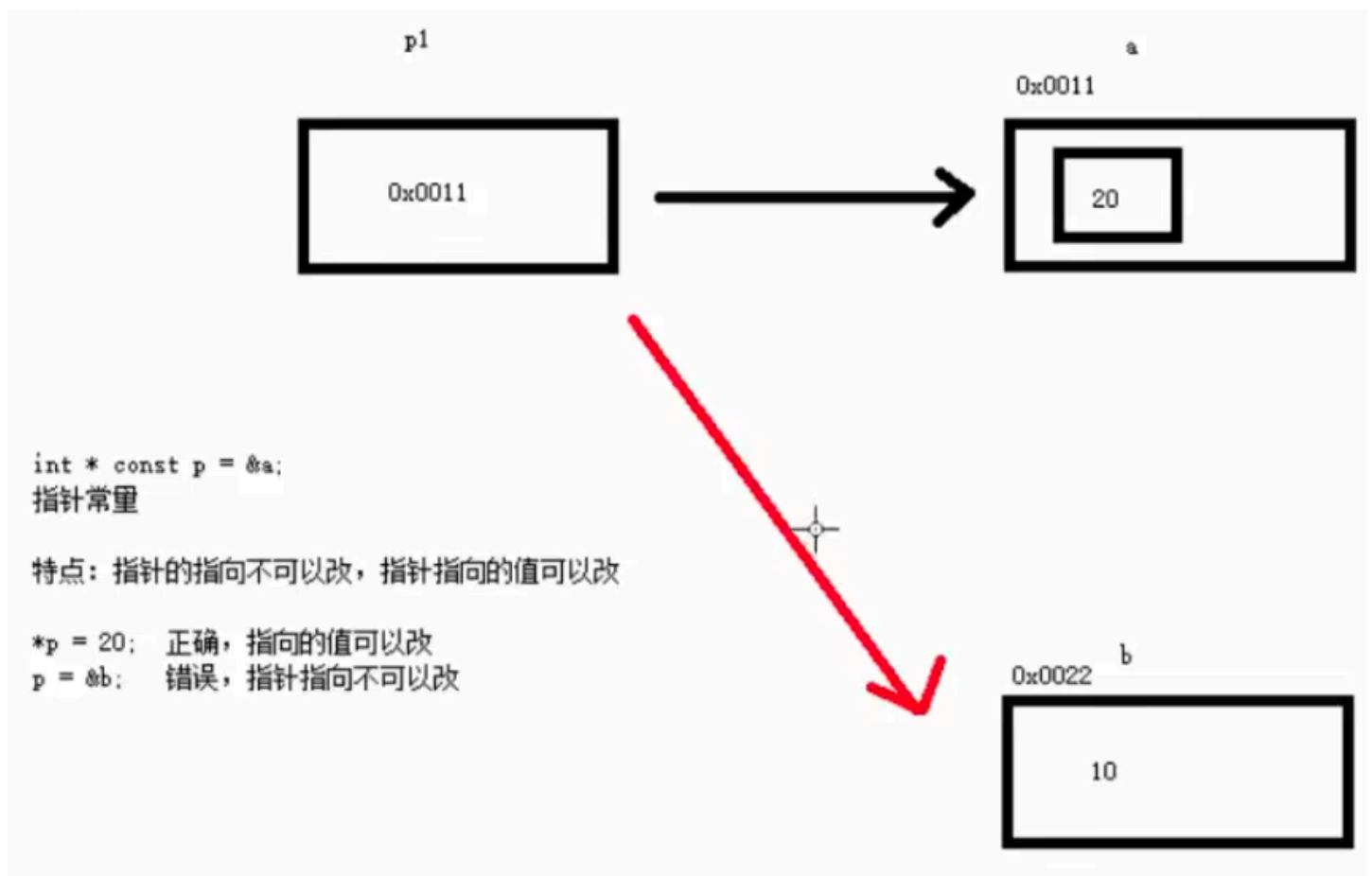
```
3 std::remove_const_t<const int *const>; // const int *
```

	特点
常量指针 <code>const int * p = &a</code>	解类型是 <code>const int</code> ，解类型不可修改。指针本身可以修改。指针的指向可以修改， <code>p=&b</code> (✓)；指针指向的值不可以修改 <code>*p=20</code> (✗)
指针常量 <code>int * const p = &a</code>	解类型是 <code>int</code> ，解类型可修改，指针本身不可以修改。指针的指向不可以改，指针指向的值可以改 <code>*p=20</code> (✓)；指针的指向不可以修改 <code>p=&b</code> (✗)

1. `const` 修饰 指针 --- 常量指针 `const int * p = &a`。其中指针就是地址



2. `const` 修饰 常量 --- 指针常量 `int * const p = &a`



3. 函数指针/指针函数

1. 函数指针：是一个**指针**，这个指针是一个指向函数的指针

代码块

```
1 // 函数返回值 (*函数指针名)(参数列表)
2 typedef double (*MyFuncPtr) mulitply(double,double);
```

应用场景

作为函数的参数

回调函数

作用：架构分层，代码优化

2. 指针函数：是一个**函数**，只不过这个函数的**返回值是一个指针**。指针函数的返回值**不要返回局部变量的地址**

代码块

```
1 int *func(){
2     static int a = 10; // 非局部变量
3     return &a;
4 }
5 int main(){
```

```
6     int *p=func();
7     std::cout<<"*p ="<<*p<<std::endl;
8 }
```

4. 指针和数组

1. 利用指针访问数组中的元素

代码块

```
1  #include <iostream>
2
3  int main() {
4      // 指针访问数组中的元素
5      int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
6      int *p = arr;
7      std::cout << "第一个元素: " << *p << std::endl;
8      std::cout << "利用指针访问数组的每个元素:" << std::endl;
9      for (int i = 0; i < sizeof(arr) / sizeof(arr[0]); i++) {
10         std::cout << *p << std::endl; //每次打印指针指向的值
11         p++; // 指针偏移 4 个字节
12     }
13
14     return 0;
15 }
```

5. 指针和函数

函数指针

- 函数的首地址
- 函数的参数、返回值类型

```
int f(int x) {
    return x + 1;
}

void Demo() {
    int (*fp)(int) = &f;
    fp = f;
    int ret = (*fp)(3);
    ret = fp(3);
}
```

- 函数类型会隐式转换为函数指针类型
- 解函数指针后仍然是函数指针类型



6. 指针与引用的区别

指针是一个存储地址的变量，可以指向不同的对象，也可以为空；引用是某个对象的别名，定义时必须初始化且之后不可修改，始终引用同一个对象，不能再引用其他对象。

特性	指针	引用
声明	T* ptr	T& ref = var
初始化	可以延迟	必须立即初始化
重新绑定	可以	不可以
空值	支持	不支持
内存占用	有自己地址	无额外内存
解引用	需要 * 运算符	自动解引用
数组访问	支持指针算术	需要特殊语法
多级间接	支持	不支持

使用指针的场景：

1. 需要处理动态内存分配
2. 需要支持空值（可选参数）

3. 需要重新指向不同对象
4. 需要指针算术运算
5. 实现数据结构（如链表、树）

使用引用的场景：

1. 函数参数传递（避免拷贝，修改原值）
2. 函数返回值（避免拷贝）
3. 运算符重载
4. 范围for循环
5. 需要简洁的语法，避免指针的复杂操作

智能指针（重要）

智能指针是一个封装普通指针的工具类，自动管理对象的内存资源。

1. unique_ptr（独享所有权）

与对象的关系是一一对应的，一个unique_ptr只能指向一个对象。在离开作用域后会自动销毁，unique_ptr不能被拷贝和赋值，只能通过std::move转移所有权。

为什么不能被拷贝：因为拷贝构造函数和拷贝构造运算符被删除了

```
unique_ptr(const _Myt&) = delete;
_Myt& operator=(const _Myt&) = delete;
```

2. shared_ptr（共享所有权）

可以有多个shared_ptr指向同一个对象，通过引用计数管理内存释放。当一个新的shared_ptr指向资源时，引用计数+1，当销毁一个shared_ptr时，引用计数-1，当引用计数为0时会将所指向的内存对象释放。

shared_ptr 线程安全性问题（轻舟智航）：

- **引用计数是线程安全的：**引用计数更新是原子操作，当创建、复制或销毁 shared_ptr 时，底层的引用计数会通过原子操作来更新。这确保了在多线程环境下，不同线程对同一个 shared_ptr 的操作不会导致数据竞争。
- **对象本身的访问不是线程安全的：**虽然 shared_ptr 的引用计数操作是线程安全的，但实际指向的对象的访问并不是线程安全的。如果多个线程同时访问同一个对象（通过 shared_ptr），那么必须使用其他同步机制（如互斥锁）来保护对该对象的访问。

- **最好避免同时操作同一个 `shared_ptr`**：尽管引用计数是线程安全的，但如果多个线程同时对同一个 `shared_ptr` 进行读写操作（如赋值、重置等），仍然可能会引发未定义行为。因此，尽量在一个线程中管理 `shared_ptr`，并通过适当的设计模式来传递指针到其他线程。

`shared_ptr` 中创建指针有 `new` 和 `make_shared` 两种方法，那么这两种方法有什么区别

1. 内存分配次数不同

`make_shared`：一次内存分配

- 将控制块（引用计数等）和对象本身分配在同一块连续内存中

`new`：两次内存分配

- 第一次：`new`分配对象内存
- 第二次：`shared_ptr`构造函数内部分配控制块内存

代码块

```
1  // 一次内存分配
2  auto p1 = std::make_shared<int>(42);
3
4  // 两次内存分配
5  std::shared_ptr<int> p2(new int(42));
```

2. 异常安全性

`make_shared`：更安全

代码块

```
1  // 安全：make_shared 是原子操作
2  function(std::shared_ptr<Widget>(new Widget()), computeSomething());
```

`new`：存在潜在的内存泄漏风险

代码块

```
1  // 不安全：可能的执行顺序
2  // 1. new Widget()
3  // 2. computeSomething() <- 如果这里抛异常
4  // 3. shared_ptr 构造函数
5  // 结果：new 出的 Widget 泄漏
```

```
6 function(std::shared_ptr<Widget>(new Widget()), computeSomething());
```

3. 自定义删除器

new：支持自定义删除器

代码块

```
1 auto deleter = [](int* p) { delete p; };
2 std::shared_ptr<int> p(new int(42), deleter);
```

make_shared：不支持自定义删除器

代码块

```
1 // 编译错误：make_shared 不支持自定义删除器
2 auto p = std::make_shared<int>(42, myDeleter); // ❌
```

4. 内存释放时机

make_shared：可能延迟内存释放

- 由于对象和控制块在同一内存块中，只有当引用计数和弱引用计数都变为0时才能释放
- 即使对象已销毁，只要还有weak_ptr存在，内存就不会释放

new：更及时的释放

- 对象和控制块分离
- 对象可以在引用计数为0时立即释放（即使还有weak_ptr）

代码块

```
1 // make_shared 示例
2 std::weak_ptr<int> wp;
3 {
4     auto sp = std::make_shared<int>(42);
5     wp = sp;
6     // sp 离开作用域，对象被析构
7     // 但内存块不会释放，因为 wp 还存在
8 } // 内存仍然被占用（包含控制块）
9
10 // new 示例
11 std::weak_ptr<int> wp;
```

```
12  {
13      std::shared_ptr<int> sp(new int(42));
14      wp = sp;
15      // sp 离开作用域，对象立即析构并释放内存
16      // 只有控制块保留
17  } // 对象内存已释放，只有控制块占用少量内存
```

性能对比

方面	make_shared	new
内存分配次数	1次	2次
内存局部性	好（数据连续）	差（分散）
内存占用	稍大（整体分配）	稍小（控制块独立）
弱引用开销	高（整个内存块存活）	低（仅控制块存活）

优先使用 make_shared：

-  大多数常规场景
-  对性能敏感、频繁创建共享指针的代码
-  不需要自定义删除器时
-  没有大量weak_ptr长期持有的场景

使用 new 的情况：

-  需要自定义删除器
-  有大量weak_ptr且对象内存较大（避免延迟释放）
-  需要控制对象的构造方式（如使用placement new）
-  与遗留代码交互时

示例：选择指南

代码块

```

1 // 好: 常规使用
2 auto sp1 = std::make_shared<MyClass>(arg1, arg2);
3
4 // 好: 需要自定义删除器
5 std::shared_ptr<FILE> sp2(fopen("test.txt", "r"), fclose);
6
7 // 好: 大对象 + 大量 weak_ptr
8 // 避免 make_shared 导致内存延迟释放
9 std::shared_ptr<HugeData> sp3(new HugeData());
10
11 // 差: 不要混合使用
12 int* raw = new int(42);
13 std::shared_ptr<int> sp4(raw); // 勉强可以, 但不推荐
14 std::shared_ptr<int> sp5(raw); // 错误! 双重删除

```

总结: 除非有特殊需求 (自定义删除器或内存延迟问题), 否则优先使用 `std::make_shared`。

shared_ptr中引用计数是线程安全的么?

`shared_ptr` 的引用计数操作本身是线程安全的, 但指向的对象不是自动线程安全的。

核心结论

✅ 线程安全的部分:

- 引用计数的增加 (拷贝构造、赋值)
- 引用计数的减少 (析构、重置)

这些操作是原子操作, 不会有数据竞争

❌ 不是线程安全的部分:

- 对指向对象的访问 (读/写)
- 两个线程同时修改同一 `shared_ptr` 对象本身 (不是指向的对象)

详细说明

1. 引用计数操作是线程安全的

代码块

```

1 std::shared_ptr<int> sp = std::make_shared<int>(42);
2
3 // 线程1: 拷贝 sp
4 std::shared_ptr<int> sp1 = sp; // ✅ 引用计数原子 +1

```

```
5
6 // 线程2: 拷贝 sp
7 std::shared_ptr<int> sp2 = sp; //  引用计数原子 +1
8
9 // 结果: 引用计数从1变成3, 线程安全
```

2. 指向的对象不是线程安全的

代码块

```
1 std::shared_ptr<Counter> sp = std::make_shared<Counter>();
2
3 // 线程1
4 sp->value++; //  不是线程安全的!
5
6 // 线程2
7 sp->value++; //  数据竞争!
8
9 // 解决方案: 需要自己加锁
10 std::mutex mtx;
11 // 线程1
12 {
13     std::lock_guard<std::mutex> lock(mtx);
14     sp->value++;
15 }
```

3. 对 shared_ptr 对象本身的修改不是线程安全的

代码块

```
1 std::shared_ptr<int> sp = std::make_shared<int>(42);
2
3 // 线程1
4 sp = std::make_shared<int>(100); //  与线程2的 reset 竞争
5
6 // 线程2
7 sp.reset(); //  与线程1的赋值竞争
8
9 // 结果: 未定义行为!
```

常见场景分析

场景1：多个线程拷贝同一个 shared_ptr

代码块

```
1  std::shared_ptr<Data> global_sp = std::make_shared<Data>();
2
3  void thread_func() {
4      // 多个线程同时拷贝是安全的
5      std::shared_ptr<Data> local_sp = global_sp; //  安全
6      local_sp->process(); //  但访问对象需要同步
7  }
```

场景2：修改控制块的 shared_ptr 操作

代码块

```
1  std::shared_ptr<int> sp;
2
3  void thread1() {
4      sp = std::make_shared<int>(10); //  与 thread2 竞争
5  }
6
7  void thread2() {
8      sp = std::make_shared<int>(20); //  与 thread1 竞争
9  }
10
11 // 解决方案：加锁保护 sp 本身
12 std::mutex sp_mutex;
13 void thread1_safe() {
14     std::lock_guard<std::mutex> lock(sp_mutex);
15     sp = std::make_shared<int>(10);
16 }
```

场景3：析构是安全的

代码块

```
1  std::shared_ptr<int> sp = std::make_shared<int>(42);
2
3  void thread_func() {
4      std::shared_ptr<int> local = sp; // 引用计数+1
```

```
5 } // local 销毁, 引用计数-1, 线程安全
6
7 // 最后一个 shared_ptr 销毁时, 对象被安全删除
```

实用建议和模式

模式1: 传递 shared_ptr 到线程

代码块

```
1 void thread_func(std::shared_ptr<Data> sp) {
2     // 每个线程有自己的 shared_ptr 副本
3     // 引用计数增加是安全的
4     sp->doWork(); // 但如果多个线程都调用, 仍需要同步
5 }
6
7 // 使用
8 auto sp = std::make_shared<Data>();
9 std::thread t1(thread_func, sp); // 拷贝 sp, 引用计数+1
10 std::thread t2(thread_func, sp); // 拷贝 sp, 引用计数+1
11 t1.join();
12 t2.join();
13 // 引用计数正确管理
```

模式2: 保护指向的对象

代码块

```
1 class SharedData {
2     std::mutex mtx;
3     int value;
4 public:
5     void increment() {
6         std::lock_guard<std::mutex> lock(mtx);
7         value++;
8     }
9 };
10
11 auto sp = std::make_shared<SharedData>();
12 // 多个线程安全地调用
13 std::thread t1(&SharedData::increment, sp);
14 std::thread t2(&SharedData::increment, sp);
```

模式3：原子操作 shared_ptr (C++20)

C++20 提供了 `std::atomic<std::shared_ptr<T>>`:

代码块

```
1  std::atomic<std::shared_ptr<int>> atomic_sp;
2
3  void thread1() {
4      auto new_sp = std::make_shared<int>(100);
5      atomic_sp.store(new_sp); // 原子操作
6  }
7
8  void thread2() {
9      auto sp = atomic_sp.load(); // 原子操作
10     if (sp) {
11         // 使用 sp
12     }
13 }
```

常见误区总结

操作 是否线程安全 说明

拷贝 shared_ptr  安全 引用计数原子操作

销毁 shared_ptr  安全 引用计数原子减

访问指向的对象  不安全 需要额外同步

修改同一个 shared_ptr 对象  不安全 需要加锁或使用 atomic

reset() 操作  不安全 修改共享状态

从同一个 shared_ptr 创建 weak_ptr  安全 控制块操作是原子的

最佳实践

1. 多个线程可以安全地拷贝同一个 shared_ptr
2. 访问对象内容时必须自己加锁

3. 不要在线程中不加保护地修改同一个 `shared_ptr` 对象
4. 如果需要在多线程中修改 `shared_ptr` 本身，使用 `std::atomic<std::shared_ptr>>` (C++20) 或互斥锁
5. `make_shared` 创建的对象析构是线程安全的

一句话总结：引用计数本身是线程安全的，但不要混淆——这不等于 `shared_ptr` 或它指向的对象是线程安全的！

3. weak_ptr (弱引用)

解决循环引用，不拥有资源所有权，不会增加引用计数，仅观察一个由 `shared_ptr` 管理的资源。

循环引用如下：

代码块

```
1  class A {
2  public:
3      shared_ptr<B> b_ptr;
4      A() { cout << "A 构造函数" << endl; }
5      ~A() { cout << "A 析构函数" << endl; }
6  };
7
8  class B {
9  public:
10     shared_ptr<A> a_ptr;
11     B() { cout << "B 构造函数" << endl; }
12     ~B() { cout << "B 析构函数" << endl; }
13 };
14
15 int main() {
16     cout << "=== 创建循环引用 ===" << endl;
17     {
18         shared_ptr<A> a = make_shared<A>();
19         shared_ptr<B> b = make_shared<B>();
20
21         // 创建循环引用
22         a->b_ptr = b;
23         b->a_ptr = a;
24
25         cout << "A 引用计数: " << a.use_count() << endl; // 输出 2
26         cout << "B 引用计数: " << b.use_count() << endl; // 输出 2
27     }
28     // 离开作用域，但由于循环引用，引用计数不会降到0
```

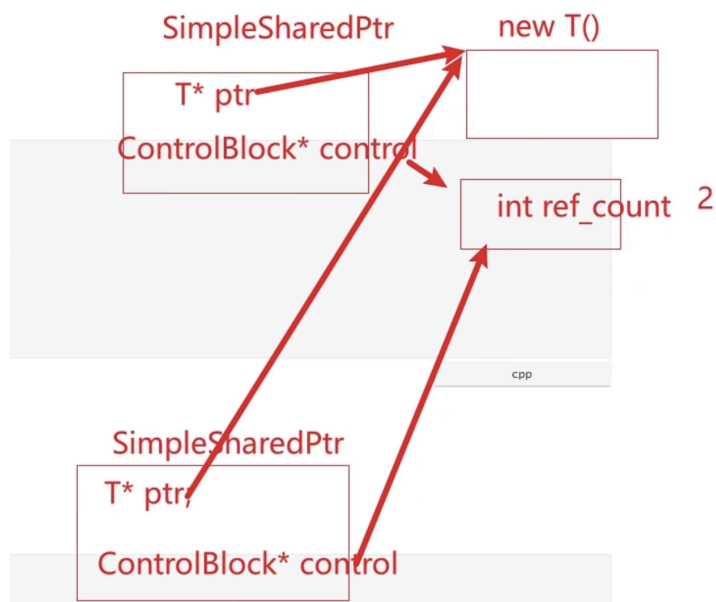
```

29     cout << "程序结束，但没有调用析构函数！" << endl;
30
31     return 0;
32 }
33 // === 创建循环引用 ===
34 // A 构造函数
35 // B 构造函数
36 // A 引用计数：2
37 // B 引用计数：2
38 // 程序结束，但没有调用析构函数！

```



手动实现智能指针



代码块

```

1  #include <iostream>
2
3  template <typename T>
4  class SharedPtr {
5  private:
6      T *ptr;          // 原始指针，指向对象
7      int *ref_count;  // 引用计数指针
8      // 减少引用计数，在引用计数为0时
9      void DecreaseCount() {
10         if (*ref_count) {
11             (*ref_count)--;
12             // 当没有shared_ptr指向对象时，释放资源
13         } if ((*ref_count) == 0) {
14             delete ptr;
15             delete ref_count;

```

```

16         ptr = nullptr;
17         ref_count = nullptr;
18     }
19 }
20 }
21
22 public:
23     // 默认构造函数-创建一个空的shared_ptr
24     SharedPtr() : ptr(nullptr), ref_count(nullptr) {}
25
26     // 显示构造函数-从原始指针创建shared_ptr
27     explicit SharedPtr(T *p) : ptr(p), ref_count(new int(1)) {
28         std::cout << "创建shared_ptr,计数: 1" << std::endl;
29     }
30
31     // 拷贝构造-增加引用计数
32     SharedPtr(const SharedPtr<T> &other) {
33         ptr = other.ptr;
34         ref_count = other.ref_count;
35         if (ref_count) {
36             (*ref_count)++;
37             std::cout << "拷贝构造, 引用计数: " << *ref_count << std::endl;
38         }
39     }
40
41     // 移动构造-不增加引用计数, 转移所有权
42     SharedPtr(SharedPtr<T> &&other) noexcept {
43         ptr = other.ptr;
44         ref_count = other.ref_count;
45         // 移动语义, 将原指针置空, 避免重复释放
46         other.ptr = nullptr;
47         other.ref_count = nullptr;
48         std::cout << "移动构造, 所有权转移" << std::endl;
49     }
50     // 析构函数-减少引用计数, 引用计数为0时释放资源
51     ~SharedPtr() {
52         DecreaseCount();
53         std::cout << "析构函数" << std::endl;
54     }
55
56     // 拷贝赋值运算符
57     SharedPtr<T> &operator=(const SharedPtr<T> &other) {
58         // 如果当前对象与other对象不是一个对象
59         if (this != &other) {
60             // 先减少当前对象的引用计数, 释放当前对象对资源的引用
61             DecreaseCount();
62

```

```

63         // 复制指针和计数
64         ptr = other.ptr;
65         ref_count = other.ref_count;
66
67         // 增加引用计数, 如果指向的对象不空
68         if (ref_count) {
69             (*ref_count)++;
70             std::cout << "拷贝赋值, 计数: " << *ref_count << std::endl;
71         }
72     }
73     // 返回当前对象, 支持链式赋值
74     return *this;
75 }
76
77 // 移动赋值运算符
78 SharedPtr<T> &operator=(SharedPtr<T> &&other) noexcept {
79     if (this != &other) {
80         // 先减少当前对象的引用计数
81         DecreaseCount();
82
83         // 接管资源
84         ptr = other.ptr;
85         ref_count = other.ref_count;
86
87         // 将原指针置空
88         other.ptr = nullptr;
89         other.ref_count = nullptr;
90         std::cout << "移动赋值, 所有权转移 " << std::endl;
91     }
92     return *this;
93 }
94 // 解引用运算符-获取指向的对象, const不能修改成员变量
95 T &operator*() const { return *ptr; }
96
97 // 成员访问运算符 - 访问对象的成员
98 T *operator->() { return ptr; }
99
100 // 获取原始指针
101 T *Get() const { return ptr; }
102
103 // 获取引用计数
104 int GetRefCount() { return ref_count ? *ref_count : 0; }
105
106 // 重置智能指针
107 void reset() {
108     DecreaseCount();
109     ptr = nullptr;

```

```

110     ref_count = nullptr;
111 }
112 };
113
114 class Example {
115 public:
116     Example() { std::cout << "Example对象创建" << std::endl; }
117     ~Example() { std::cout << "Example对象销毁" << std::endl; }
118     void test() { std::cout << "Example::test()调用" << std::endl; }
119 };
120
121 int main() {
122     std::cout << "=== 创建第一个shared_ptr ===" << std::endl;
123     SharedPtr<Example> ptr1(new (Example));
124     std::cout << "引用计数: " << ptr1.GetRefCount() << std::endl;
125     std::cout << "\n=== 创建第二个shared_ptr(拷贝构造) ===" << std::endl;
126     SharedPtr<Example> ptr2 = ptr1;
127     std::cout << "ptr1引用计数: " << ptr1.GetRefCount() << std::endl;
128     std::cout << "ptr2引用计数: " << ptr2.GetRefCount() << std::endl;
129     std::cout << "\n=== 创建第三个shared_ptr(移动构造) ===" << std::endl;
130     SharedPtr<Example> ptr3 = std::move(ptr1);
131     std::cout << "ptr2引用计数: " << ptr2.GetRefCount() << std::endl;
132     std::cout << "ptr3引用计数: " << ptr3.GetRefCount() << std::endl;
133     std::cout << "\n=== 使用对象 ===" << std::endl;
134     ptr2->test();
135     (*ptr3).test();
136
137     std::cout << "\n=== 重置ptr2 ===" << std::endl;
138     ptr2.reset();
139     std::cout << "ptr3引用计数: " << ptr3.GetRefCount() << std::endl;
140
141     std::cout << "\n=== 退出作用域, 自动释放资源 ===" << std::endl;
142 }

```

STL (重点)

容器、算法、迭代器、仿函数、适配器、空间配置器

1. vector 的使用场景

vector 的本质是动态数组，拥有三个关键特性：

1. 连续内存：元素在内存中紧密排列

2. O(1) 随机访问：通过索引直接访问

3. 尾部高效操作：尾部插入/删除均摊 O(1)

场景 1：需要频繁随机访问元素

代码块

```
1 // 场景：图像处理、矩阵运算、游戏中的对象池
2 std::vector<Pixel> image(1920 * 1080);
3 Pixel p = image[y * width + x]; // O(1) 随机访问
4
5 // 场景：数据分析和科学计算
6 std::vector<double> data(1000000);
7 double sum = 0;
8 for (size_t i = 0; i < data.size(); ++i) { // 顺序访问也极快
9     sum += data[i];
10 }
```

场景 2：元素数量相对稳定或可预测

代码块

```
1 // 场景：读取文件中的所有行（已知总数或可预估）
2 std::vector<std::string> lines;
3 lines.reserve(estimated_line_count); // 提前预留，避免重新分配
4 // 读取并追加...
5
6 // 场景：配置参数列表（大小固定）
7 std::vector<ConfigEntry> configs = {
8     {"timeout", 30},
9     {"retries", 3}
10 };
```

场景 3：主要在尾部进行插入/删除

代码块

```
1 // 场景：实现栈
2 std::vector<int> stack;
3 stack.push_back(1); // 压栈
4 stack.push_back(2);
5 int top = stack.back(); // 查看栈顶
6 stack.pop_back(); // 弹栈
7
8 // 场景：动态收集数据后统一处理
9 std::vector<Event> events;
```

```
10 events.push_back(parse_event()); // 收集事件
11 // ... 处理所有 events
```

场景 4：数据需要频繁访问（顺序遍历）

代码块

```
1 // 场景：遍历所有元素进行计算
2 std::vector<int> scores(1000000);
3 int total = 0;
4 for (int s : scores) { // 顺序访问，缓存命中率极高
5     total += s;
6 }
7
8 // 场景：序列化/反序列化
9 std::vector<char> serialized_data;
10 // 连续内存可直接写入文件或网络
11 write(fd, serialized_data.data(), serialized_data.size());
```

总结：

代码块

```
1 开始
2  |
3  |— 需要频繁随机访问? ——— 是 —→ 考虑 vector
4  |   |
5  |   |— 否
6  |
7  |— 主要在尾部操作? ——— 是 —→ 考虑 vector (可配合 reserve)
8  |   |
9  |   |— 否
10 |
11 |— 需要在头部/中间频繁插入删除?
12 |   |
13 |   |— 是 —→ ❌ 避免 vector, 考虑 deque 或 list
14 |   |
15 |   |— 否
16 |
17 |— 元素数量是否已知或可预测?
18 |   |
19 |   |— 是 —→ ✅ vector + reserve
20 |   |
21 |   |— 否 —→ 谨慎使用 vector, 或考虑 deque
22 |
23 |— 需要与 C API 交互? — 是 —→ ✅ vector (data() 方法)
```

24 |
25 └─ 需要稳定的迭代器? —— 是 —→ ❌ 避免 vector, 用 list

2. sizeof(std::vector<T>)的大小是多少

`sizeof(std::vector<T>)` 是一个编译时常量, 它返回的是 `vector` 对象本身的大小 (即栈上该对象占用的字节数), 与 `vector` 中存储了多少个元素无关 (堆上动态分配的内存不计入)。

通常情况下, 在 32 位系统中是 12 字节, 在 64 位系统中是 24 字节 (不同标准库实现可能略有差异)。

`vector` 对象内部通常包含三个指针 (或指针大小的成员) :

- `_M_start` : 指向数据起始位置
- `_M_finish` : 指向数据末尾
- `_M_end_of_storage` : 指向当前分配容量的末尾

代码块

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> v1;
6      std::vector<double> v2(1000); // 有 1000 个元素
7      std::vector<char> v3;
8
9      std::cout << sizeof(v1) << std::endl; // 24 (64位)
10     std::cout << sizeof(v2) << std::endl; // 24 (与v1相同)
11     std::cout << sizeof(v3) << std::endl; // 24 (类型不同,但大小相同)
12
13     // 验证内存布局
14     std::cout << sizeof(void*) << std::endl; // 8 (64位下指针大小)
15     // 3个指针 = 24字节
16 }
```

3. vector 的 resize 和 reverse 的区别

区别:

核心区别一览表

特性	resize(n)	reserve(n)
主要目的	改变元素数量 (size)	预留内存空间 (capacity)
是否创建元素	是，会构造/析构元素	否，只分配内存
是否可访问新空间	是，新元素可立即访问	否，需先添加元素
影响size()	会改变size	不会改变size
影响capacity()	可能改变 (如果n>当前capacity)	可能改变 (如果n>当前capacity)
参数默认值	有 (默认构造值)	无

1. reserve(size_type n)-预留内存

预先分配足够的内存空间，避免后续插入元素时多次重新分配。size() 不变，capacity() 至少变为 n。就像一个教室 capacity 是能坐60 个人，但现在只有 3 个人坐在教室（即 size()）。

代码块

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> vec;
6
7      std::cout << "初始状态:" << std::endl;
8      std::cout << "size: " << vec.size() << ", capacity: " << vec.capacity() <<
std::endl;
9
10     // reserve只分配内存，不创建元素
11     vec.reserve(100);
12
13     std::cout << "\nreserve(100)后:" << std::endl;
14     std::cout << "size: " << vec.size() << ", capacity: " << vec.capacity() <<
std::endl;
15
16     // 访问预留空间是未定义行为!
17     // vec[0] = 1; // 错误! size仍然是0，不能访问
18
19     // 必须先添加元素
20     vec.push_back(10); // 现在可以，不会引起重新分配
21     std::cout << "push_back后 size: " << vec.size() << std::endl;
22 }
```

```
23     return 0;
24 }
```

2. `resize(size_type n)` - 改变大小

改变向量中的元素数量

代码块

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> vec = {1, 2, 3};
6
7      std::cout << "初始: ";
8      for (int x : vec) std::cout << x << " ";
9      std::cout << "| size: " << vec.size() << std::endl;
10
11     // 1. 缩小size
12     vec.resize(2);
13     std::cout << "resize(2): ";
14     for (int x : vec) std::cout << x << " ";
15     std::cout << "| size: " << vec.size() << std::endl;
16
17     // 2. 增大size - 默认初始化
18     vec.resize(5);
19     std::cout << "resize(5): ";
20     for (int x : vec) std::cout << x << " ";
21     std::cout << "| size: " << vec.size() << std::endl;
22
23     // 3. 增大size - 指定初始化值
24     vec.resize(8, 99);
25     std::cout << "resize(8, 99): ";
26     for (int x : vec) std::cout << x << " ";
27     std::cout << "| size: " << vec.size() << std::endl;
28
29     return 0;
30 }
```

何时使用 `reserve()`?

代码块

```
1  // 场景1: 已知要插入大量元素, 避免多次重新分配
2  std::vector<int> processData() {
```

```

3     std::vector<int> result;
4     // 提前知道大概需要10000个元素
5     result.reserve(10000);
6     for (int i = 0; i < 10000; ++i) {
7         // 这10000次push_back不会引起重新分配
8         result.push_back(calculateValue(i));
9     }
10    return result;
11 }
12
13 // 场景2: 优化性能 - 减少内存碎片和拷贝开销
14 void optimizePerformance() {
15     std::vector<std::string> strings;
16     // 如果不reserve, 每次capacity不够时都要:
17     // 1. 分配新内存 2. 拷贝所有元素 3. 释放旧内存
18     strings.reserve(1000);
19     for (int i = 0; i < 1000; ++i) {
20         strings.push_back("string_" + std::to_string(i));
21     }
22 }

```

何时使用resize()?

代码块

```

1 // 场景1: 需要固定大小的数组式访问
2 void matrixOperation() {
3     std::vector<std::vector<int>> matrix(10);
4     // 创建10x10的矩阵
5     for (auto& row : matrix) {
6         row.resize(10, 0); // 每行10个元素, 初始化为0
7     }
8     // 可以直接通过下标访问
9     matrix[3][5] = 42;
10 }
11
12 // 场景2: 预填充默认值
13 void initializeWithDefaults() {
14     // 创建100个元素, 全部初始化为-1
15     std::vector<int> buffer(100, -1); // 构造函数也是resize
16     // 或者之后改变大小
17     buffer.resize(200, -1); // 新增的100个也是-1
18 }

```

4. vector 的push_back 与 emplace_back 的区别

1. push_back() - 加入已存在的对象。接受一个已构造好的对象，将其拷贝或移动到容器末尾

代码块

```
1  class Person {
2  public:
3      Person(const std::string& name, int age)
4          : name(name), age(age) {
5          std::cout << "构造: " << name << std::endl;
6      }
7
8      // 拷贝构造函数
9      Person(const Person& other)
10         : name(other.name), age(other.age) {
11         std::cout << "拷贝构造: " << name << std::endl;
12     }
13
14     // 移动构造函数
15     Person(Person&& other) noexcept
16         : name(std::move(other.name)), age(other.age) {
17         std::cout << "移动构造: " << name << std::endl;
18     }
19
20 private:
21     std::string name;
22     int age;
23 };
24
25 int main() {
26     std::vector<Person> people;
27     std::cout << "=== push_back 示例 ===" << std::endl;
28
29     // 情况1: 传递临时对象 (右值)
30     std::cout << "\n1. push_back临时对象:" << std::endl;
31     people.push_back(Person("Alice", 25));
32     // 输出:
33     // 构造: Alice          <- 临时对象构造
34     // 移动构造: Alice      <- 移动到vector中
35
36     // 情况2: 传递已存在的对象 (左值)
37     std::cout << "\n2. push_back已存在对象:" << std::endl;
38     Person bob("Bob", 30);
39     people.push_back(bob);
40     // 输出:
41     // 构造: Bob            <- bob对象构造
42     // 拷贝构造: Bob        <- 拷贝到vector中
43 }
```

```
44     return 0;
45 }
```

2. `emplace_back()` - 直接构造对象。在容器尾部直接构造对象，无需先创建临时对象

代码块

```
1  int main() {
2      std::vector<Person> people;
3
4      std::cout << "\n=== emplace_back 示例 ===" << std::endl;
5
6      // 直接传递构造函数的参数
7      std::cout << "\n1. emplace_back构造参数:" << std::endl;
8      people.emplace_back("Charlie", 35);
9      // 输出:
10     // 构造: Charlie      <- 直接在vector内存中构造
11
12     std::cout << "\n2. emplace_back复杂构造:" << std::endl;
13     std::string name = "David";
14     people.emplace_back(name + " Smith", 40);
15     // 输出:
16     // 构造: David Smith   <- 直接在vector内存中构造
17
18     return 0;
19 }
```

使用黄金法则：

1. 传递构造函数的参数时 → 优先使用 `emplace_back`
2. 已经有一个对象要添加时 → 使用 `push_back` （可读性更好）
3. 对象需要移动时 → 两者都可以，但 `push_back(std::move(x))` 更明确
4. 当数据不大时两者基本无差异，但是当数据为结构体等大量数据时优先使用 `emplace_back`

5. vector 扩容机制

扩容机制：

1. 当vector的size（当前元素数量）即将超过capacity（当前容量）时，就会触发扩容。
2. 扩容通常会导致重新分配内存，并将原有元素拷贝到新内存中。释放旧内存，插入新元素。这个过程可能会很耗时，因为涉及到元素的拷贝（或移动，如果元素类型支持移动语义且vector使用移动操作）。

3. 扩容策略因编译器实现而异，但常见的策略是每次扩容将容量增加为原来的两倍（或1.5倍，取决于实现）。例如，在MSVC中，通常每次扩容为原来的1.5倍；在GCC中，通常是2倍。

注意：扩容操作会使指向vector元素的迭代器、指针和引用失效，因为元素可能已经被移动到新的内存位置。

代码块

```
1  std::vector<int> vec; // 初始容量为0
2  vec.push_back(1);    // 第一次分配内存
3  // 当空间不足时，vector会：
4  // 1. 申请更大的内存块（通常是当前容量的倍数）
5  // 2. 将旧元素复制/移动到新内存
6  // 3. 释放旧内存
7  // 4. 插入新元素
```

避免扩容：

代码块

```
1  // 1. 使用emplace_back（减少拷贝）
2  std::vector<std::string> vec;
3  vec.reserve(10);
4  vec.emplace_back("Hello"); // 直接在vector中构造
5
6  // 2. 批量插入
7  std::vector<int> source = {1, 2, 3, 4, 5};
8  std::vector<int> target;
9  target.reserve(source.size());
10 target.insert(target.end(), source.begin(), source.end());
11
12 // 3. 使用移动语义
13 std::vector<std::string> getStrings() {
14     std::vector<std::string> temp;
15     // ... 填充数据
16     return temp; // RVO或移动，不会复制
17 }
```

6. C++中什么时候优先使用 vector，什么时候优先使用 list

特性	vector	list
内存布局	连续内存	非连续内存（节点链接）
随机访问	✅ $O(1)$	❌ $O(n)$
尾部插入	✅ $O(1)$ 均摊	✅ $O(1)$
中间插入	❌ $O(n)$	✅ $O(1)$
头部插入	❌ $O(n)$	✅ $O(1)$
缓存友好性	✅ 很好	❌ 差
内存开销	小（仅容量）	大（每个节点有2个指针）
迭代器类型	随机访问	双向迭代器

优先使用 vector：

1. 需要频繁随机访问元素

代码块

```
1 // vector:  $O(1)$  随机访问
2 vector<int> vec = {1, 2, 3, 4, 5};
3 cout << vec[3]; //  $O(1)$  - 直接索引访问
4 cout << vec.at(2); //  $O(1)$  - 带边界检查
5
6 // list: 只能顺序访问, 不支持随机访问
7 list<int> lst = {1, 2, 3, 4, 5};
8 // cout << lst[3]; // 错误! 不支持[]
9 auto it = lst.begin();
10 advance(it, 3); //  $O(n)$  - 需要遍历
11 cout << *it;
```

2. 主要进行尾部插入

代码块

```
1 // vector的尾部操作非常高效
2 vector<int> vec;
3 vec.push_back(10); //  $O(1)$  均摊
4 vec.pop_back(); //  $O(1)$ 
5
6 // 尾部批量添加
```

```
7  vec.insert(vec.end(), {1, 2, 3, 4}); // 相对高效
```

3. 对缓存性能要求高

代码块

```
1  // vector的内存连续性对CPU缓存友好
2  vector<int> vec(1000);
3  // 顺序访问：高缓存命中率，性能好
4  int sum = 0;
5  for (int i = 0; i < vec.size(); i++) {
6      sum += vec[i]; // 缓存友好
7  }
8
9  // list的内存分散，缓存命中率低
10 list<int> lst(1000);
11 for (auto& num : lst) { // 每次访问可能缓存未命中
12     sum += num;
13 }
```

4. 节省内存

代码块

```
1  // vector内存使用更紧凑
2  vector<int> vec(1000); // ~4KB (假设int=4字节)
3  // 存储1000个int，只需要4000字节
4
5  // list内存开销大
6  list<int> lst(1000); // ~24KB (假设int=4字节，两个指针各8字节)
7  // 每个节点：int(4) + prev指针(8) + next指针(8) = 20字节
8  // 加上内存对齐和分配开销，实际可能更大
```

优先使用list的场景：

1. 频繁在中间插入/删除

代码块

```
1  // list在中间插入非常高效
2  list<int> lst = {1, 2, 4, 5};
3  auto it = lst.begin();
4  advance(it, 2); // 找到位置
5  lst.insert(it, 3); // O(1) - 在4之前插入3
6
```



```
7 // vector在中间插入需要移动元素
8 vector<int> vec = {1, 2, 4, 5};
9 vec.insert(vec.begin() + 2, 3); // O(n) - 需要移动后续元素
```

7. map 和 unorderedmap 的区别

核心区别概览

特性	map	unordered_map
底层实现	红黑树（平衡二叉搜索树）	哈希表
元素顺序	✅ 按键排序	❌ 无序
查找时间复杂度	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
插入时间复杂度	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
删除时间复杂度	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
迭代器稳定性	✅ 稳定（除删除元素）	❌ 可能因rehash失效
内存使用	较高（树节点开销）	较低（但需负载因子控制）
需要自定义哈希函数	❌ 不需要	✅ 自定义类型需要
需要比较函数	✅ 需要 <code>operator<</code> 或自定义比较	❌ 不需要（需要相等比较）

优先使用 map 场景

- 1. 需要元素有序
- 2. 需要查询范围

代码块

```
1 // 查找年龄在20-30之间的所有学生
2 auto lower = students.lower_bound(20);
3 auto upper = students.upper_bound(30);
```

- 3. 内存充足，数据量不大
- 4. 需要稳定的性能保证（map的 $O(\log n)$ 性能稳定，没有哈希冲突问题）

优先使用unordered_map的场景

1. 需要极速查询
2. 不关心元素顺序
3. 数据量大
4. 自定义类型但难以定义排序



默认建议优先使用 `unordered_map`

8. `std::multiset`

在 C++ 标准库中，`std::multiset` 是一种关联容器，用于存储多个相同的元素，并且会自动对这些元素进行排序。与 `std::set` 不同的是，`std::multiset` 允许存储重复的元素。

主要特点：

1. 自动排序：`std::multiset` 中的元素会根据其值自动排序，默认是升序排列。
2. 允许重复：可以插入多个相同的元素。
3. 高效查找：查找、插入和删除操作的时间复杂度为 $O(\log n)$ ，这得益于底层的红黑树实现。
4. 迭代器支持：可以使用迭代器遍历容器中的元素。

9. STL 中的迭代器，什么时候迭代器会失效

提供了一种方法来顺序访问容器中的元素，而无需暴露容器的内部表示。迭代器类似于指针，它指向容器中的某个元素，并且可以通过递增、递减等操作来移动。主要作用如下：

1. 统一的访问接口：迭代器为所有 STL 容器提供了统一的访问方式，无论底层数据结构如何（数组、链表、树等），都可以用相同的方式遍历和访问元素。
2. STL 算法通过迭代器操作容器，而不需要知道容器的具体类型。这种设计实现了算法与数据结构的解耦。
3. 范围定义：迭代器可以定义操作的区间范围（如 `begin()` 到 `end()`），支持部分操作。
4. 类型安全：迭代器提供了类型检查，减少了运行时错误。

代码块

```
1 // 五类迭代器（从弱到强）：
2 1. 输入迭代器（Input Iterator）：只读，只能单次遍历
3 2. 输出迭代器（Output Iterator）：只写，只能单次遍历
4 3. 前向迭代器（Forward Iterator）：可读写，可多次遍历
5 4. 双向迭代器（Bidirectional Iterator）：可前后移动
6 5. 随机访问迭代器（Random Access Iterator）：支持跳跃访问
```

vector 和 deque- 随机访问迭代器

代码块

```
1  vector<int> vec = {1, 2, 3, 4, 5};
2  vector<int>::iterator it; // 随机访问迭代器// 支持的操作:
3  it = vec.begin();        // 指向第一个元素
4  it += 3;                 // 随机访问: 指向第4个元素 int val = it[2]; // 获取相对
    位置的元素
5  it > vec.begin();        // 支持比较
```

list/set/map- 双向迭代器

代码块

```
1  list<int> lst = {1, 2, 3, 4, 5};
2  list<int>::iterator it;
3
4  // 支持的操作:
5  ++it;    // 前进
6  --it;    // 后退
7  // it += 2; // 错误: 不支持随机访问
8  // it[3];   // 错误: 不支持下标访问
9
10 set<int> s = {1, 3, 5, 2, 4};
11 set<int>::iterator it = s.begin();
12
13 // 注意: set/map的迭代器是const的
14 // *it = 10; // 错误: 不能修改元素值 (对于map只能修改value, 不能修改key)
```

unordered_set/unordered_map - 前向迭代器

代码块

```
1  unordered_set<int> us = {1, 2, 3, 4, 5};
2  // 支持++操作, 但不支持--操作 (单向遍历)
```

迭代器失效

迭代器失效是指在使用迭代器遍历集合（如列表、集合或字典）时，集合的结构发生了变化（例如，添加、删除元素），导致原有的迭代器不再有效，从而引发错误或不可预知的行为

1. 对于序列容器vector, deque来说, 使用erase后, 后边的每个元素的迭代器都会失效, 后边每个元素都往前移动一位, erase返回下一个有效的迭代器。**vector 在重新分配或者移动元素时会导致迭代器失效**, 比如resize (当新大小超过容量时) 和 reverse (当请求容量大于当前容量时) 以及 insert (当容量不足时); 如果插入时容量足够, 不会重新分配, 但插入位置之后的所有元素都会向后移动, 导致: 插入点之后的迭代器全部失效, 插入点之前的迭代器保持有效。
2. 对于关联容器map, set来说, 使用了erase后, 当前元素的迭代器失效, 但是其结构是红黑树, 删除当前元素, 不会影响下一个元素的迭代器, 所以在调用erase之前, 记录下一个元素的迭代器即可。
3. 对于list来说, 它使用了不连续分配的内存, 并且它的erase方法也会返回下一个有效的迭代器, 因此上面两种方法都可以使用。

迭代器和指针的区别

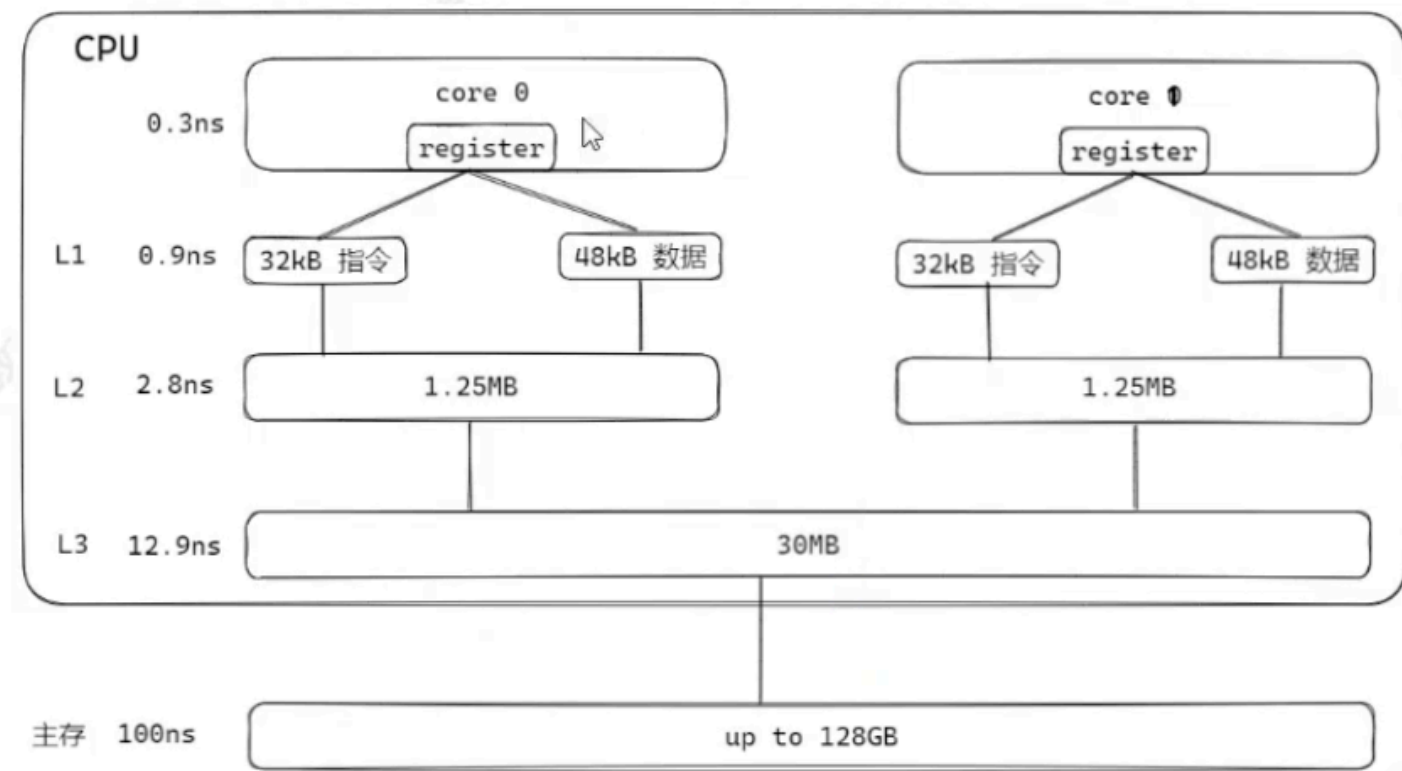
迭代器不是指针, 是类模板, 表现的像指针。他只是模拟了指针的一些功能, 重载了指针的一些操作符, -->、++、--等。迭代器封装了指针, 是一个”可遍历STL (Standard Template Library) 容器内全部或部分元素”的对象, **本质**是封装了原生指针, 是指针概念的一种提升, 提供了比指针更高级的行为, 相当于一种智能指针, 他可以根据不同类型的数据结构来实现不同的++, --等操作。

迭代器返回的是对象引用而不是对象的值, 所以cout只能输出迭代器使用取值后的值而不能直接输出其自身。

volatile 多线程

它告诉编译器: **这个变量的值可能会在程序的控制之外被意外改变**。这主要用于阻止编译器对该变量进行过度优化。

寄存器会将数据先写入缓存中然后再更新到主存中。



作用：

1. **易变的（可见性）**：多个线程对变量i读写，线程1将数据写到了core0的缓存中并没有更新到主存中，线程2从主存中读取变量，这时变量不是最新的就存在问题了。volatile告诉寄存器不要将变量写到缓存中，变量的读写都从主存中获取，确保读数据是最新的
2. **不可优化的：编译器不要做优化**
3. **顺序执行的（volatile变量间有序性）**

使用场景：

4. 多线程共享字段（标志位）且常被修改
5. 中断服务程序和硬件设备访问相关的情况

误区：

1. volatile没有原子性
2. 原子性需要通过原子操作或互斥锁来实现

左值与右值

左值：可寻址的变量（有地址），有持久性。

右值：不可寻址的常量。或在表达式求值过程中创建的无名临时变量，短暂性的。

1. 左值和右值主要区别之一是左值可以被修改，而右值不可以。

 **x++ 右值**，先把x值取出来存在临时变量中，然后再给临时变量+1，最后返回这个临时变量。

++x 左值，自增后把自己返回

2. T&只能绑定到左值；**const T&可以绑定到左值或纯右值**，并延长右值生命周期；**T&&只能绑定到纯右值或将亡值**，用于移动语义和完美转发。

右值引用 T&&：对纯右值/将亡值的引用

std::move()内部实现就是**static_cast<T &&>**，也就是类型转化成右值引用。

绑定将亡值和纯右值：

```
8 String test() {
9     return String("ABC");
10 }
1
2 void Demo() {
3     String s1 = "213";
4     f(s1);
5
6     String s2(std::move(s1));
7
8     // 移动语义本身是软约束
9
10
11 // String &&s2 = std::move(s1); // 将亡值
12 // 移动语义 std::move
13 // f(std::move(s1)); // 如果在f之后，不再使用s1
14 // 右值引用
15
16 String &&s3 = test(); // 右值引用绑定纯右值
```

这里s3是左值，右值引用的本质是左值（普通变量）

左值引用与右值引用的作用

左值引用主要作用：

- 作为函数参数，避免拷贝，提高效率。

```
1 void process(const std::string& str) { // 避免字符串拷贝
2     // ...
3 }
```

- **修改实参的值**

代码块

```
1 void increment(int& x) { // 可以直接修改调用者的变量
2     x++;
3 }
```

右值引用核心作用：

- **实现移动语义**：允许资源从临时对象转移到另一个对象，避免深拷贝，提高性能
- **实现完美转发**：在模板中保持参数的值类别（左值或右值），以便将参数原封不动地传递给其他函数

移动语义与完美转发

移动语义：通过右值引用（`T &&`），将对象的资源从一个临时对象（右值）转移到另一个对象，避免不必要的拷贝，提高性能。

完美转发：在模板函数中使用`std::forward`将参数原封不动地转发给另一个函数，**保留左值或右值属性**。

代码块

```
1 #include <iostream>
2 #include <utility>
3
4 void process(const std::string& s) {
5     std::cout << "处理左值:" << s << std::endl;
6 }
7
8 void process(std::string&& s) { std::cout << "处理右值:" << s << std::endl; }
9
10 template <typename T>
11 void wapper(T&& param) {
12     // 完美转发, 将 param 原封不动地传给 process
13     process(std::forward<T>(param));
14 }
15 int main() {
16     std::string str = "示例";
```

```

17 // 移动语义示例
18 std::string a = "Hello";
19 std::string b = std::move(a); // b 从 a 搬移资源后, a 变为空
20 std::cout << "a = " << a << ", b = " << b << std::endl;
21 // 完美转发示例
22 wapper(str); // 调用左值
23 wapper(std::move(str)); // 调用右值
24 return 0;
25 }

```

struct VS class

struct默认权限是public，常用于纯数据/工具类；class的默认权限是private，常用于面向对象开发

多态/虚函数（重点）

多态是指基于相同的接口（父类或者基类的函数声明），能够表现出不同的行为的能力。多态允许我们在程序运行时根据对象的实际类型来调用适当的方法。

多态形式：

1. 编译时多态（静态多态）：通过**函数重载和运算符重载**实现，它是编译器在编译阶段根据**参数的不同类型**来选择合适的函数。C++通过**名称修饰将参数类型信息编码到函数符号名中**，而C语言只使用**简单的函数名作为符号**，C语言不能实现函数重载
2. 运行时多态（动态多态）：通过继承和虚函数实现。它允许**基类指针或引用指向派生类对象**，并且在运行时决定调用哪一个版本的函数。

代码块

```

1  #include <iostream>
2
3  class Animal {
4  public:
5      virtual void speak() { // 虚函数
6          std::cout << "Animal speaks" << std::endl;
7      }
8  };
9
10 class Dog : public Animal {
11 public:
12     void speak() override { // 重写基类的虚函数

```



```
13     std::cout << "Dog barks" << std::endl;
14 }
15 };
16
17 class Cat : public Animal {
18 public:
19     void speak() override { // 重写基类的虚函数
20         std::cout << "Cat meows" << std::endl;
21     }
22 };
23
24 int main() {
25     Animal* animal1 = new Dog(); // 创建 Dog 对象
26     Animal* animal2 = new Cat(); // 创建 Cat 对象
27
28     animal1->speak(); // 运行时决定调用 Dog 的 speak()
29     animal2->speak(); // 运行时决定调用 Cat 的 speak()
30
31     delete animal1;
32     delete animal2;
33     return 0;
34 }
```

Animal类内部结构

```
class Animal {
public:
    virtual void speak() { // 虚函数
        std::cout << "Animal speaks" << std::endl;
    }
};
```

vfptr

虚函数指针，指向虚函数表

vftable

虚函数表,一维数组，存放所有虚函数的地址

&Animal::speak

Dog类内部结构

```
class Dog : public Animal {
public:
    void speak() override { // 重写基类的虚函数
        std::cout << "Dog barks" << std::endl;
    }
};
```

vfptr

当父类指针或者引用指向子类
发生多态

vftable

Animal* animal1 = new Dog(); // 创建 Dog 对象
animal1->speak(); // 运行时决定调用 Dog 的 speak()

&Animal::speak被替换
&Dog::speak

当子类重写父类虚函数时，子类的虚函数表
内部会替换成子类的虚函数地址

Cat类内部结构

```
class Cat : public Animal {
public:
    void speak() override { // 重写基类的虚函数
        std::cout << "Cat meows" << std::endl;
    }
};
```

vfptr

vftable

&Animal::speak被替换
&Cat::speak

Animal* animal1 = new Dog(); 这里，animal1 是指向 Animal 类型的指针，但它指向的是 Dog 类型的对象。

当调用 animal1->speak() 时，程序会根据 animal1 指向的实际对象（即 Dog）来调用 Dog 中重写的 speak 方法，而不是 Animal 中的 speak 方法。

这种行为称为**动态绑定**，即程序在运行时根据实际的对象类型来决定调用哪个方法。

多态实现原理

每个类中会包含一个**虚函数指针vfptr**，这个**虚函数指针指向虚函数表vftable**。虚函数表是编译器在编译时生成的一维数组，存放该类中所有虚函数地址，虚函数表本身存放在内存的**只读数据段**（编译时生成的静态数据，通常存放在程序的只读数据段）（虚表（vftable）在编译阶段生成，对象内存空间开辟以后，写入对象中的 vfptr，然后调用构造函数）。**在子类重写父类虚函数时，子类的虚函数表内部会替换成子类的虚函数地址，当父类指针或引用指向子类时，就会在子类的虚函数表中寻找相应的虚函数地址并执行。**



虚函数不能用于非成员函数、构造函数、静态成员函数（不属于某个实例化对象）。构造函数不能是虚函数，因为在对象尚未完全构造时无法确定 vfptr 指向哪个虚函数表（vtable），vfptr指向的设置是发生在构造函数体执行之前。

代码块

```
1  class Base {
2  public:
3      Base() {
4          // 在构造函数中, 对象还没有完全构造完成
5      }
6  };
7
8  class Derived : public Base {
9  public:
10     Derived() : Base() {
11         // 基类先构造, 然后才是派生类
12     }
13 };
14
15 // 创建对象时的实际过程:
16 // 1. 分配内存
17 // 2. 调用Base构造函数
18 // 3. 设置vptr指向Base的vtable (如果有虚函数)
19 // 4. 执行Base构造函数体
20 // 5. 调用Derived构造函数
21 // 6. 更新vptr指向Derived的vtable
22 // 7. 执行Derived构造函数体
```

!! 模板函数能不能是虚函数（小马智行）

答：不能。因为模板函数在编译时会生成特定类型的函数实例，而虚函数需要在运行时根据对象的实际类型来决定调用哪个函数；虚函数需要一个虚表（vtable）来支持动态绑定，而模板函数实例化后并不形成虚表

!! C++中可以将内联函数设置为虚函数吗

语法上可以，但实际不会内联

原因：

1. 虚函数需要运行时动态绑定
2. 内联需要在编译期确定具体实现

结果：

1. 编译器会忽略inline关键字
2. 仍通过虚函数表调用

建议：

1. 避免这种写法(没有优化效果)
2. 只有final类的虚函数可能被内联

虚析构的好处

防止内存泄漏。当使用基类指针指向派生类对象，并通过该指针删除对象时，如果基类的析构函数不是虚函数，那么**只会调用基类的析构函数，而不会调用派生类的析构函数**。这可能导致派生类中分配的资源无法正确释放。将基类的析构函数声明为虚函数后，通过基类指针删除派生类对象时，**会先调用派生类的析构函数，再调用基类的析构函数，确保完整地销毁对象**。

代码块

```
1  class Base {
2  public:
3      Base() { std::cout << "Base constructor\n"; }
4      virtual ~Base() { std::cout << "Base destructor\n"; } // 虚析构
5  };
6
7  class Derived : public Base {
8  public:
9      Derived() {
10         data = new int[100];
11         std::cout << "Derived constructor\n";
12     }
13     ~Derived() {
14         delete[] data; // 正确释放内存
15         std::cout << "Derived destructor\n";
16     }
17 private:
18     int* data;
19 };
20
21 int main() {
22     Base* obj = new Derived();
23     delete obj; // 正确调用Derived的析构函数
24     // 输出:
25     // Base constructor
26     // Derived constructor
27     // Derived destructor
28     // Base destructor
29     return 0;
30 }
```

虚继承

为了解决菱形继承带来的子类方法继承二义性问题。

假设有如下继承结构：

代码块

```
1      A
2      / \
3     B  C
4      \ /
5      D
```

- 类 A 是基类，B 和 C 都继承自 A。
- 类 D 同时继承自 B 和 C。

如果类 A 有一个成员变量或方法，而 D 又继承了 B 和 C，那么 D 将会拥有两份来自类 A 的副本，这就造成了数据冗余和二义性。

虚继承通过虚基类实现A类只有一个副本被继承，B 和 C 都是虚继承自 A，因此 A 的构造函数只会被调用一次，而不是像普通继承那样被调用两次。这时只有最底层的派生类（D）会持有 A 的实例，而 B 和 C 不会再拥有 A 的副本，避免了菱形继承带来的问题。

代码块

```
1  // 基类 A
2  class A {
3  public:
4      A() { cout << "A's Constructor" << endl; }
5      void show() { cout << "Class A" << endl; }
6  };
7
8  // 类 B 虚继承自 A
9  class B : virtual public A {
10 public:
11     B() { cout << "B's Constructor" << endl; }
12 };
13
14 // 类 C 虚继承自 A
15 class C : virtual public A {
16 public:
17     C() { cout << "C's Constructor" << endl; }
18 };
19
20 // 类 D 继承自 B 和 C
21 class D : public B, public C {
```

```

22     public:
23         D() { cout << "D's Constructor" << endl; }
24     };
25
26     int main() {
27         D d; // 创建 D 类对象
28         d.show(); // 调用 A 类的 show 方法
29         return 0;
30     }

```

纯虚函数

纯虚函数是在基类中声明但不提供实现的虚函数，语法是在函数后面加 `=0`

代码块

```

1     class Shape { // 抽象基类
2     public:
3         virtual void draw() const = 0; // 纯虚函数
4         virtual ~Shape() {} // 虚析构函数
5     };

```

关键特性

- 抽象类：包含至少一个纯虚函数的类成为抽象类，**不能直接实例化**：虚函数的原理采用 vtable，类中含有纯虚函数时，其 vtable 不完全，有个空位。即“**纯虚函数在类的 vtable 表中对应的表项被赋值为 0。也就是指向一个不存在的函数。由于编译器绝对不允许有调用一个不存在的函数的可能，所以该类不能生成对象。在它的派生类中，除非重写此函数，否则也不能生成对象。**”所以纯虚函数不能实例化。
- 强制实现：派生类必须实现所有纯虚函数才能成为具体类
- 接口定义：纯虚函数定义了类的接口规范，实现与接口分离

✓ 在虚函数和纯虚函数的定义中**不能有 static 标识符**，原因很简单，被 static 修饰的函数在编译时要求前期绑定，然而虚函数却是动态绑定，而且被两者修饰的函数生命周期也不一样

四种类型转换

- `static_cast`: 编译期类型转换, 适用于已知安全转换 (如基类↔派生类指针/引用、数值类型之间) 时使用。

代码块

```
1 struct Base { virtual ~Base() = default; };
2 struct Derived : Base { void doSomething() {} };
3 Base* b = new Derived;
4 // 向上转换 (Derived* → Base*) 自动完成, 无需cast
5 Derived* d1 = static_cast<Derived*>(b); // 假设 b 确实指向 Derived
```

- `dynamic_cast`: 运行期安全向下转型, 处理类之间的多态类型转换, 通过 RTTI 检查转换是否合法, 失败时返回 `nullptr` 或抛出异常。适合在多态环境中进行安全的向下转换
- `const_cast`: 添加或移除对象的 `const/volatile` 限定, 用于调用需要非常量接口或暂时去除常量属性。
- `reinterpret_cast`: 最低安全级别的强制转换, 用于不同类型之间的按位重解释 (如指针与整数互转、不同指针类型互转), 须谨慎使用。

this指针

每个非静态成员函数在编译时都隐含地增加了一个参数: `this`。它是一个指向当前对象的指针。

作用:

1. 访问当前对象的成员, 若成员函数参数名与成员变量同名, 可用 `this->` 前缀来指定访问成员。

代码块

```
1 class Example {
2     int value;
3 public:
4     void setValue(int value) {
5         this->value = value; // 将成员 value 赋值为参数 value
6     }
7 };
```

2. 返回当前对象的引用以实现链式调用

代码块

```
1 class Accumulator {
2     int sum = 0;
3 public:
```

```

4     Accumulator& add(int x) {
5         sum += x;
6         return *this; // 返回当前对象引用
7     }
8     int get() const { return sum; }
9 };
10
11 // 用法:
12 Accumulator acc;
13 acc.add(5).add(10).add(3); // 链式调用

```

3. 在常量成员函数中的类型转换：如果成员函数被声明为常量成员函数（如 `void func() const`），那么 `this` 的类型为 `const ClassName*`，只能访问常量成员；此时尝试修改成员会编译错误

代码块

```

1  class Sample {
2      int value;
3  public:
4      void print() const {
5          std::cout << this->value << "\n"; // this 是 const Sample*
6      }
7      // void modify() const { this->value = 5; } // 错误: 在 const 成员函数中无法修
      改成员
8  };

```

什么是大端序和小端序

大端序：将数据的**最高有效字节（高数据位）**存放在内存的**低地址**，其余字节存放在更高的地址

小端序：将数据的**最低有效字节（低数据位）**存放在内存的低地址，其余字节存放在更高的地址

如果有一个 32 位整数 0x12345678：

代码块

```

1  // 大端序
2  地址  a      a+1    a+2    a+3
3  数据  0x12   0x34   0x56   0x78
4
5  // 小端序
6  地址  a      a+1    a+2    a+3
7  数据  0x78   0x56   0x34   0x12

```


模板

C++中使用模版的优缺点

优点：

- 1. 代码复用，对于不同类型不用重复写相同的代码，使得代码更加简洁。
- 2. 编译时检查，而不是运行时检查，若传入不兼容类型，会在编译时报错而非运行时出错，增强了类型安全性

缺点：

- 1. 代码膨胀：由于模版是编译期实例化的，如果多个不同类型频繁实例化模版会生成大量的代码，增加二进制文件的大小
- 2. 编译时间长
- 3. 调试与调试信息负担：模板在编写时需要考虑更多的细节，且编译器在处理模板时的错误信息通常较为晦涩

类模板和模板类的区别

术语	本质	角色	类比
类模板	是一个模板	代码的“蓝图”或“配方”，不是完整的类。它用参数 <code>T</code> 描述了一族类应该如何生成。	像“饼干的模具”或“菜谱”。本身不能直接“吃”或使用，但可以生产出具体的饼干。
模板类	是一个类	由类模板经过实例化（填入具体类型参数）后生成的、完全具体的、可用的类。	像“用模具压出的巧克力饼干”或“按菜谱做好的红烧肉”。它是一个可以直接使用的具体事物。

代码块

```
1 // =====
2 // 【类模板】 - Class Template
3 // 这是一个“蓝图”，告诉编译器：“如果有人给你一个类型T，你就按照这个结构生成一个类”。
4 // =====
5 template <typename T>
6 class Box { // `Box` 本身是一个类模板
7 private:
8     T content;
9 public:
10     Box(const T& t) : content(t) {}
11     T get() const { return content; }
```

```

12 };
13 // 注意：此时编译器并没有为`Box`创建任何代码，因为它不知道`T`是什么。
14
15 // =====
16 // 【模板类】 - Template Class
17 // 这是将【类模板】实例化后得到的【具体类】。
18 // =====
19 // 当我们在代码中这样使用时：
20 Box<int> intBox(123); // `Box<int>` 就是一个“模板类”
21 Box<std::string> strBox("Hello"); // `Box<std::string>` 是另一个“模板类”
22
23 // 编译器看到 `Box<int>` 后，会拿 `int` 替换掉类模板 `Box` 中的所有 `T`，
24 // 生成一个实实在在的类，其等价于：
25 /*
26 class Box_int { // 一个编译器生成的、真实存在的类（名字是示意）
27 private:
28     int content; // T 被替换为 int
29 public:
30     Box_int(const int& t) : content(t) {}
31     int get() const { return content; }
32 };
33 */
34 // 这个 `Box<int>`（或者说想象中的 `Box_int`）就是一个“模板类”。

```

模板类是在什么时候实现的

1. 编写与定义阶段（编码时）

编写了模板类的通用“蓝图”代码。此时，编译器只是将其作为一套规则记录下来，并不会生成任何实际的类代码。

2. 实例化阶段（编译时）

当编译器在源代码中“看到”你以**具体类型**使用该模板时，它会根据“蓝图”为该特定类型生成一份真实的类或函数代码。这个过程就是“**实例化**”。触发时机：“按需实例化”。只有被用到的模板特化版本才会被生成。

代码块

```

1  template<typename T>
2  class MyBox { T item; public: void set(const T& i) { item = i; } };
3
4  int main() {
5      MyBox<int> intBox;           // 编译器在此处实例化 MyBox<int>
6      intBox.set(42);             // 同时实例化 MyBox<int>::set

```

```
7
8      // MyBox<std::string> stringBox; // 如果这行被注释，则 MyBox<std::string> 不会
      被实例化
9      return 0;
10 }
```

3. 编译与链接阶段（链接时）

每个源文件（.cpp）独立编译。如果多个源文件都使用了 `MyBox<int>`，每个源文件都会生成一份自己的 `MyBox<int>` 代码。链接器最终会合并这些重复的实例，确保最终可执行文件中只有一份 `MyBox<int>` 的定义

什么是模板特化和偏特化

模板特化是指为一个特定的类型实例化提供一个特殊的实现。简单来说，模板特化可以让你为某个特定数据类型定义不同的行为。

1. 完全特化是指为模板的所有参数都指定具体的类型或值，从而为特定类型提供特殊的实现

代码块

```
1  template<typename T>
2  void func(T a) {
3      std::cout << "Generic version: " << a << endl;
4  }
5
6  // 完全特化 int 类型
7  template<>
8  void func<int>(int a) {
9      std::cout << "Specialized version for int: " << a << endl;
10 }
11
12 int main() {
13     func(10); // 调用特化版本
14     func(3.14); // 调用通用版本
15     return 0;
16 }
```

2. 偏特化是指对模板参数的部分进行特化，而不是所有参数。这通常用于类模板，因为函数模板不支持偏特化

代码块

```
1  // 类模板
```

```

2  template<typename T, typename U>
3  class MyPair {
4  public:
5      void print() {
6          std::cout << "Generic Pair" << endl;
7      }
8  };
9
10 // 偏特化, 当第二个参数为 int 时
11 template<typename T>
12 class MyPair<T, int> {
13 public:
14     void print() {
15         std::cout << "Pair with second type as int" << endl;
16     }
17 };
18
19 int main() {
20     MyPair<double, double> p1; // 调用通用模板
21     p1.print(); // 输出: Generic Pair
22
23     MyPair<double, int> p2; // 调用偏特化模板
24     p2.print(); // 输出: Pair with second type as int
25
26     return 0;
27 }

```

可变参数模板

允许创建**可以接受任意数量参数的模板**。这个特性使得编写泛型代码变得更加灵活和强大。具体来说, 可变参数模板通过模板参数包 (template parameter pack) 来实现, 支持任意数量的参数

代码块

```

1  template<typename... Args>
2  void func(Args... args) {
3      // ...
4  }

```

eg: 计算多个数的和

代码块

```

1  #include <iostream>

```

```

2
3 // 基础情况：当没有参数时，返回0
4 int sum() {
5     return 0;
6 }
7
8 // 递归情况：将第一个参数与其余参数的和相加
9 template<typename T, typename... Args>
10 T sum(T first, Args... args) {
11     return first + sum(args...);
12 }
13
14 int main() {
15     std::cout << "Sum: " << sum(1, 2, 3, 4, 5) << std::endl; // 输出15
16     return 0;
17 }

```

注意事项：

- 类型安全：可变参数模板在类型上是安全的，但调用时要确保提供的参数类型与模板要求的类型匹配。
- 参数包的使用：可以结合标准库中的一些特性，比如 `std::tuple`、`std::variant` 等来进行更复杂的数据结构处理。
- 性能：在大多数情况下，可变参数模板提供了良好的性能，但是在处理大量参数时要注意可能导致的编译时间增加。

Lambda表达式

Lambda表达式是C++11引入的一种**匿名函数对象**，它可以在**需要函数对象的地方内联定义**，通常用于封装传递给算法的少量代码。Lambda表达式可以捕获作用域中的变量，并且具有与函数对象类似的行为

基本语法：

代码块

```

1 [capture](parameters) -> return_type {
2     function_body
3 }

```

1. 捕获列表：Lambda 如何访问外部变量

捕获方式	语法	说明
不捕获	<code>[]</code>	不访问任何外部变量
值捕获	<code>[x, y]</code>	创建x, y的副本
引用捕获	<code>[&x, &y]</code>	引用x, y, 可修改原值
隐式值捕获	<code>[=]</code>	捕获所有外部变量的副本
隐式引用捕获	<code>[&]</code>	捕获所有外部变量的引用
混合捕获	<code>[=, &x]</code>	大部分变量值捕获, x引用捕获
移动捕获 (C++14)	<code>[x = std::move(y)]</code>	移动语义捕获

代码块

```

1  int a = 10, b = 20;
2  int c = 30;
3
4  // 值捕获
5  auto lambda1 = [a, b]() { return a + b; };
6
7  // 引用捕获
8  auto lambda2 = [&a, &b]() {
9      a++; b++;
10     return a + b;
11 };
12
13 // 捕获所以外部变量
14 auto lambda3 = [=]() { return a + b + c; };
15
16 // 捕获外部变量引用
17 auto lambda4 = [&]() {
18     a++; b++; c++;
19     return a + b + c;
20 };
21
22 // 混合捕获
23 auto lambda5 = [=, &c]() {
24     c++; // 只有c可以修改
25     return a + b + c;
26 };

```

2. 参数列表 (parameters): 与普通函数参数类似

代码块

```
1 // 无参数
2 auto f1 = []() { return 42; };
3
4 // 带参数
5 auto add = [](int x, int y) { return x + y; };
6
7 // auto参数 (C++14)
8 auto generic = [](auto x, auto y) { return x + y; };
9 cout << generic(3, 4) << endl; // 7
10 cout << generic(3.5, 2.5) << endl; // 6.0
11
12 // 可变参数模板 (C++14)
13 auto sum_all = [](auto... args) {
14     return (args + ...); // 折叠表达式 C++17
15 };
```

3. 返回类型 -> return_type

代码块

```
1 // 自动推导返回类型
2 auto add = [](int a, int b) { return a + b; };
3
4 // 显式指定返回类型
5 auto divide = [](double a, double b) -> double {
6     if (b == 0.0) return 0.0;
7     return a / b;
8 };
9
10 // 多返回类型需要显式指定
11 auto get_value = [](bool flag) -> std::variant<int, std::string> {
12     if (flag) return 42;
13     return "error";
14 };
```

4. 函数体

代码块

```
1 // 简单表达式
2 auto square = [](int x) { return x * x; };
3
4 // 复杂逻辑
5 auto process = [](int x) {
```

```

6     if (x < 0) {
7         return -x;
8     } else if (x > 100) {
9         return 100;
10    }
11    return x;
12 };
13
14 // 捕获外部变量并修改
15 int counter = 0;
16 auto increment = [&counter]() {
17     counter++;
18     return counter;
19 };

```

mutable关键字

默认情况下，**值捕获**的变量在Lambda内是const的，使用 `mutable` 可以修改：

代码块

```

1  int x = 10;
2
3  // 错误：不能修改值捕获的变量
4  // auto lambda = [x]() { x++; };
5
6  // 正确：使用mutable
7  auto lambda = [x]() mutable {
8      x++;
9      cout << "内部 x: " << x << endl;
10 };
11
12 cout << "外部 x: " << x << endl; // 10
13 lambda();                       // 内部 x: 11
14 lambda();                       // 内部 x: 12
15 cout << "外部 x: " << x << endl; // 10 (未改变)

```

线程和进程的区别

进程是计算机正在执行的程序的一个实例，进程是**操作系统资源分配的基本单位**。每个进程都有自己的地址空间，包括代码段、数据段和堆栈。不同进程之间的内存空间互不干扰，操作系统为每个进程

分配一个进程控制块来管理进程。

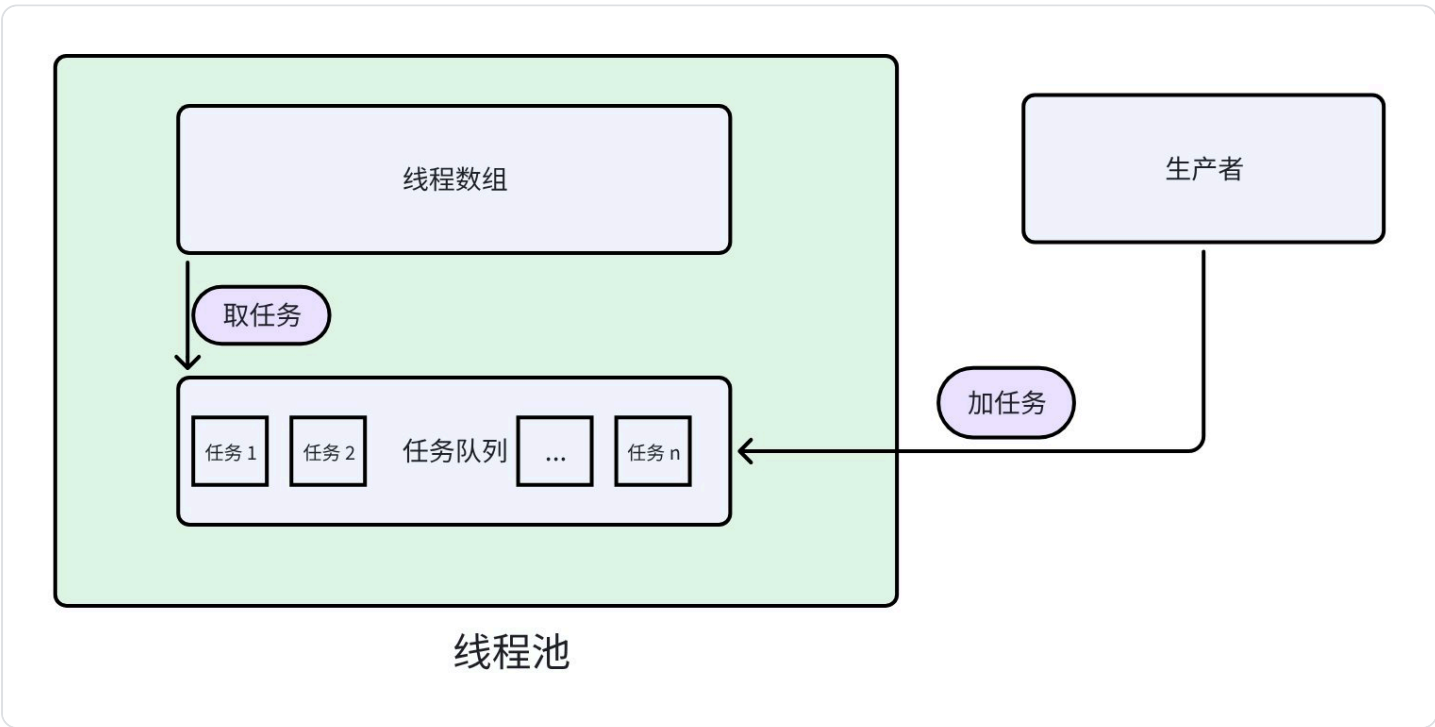
线程是进程执行的执行单元，是**CPU调度的基本单位，是执行的最小单位**。一个进程可以包含多个**线程**，这些线程之间共享全局和堆的数据，每个线程有自己的栈空间和程序计数器，用于跟踪它的执行状态。

线程池

线程池的核心是使用已经创建的线程执行任务而不是有新任务时就创建线程。

线程池的工作原理

- 1. 线程池初始化：会创建一个固定数量的线程，并将它们保存在一个线程队列中，这些线程是等待状态，直到有任务需要执行时，它们会被唤醒并开始工作
- 2. 任务提交：用户将任务（通常是一个可调用对象，如函数或函数对象）提交给线程池，线程池会将任务加入任务队列中
- 3. 任务调度：线程池中的工作线程会不断检查任务队列，如果任务队列中有待执行的任务，线程池的线程就会从任务队列中取出任务并执行
- 4. 任务执行：工作线程从任务队列中取出任务后，执行该任务，完成后返回等待状态，继续等待下一个任务
- 5. 线程复用：线程池中的线程在执行完一个任务后不会退出，而是继续等待新的任务，避免频繁的创建销毁线程带来的开销
- 6. 线程销毁：当程序结束或线程池不再需要时，可以销毁线程池，释放资源



```

1  #include <condition_variable>
2  #include <functional>
3  #include <iostream>
4  #include <mutex>
5  #include <queue>
6  #include <thread>
7  #include <vector>
8
9  class ThreadPool {
10 private:
11     std::vector<std::thread> threads;           // 线程数组
12     std::queue<std::function<void()>> tasks;    // 任务队列
13     std::mutex mtx;                            // 互斥锁
14     std::condition_variable cv;                // 条件变量
15     bool stop;                                 // 线程池终止
16
17 public:
18     ThreadPool(int numThreads) : stop(false) {
19         for (int i = 0; i < numThreads; i++) {
20             threads.emplace_back([this]() {
21                 while (true) {
22                     std::unique_lock<std::mutex> lock(mtx);
23                     // 等待：任务队列不空可以直接取 || 线程终止
24                     cv.wait(lock, [this]() { return !tasks.empty() || stop; });
25                     // 线程终止并且任务队列空，退出
26                     if (stop && tasks.empty()) {
27                         return;
28                     }
29                     // 取任务：取任务队列队头元素，std::move为了避免发生复制节省空间
30                     std::function<void()> getTask(std::move(tasks.front()));
31                     // 取到任务后，把该任务从任务队列中出队
32                     tasks.pop();
33                     // 解锁，让其他进程取
34                     lock.unlock();
35                     getTask();
36                 }
37             });
38         }
39     }
40     ~ThreadPool() {
41     {
42         std::unique_lock<std::mutex> lock(mtx);
43         stop = true;
44     }
45     // 通知所有线程将任务队列中的任务全部完成
46     cv.notify_all();
47     // 等待所有线程完成，用引用是因为线程不支持拷贝

```

```

48     for (auto& t : threads) {
49         t.join(); // 线程等待某个线程完成执行
50     }
51 }
52 // 任务队列中加任务
53 // 参数个数不确定, 适用模板+可变参数
54 template <class F, class... Args>
55 void enqueue(F&& f, Args&&... args) {
56     // 任务
57     std::function<void()> task =
58         std::bind(std::forward<F>(f), std::forward<Args>(args)...);
59     {
60         std::unique_lock<std::mutex> lock(mtx);
61         // 往任务队列中加任务
62         tasks.emplace(std::move(task));
63     }
64     // 通知线程处理
65     cv.notify_one();
66 }
67 };
68
69 int main() {
70     ThreadPool pool(4); // 4个线程
71     // 加任务
72     for (int i = 0; i < 10; i++) {
73         pool.enqueue([i]() {
74             std::cout << "task: " << i << " is runing" << std::endl;
75             std::this_thread::sleep_for(std::chrono::seconds(1));
76             std::cout << "task: " << i << " is done" << std::endl;
77         });
78     }
79
80     return 0;
81 }

```

多线程间如何同步，避免资源竞争

数据资源竞争发生在多个线程读取数据，并且至少有一个线程在进行写操作，而未进行及时的写操作时。

1. 互斥锁（std::mutex）：确保同一时间只有一个线程能访问临界区

```

1  #include <mutex>
2
3  void increment_safe() {
4      for (int i = 0; i < 1000; ++i) {
5          std::lock_guard<std::mutex> lock(mtx); // 自动管理锁
6          ++shared_counter;
7          // lock在作用域结束时自动释放
8      }
9  }
10
11 // 更灵活版本
12 void increment_flexible() {
13     for (int i = 0; i < 1000; ++i) {
14         std::unique_lock<std::mutex> lock(mtx);
15         ++shared_counter;
16         // 可以手动解锁
17         // lock.unlock(); // 如果需要提前解锁
18     }
19 }

```

特性	std::lock_guard	std::unique_lock
设计哲学	轻量、简单、零开销	功能齐全、灵活、有微小开销
所有权与转移	不可转移或复制	可以移动（转移所有权）
锁管理	构造时立即上锁，析构时解锁。中途无法操作	可延迟上锁、手动解锁/重新上锁、尝试上锁
配合条件变量	不能与 std::condition_variable 配合使用	必须使用 std::unique_lock
性能	无额外状态存储，无运行时开销	需维护锁状态，有微小运行时开销

2. 原子操作（std::atomic）：提供无需锁的线程安全操作

代码块

```

1  #include <atomic>
2  #include <thread>
3
4  // 原子变量：操作是原子的，无需加锁

```

```

5  std::atomic<int> atomic_counter(0);
6
7  void atomic_increment() {
8      for (int i = 0; i < 1000; ++i) {
9          // 这些操作都是原子的
10         ++atomic_counter;           // 原子递增
11         atomic_counter.fetch_add(1); // 原子加法
12         atomic_counter.store(10);   // 原子存储
13         int val = atomic_counter.load(); // 原子加载
14     }
15 }
16
17 // 原子标志
18 std::atomic_flag flag = ATOMIC_FLAG_INIT;
19
20 void spinlock_example() {
21     // 自旋锁实现
22     while (flag.test_and_set(std::memory_order_acquire)) {
23         // 忙等待
24     }
25     // 临界区
26     flag.clear(std::memory_order_release);
27 }

```

3. 条件变量（std::condition_variable）：通常与互斥锁一起使用，用于线程间等待特定条件，线程间通信与协调

C++ 中条件变量（`std::condition_variable`）用于在多线程环境中实现线程间的同步，通常配合 `std::mutex` 和 `std::unique_lock` 使用。它允许一个线程等待另一个线程满足某个条件后再继续执行，避免忙等待。

基本使用流程：

1. 等待线程

代码块

```

1  std::mutex mtx;
2  std::condition_variable cv;
3  bool ready = false; // 共享条件
4
5  void waiting_thread() {
6      std::unique_lock<std::mutex> lock(mtx);
7      // 等待条件满足，lambda 防止虚假唤醒
8      cv.wait(lock, []{ return ready; });

```

```
9      // 条件满足后继续执行
10     std::cout << "条件满足，继续执行\n";
11 }
```

2. 通知线程

代码块

```
1 void notifying_thread() {
2     {
3         std::lock_guard<std::mutex> lock(mtx);
4         ready = true; // 改变条件
5     }
6     cv.notify_one(); // 唤醒一个等待线程
7     // 或 cv.notify_all(); 唤醒所有等待线程
8 }
```

完整示例：

代码块

```
1 #include <condition_variable>
2
3 std::mutex cv_mutex;
4 std::condition_variable cv;
5 bool ready = false;
6 int shared_data;
7
8 void producer() {
9     // 生产数据
10    std::this_thread::sleep_for(std::chrono::seconds(1));
11    {
12        std::lock_guard<std::mutex> lock(cv_mutex);
13        shared_data = 42;
14        ready = true;
15    }
16    cv.notify_one(); // 通知一个等待线程
17    // cv.notify_all(); // 通知所有等待线程
18 }
19
20 void consumer() {
21    std::unique_lock<std::mutex> lock(cv_mutex);
22    // 等待条件成立
23    cv.wait(lock, [] { return ready; });
24    // 条件成立，处理数据
25    std::cout << "Data: " << shared_data << std::endl;
```

为什么需要互斥锁？

1. 条件变量本身不保证条件状态的线程安全
2. 必须保护 `ready` 等共享变量的读写
3. `wait()` 会原子性地释放锁并进入等待，被唤醒后重新获取锁

4. 信号量

- 定义：信号量是一个计数器，用于控制对某个共享资源的访问。它可以用来表示可用资源的数量。
- 操作：
 - P 操作（等待/获取，通常称为 `acquire`）：当信号量的值大于 0 时，线程可以获得资源，同时信号量的值减 1；如果信号量的值为 0，则该线程会被阻塞，直到有资源可用。
 - V 操作（释放/信号，通常称为 `release`）：增加信号量的值，表示一个资源已经被释放，并可能唤醒一个等待的线程。

代码块

```

1  #include <mutex>
2  #include <condition_variable>
3
4  class Semaphore {
5  private:
6      int count;
7      std::mutex mtx;
8      std::condition_variable cv;
9
10 public:
11     Semaphore(int initial = 0) : count(initial) {}
12
13     void acquire() {
14         std::unique_lock<std::mutex> lock(mtx);
15         cv.wait(lock, [this] { return count > 0; });
16         --count;
17     }
18
19     void release() {
20         std::lock_guard<std::mutex> lock(mtx);
21         ++count;
22         cv.notify_one();
23     }

```

死锁

概念：死锁是指两个或两个以上的线程相互等待对方释放资源，从而导致他们都无法继续执行的情况。换句话说，死锁是一种资源竞争的情况，其中每个线程都持有一个资源并等待另一个线程持有的资源，结果是所有相关线程都被阻塞。

避免死锁的策略

策略	核心优点	适用场景	注意事项
固定顺序	简单直观，从根源预防	锁数量少、关系清晰	需要团队共识和良好设计，顺序可能不直观
作用域锁与std::lock	标准库支持，安全可靠	需要同时获取多个锁	必须配合RAII使用，是C++最佳实践
层级锁	系统性强，可动态检查	大型复杂系统，锁层级清晰	需要自行实现或使用特定库
超时与检测	提供系统弹性，避免永久阻塞	网络、外部资源调用等不确定场景	不能预防死锁，只能减轻后果

1. 固定顺序上锁：确保所有线程以相同的顺序请求锁。例如，如果有多个资源（锁），要求所有线程都按照固定的顺序获取这些锁。这种方法可以防止形成循环等待的条件。

代码块

```
1  std::mutex mutex_A;
2  std::mutex mutex_B;
3
4  // 正确：所有线程都遵守相同的顺序 (A -> B)
5  void correct_order() {
6      std::lock_guard<std::mutex> lock_a(mutex_A);
7      std::lock_guard<std::mutex> lock_b(mutex_B);
8      // 操作共享资源
9  }
10
11 // 危险：顺序不一致，可能引发死锁
12 void wrong_order() {
13     std::lock_guard<std::mutex> lock_b(mutex_B); // 先锁B，违反顺序
```



```
14     std::lock_guard<std::mutex> lock_a(mutex_A);
15 }
```

2. **作用域锁与 std::lock**：使用RAII包装器管理锁的生命周期，并利用标准库提供的原子性批量上锁功能。

- 使用 `std::lock_guard` 或 `std::unique_lock` 确保锁在离开作用域时被释放。
- 当需要同时获取多个锁时，务必使用 `std::lock` 函数，它可以一次性锁定所有互斥量，避免了因分步上锁而可能产生的竞争条件导致的死锁。

代码块

```
1  std::mutex mutex1, mutex2;
2
3  void safe_with_std_lock() {
4      // 1. 仅关联，不上锁
5      std::unique_lock<std::mutex> lock1(mutex1, std::defer_lock);
6      std::unique_lock<std::mutex> lock2(mutex2, std::defer_lock);
7
8      // 2. 关键：原子性地同时锁定两个互斥量
9      std::lock(lock1, lock2);
10
11     // 3. 此时已安全持有两把锁，操作共享资源
12     // ...
13 } // 作用域结束，lock1, lock2 自动解锁
```

3. 层级锁

核心思想：一种更高级、更系统的固定顺序实现。每个锁被赋予一个层级号，线程只能获取比当前所持锁层级更高的锁。这在**编译期或运行时**就能动态检查，违反规则直接报错或抛出异常。

代码块

```
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4
5  std::mutex lockA;
6  std::mutex lockB;
7
8  // 定义锁的层级
9  enum LockLevel {
10     LEVEL_A = 1,
11     LEVEL_B = 2
12 };
13
```

```

14 // 获取锁的函数，按层级顺序请求锁
15 void acquireLocks(std::mutex& firstLock, std::mutex& secondLock) {
16     // 按照层级顺序获取锁
17     std::lock(firstLock, secondLock); // C++17 提供的 lock 函数，避免死锁
18     std::lock_guard<std::mutex> guard1(firstLock, std::adopt_lock);
19     std::lock_guard<std::mutex> guard2(secondLock, std::adopt_lock);
20
21     // 临界区
22     std::cout << "Locks acquired in order!" << std::endl;
23 }
24
25 void threadFunc() {
26     acquireLocks(lockA, lockB);
27 }
28
29 int main() {
30     std::thread t1(threadFunc);
31     std::thread t2(threadFunc);
32
33     t1.join();
34     t2.join();
35
36     return 0;
37 }

```

4. 超时与检测

核心思想：当上述预防措施难以实施时，作为一种防御性和补救性策略。不试图阻止死锁发生，而是设定一个等待上限，超过则主动放弃，或定期检测系统中是否存在死锁。

做法与示例：

- 使用 `std::mutex` 的 `try_lock_for`（如果支持）或带超时的 `std::unique_lock`。
- 设定一个合理的超时时间，若在时间内未获得锁，则释放已持有的锁，进行回退（可能伴随重试或失败处理）。

代码块

```

1  std::timed_mutex mutex1, mutex2;
2
3  bool try_lock_with_timeout() {
4      auto timeout = std::chrono::milliseconds(100);
5      std::unique_lock<std::timed_mutex> lock1(mutex1, timeout);
6      if(!lock1) return false; // 获取锁1失败
7
8      std::unique_lock<std::timed_mutex> lock2(mutex2, timeout);
9      if(!lock2) {

```

```
10         // 获取锁2失败, lock1会在离开作用域时自动释放
11         return false;
12     }
13     // 成功获取两把锁
14     // ... 操作 ...
15     return true;
16 }
17 // 可以在此处加入重试逻辑或错误上报
```

什么是内存泄露，内存泄露怎么检测？

内存泄露：申请了一块内存空间，使用完毕后没有释放掉。随着程序运行，占用内存持续增加
最好使用智能指针，排查时可以使用Valgrind

代码块

```
1  # 编译时加上-g选项包含调试信息
2  g++ -g -o program program.cpp
3
4  # 使用Valgrind检测
5  valgrind --leak-check=full ./program
6
7  # 输出示例：
8  # ==12345== 100 bytes in 1 blocks are definitely lost
9  # ==12345==    at 0x4C2A2DB: malloc (vg_replace_malloc.c:299)
10 # ==12345==    by 0x4005B6: main (program.cpp:10)
```

操作系统

并发与并行

并发：两个或多个事件在同一时间间隔内发生。这些事件宏观上是同时发生，微观上交替发生的。

并行：两个或多个事件同一时刻同时发生。

单核 cpu 同一时刻只能执行一个程序，各个程序只能**并发**执行；多核 cpu 同一时刻可以执行多个程序，多个程序可以**并行**执行。

内核态和用户态

CPU 有两种状态，“**内核态**”和“**用户态**”

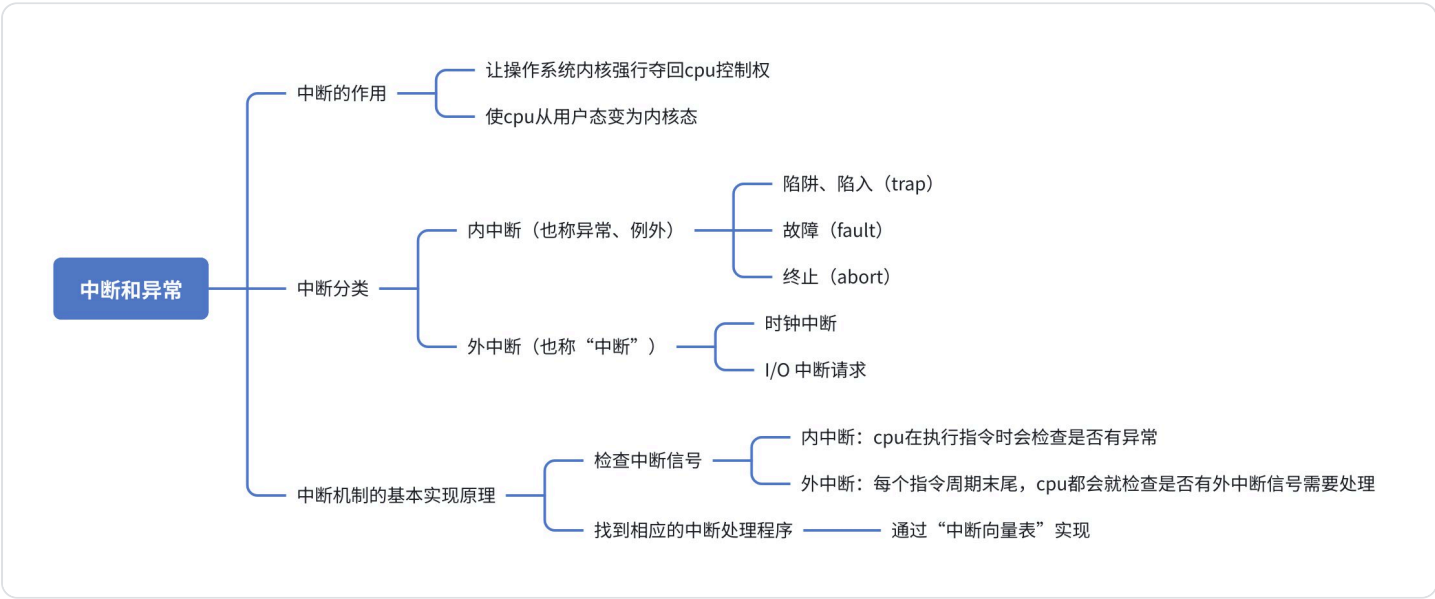
处于**内核态**时，说明此时正在**运行的是内核程序**，此时**可以执行特权指令**

处于**用户态**时，说明此时正在**运行的是应用程序**，此时**只能执行非特权指令**

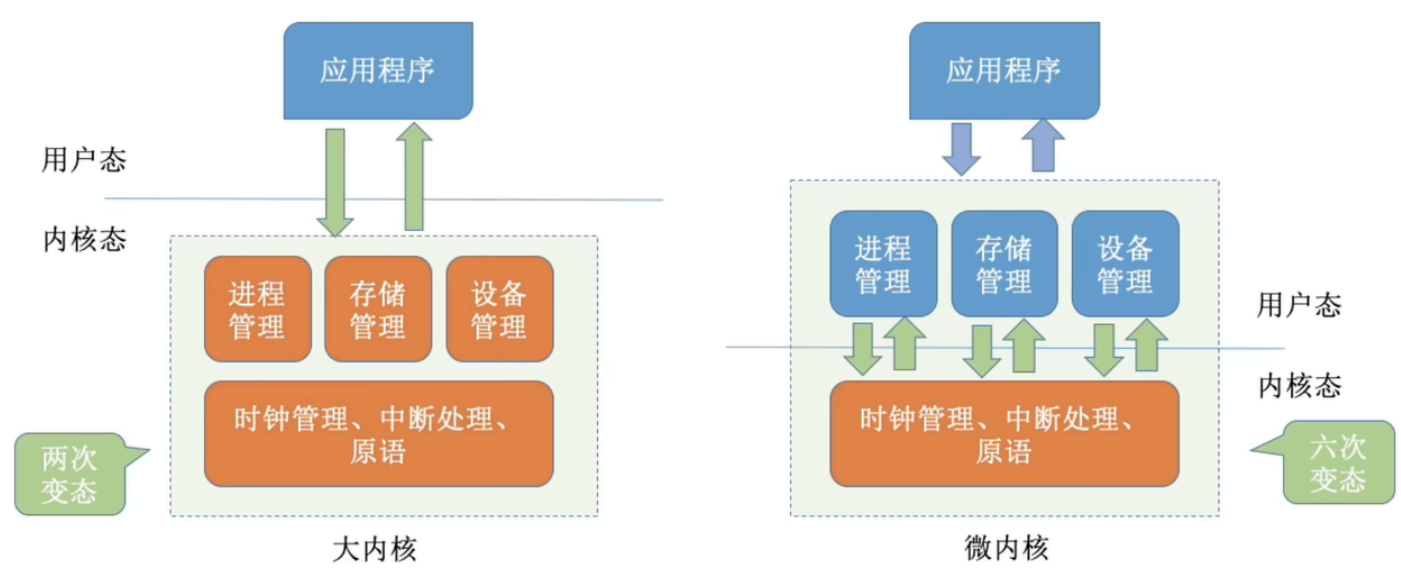
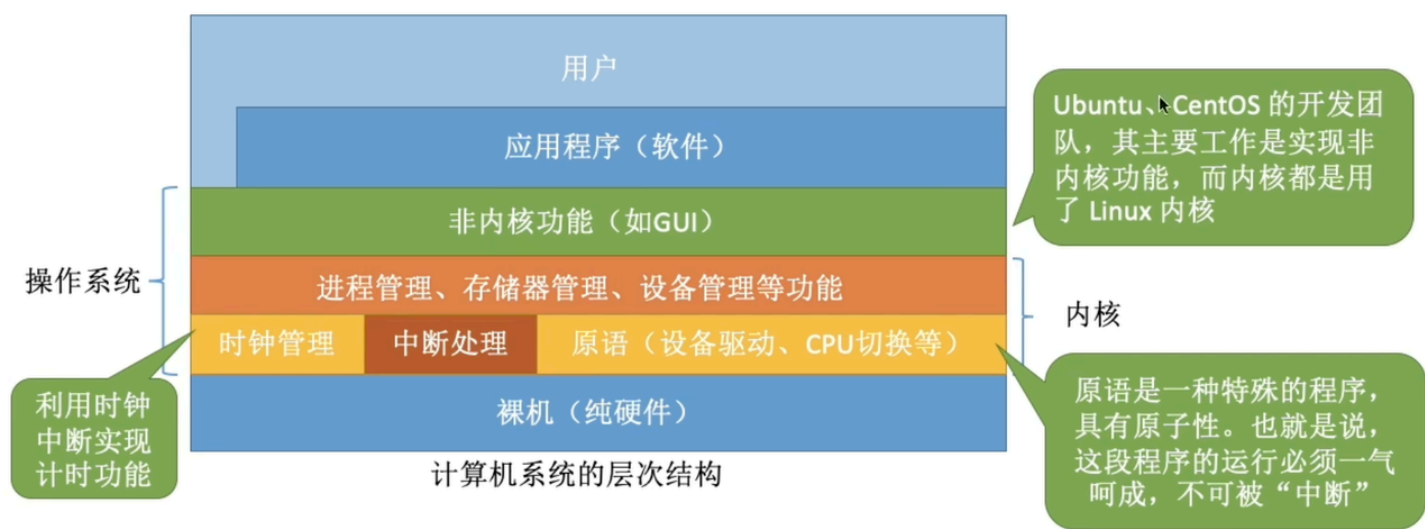
拓展：CPU 中有一个寄存器叫 **程序状态字寄存器（PSW）**，其中有个二进制位，1表示“**内核态**”，0表示“**用户态**”

别名：内核态=核心态=管态；用户态=目态

内核态→用户态：执行一条**特权指令**——**修改PSW**的标志位为“用户态”，这个动作意味着操作系统将主动让出CPU使用权
用户态→内核态：由“**中断**”引发，**硬件自动完成变态过程**，触发中断信号意味着操作系统将强行夺回CPU的使用权



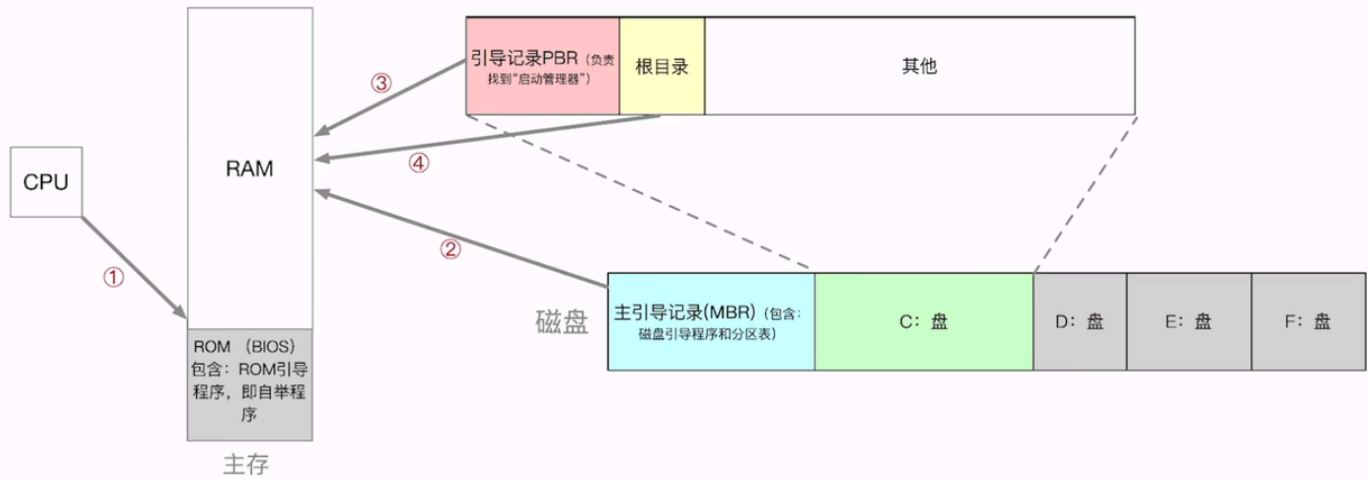
操作系统内核：两种内核形态：大内核、微内核



一个故事：现在，应用程序想要请求操作系统的服务，这个服务的处理同时涉及到进程管理、存储管理、设备管理

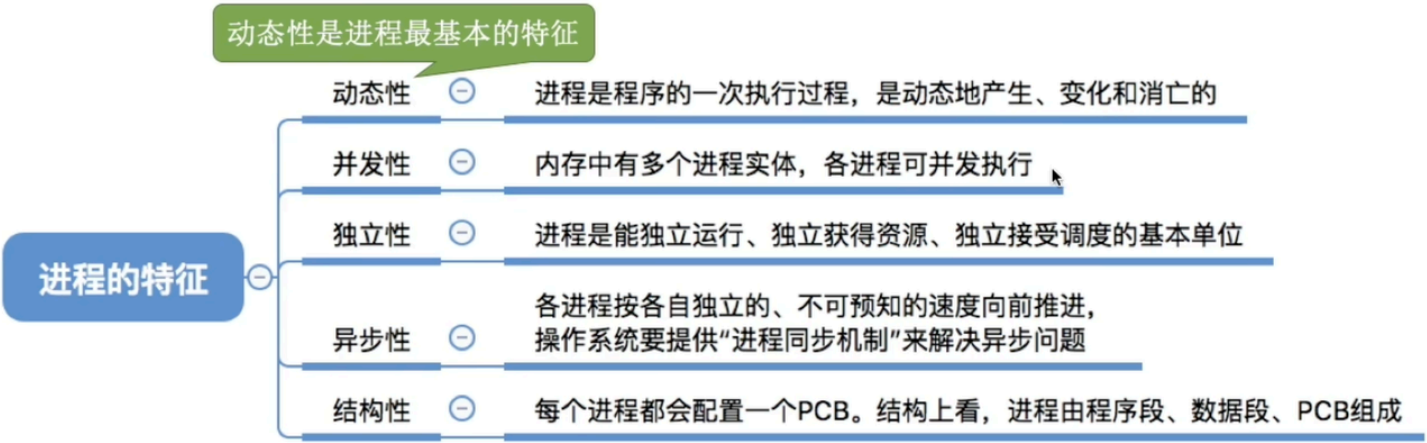
注意：变态的过程是有成本的，要消耗不少时间，频繁地变态会降低系统性能

操作系统引导（开机过程）



进程

4.1 概念：进程是进程实体的运行过程，是系统进行资源分配和调度的一个独立单位。程序是静态的，进程是动态的，相比于程序，进程有以下特征：

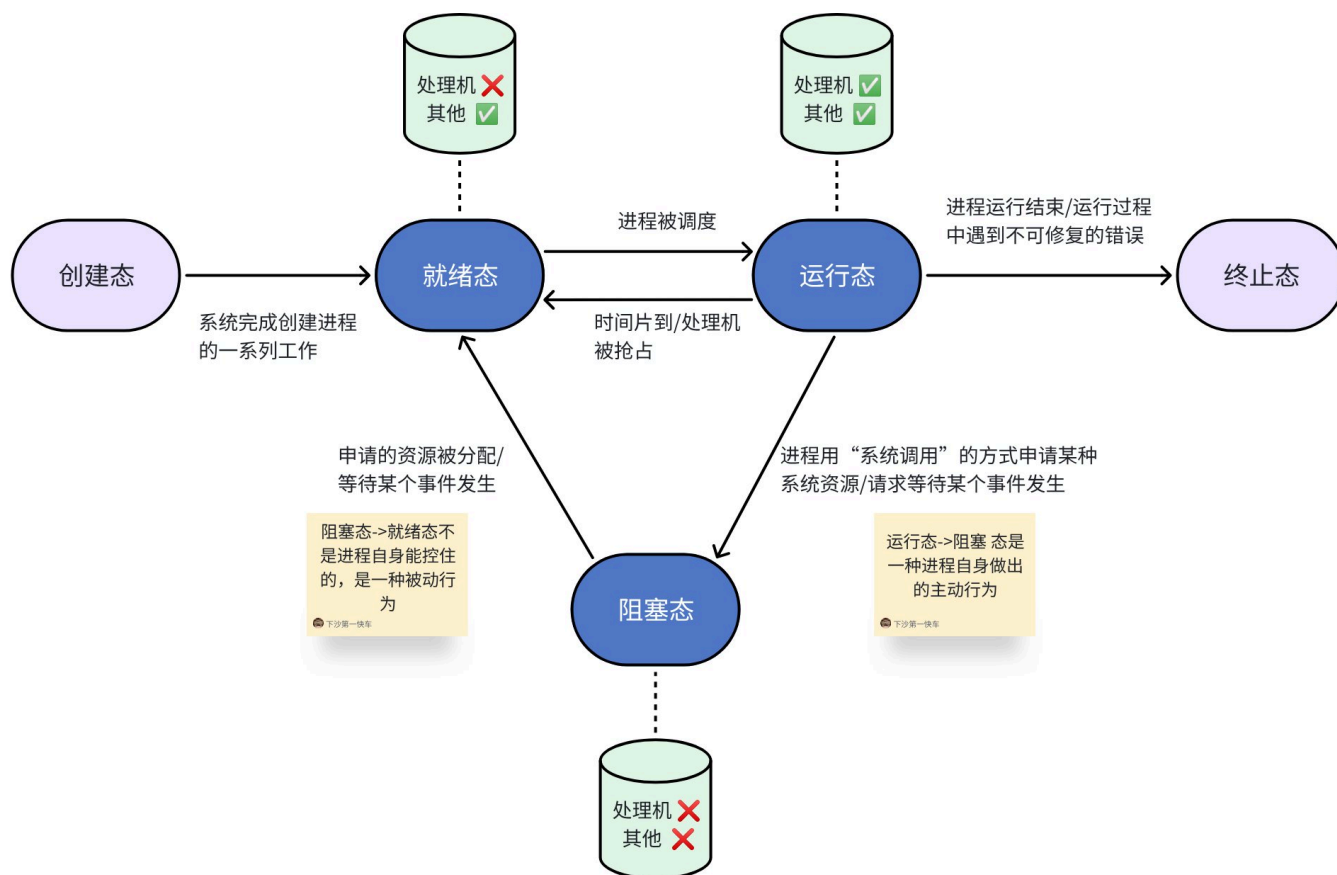


PCB（进程控制块，Process Control Block）是操作系统中用于存储一个进程相关信息的数据结构：

代码块

- 1 **进程标识**：包括进程ID（PID）、父进程ID等，用于唯一标识一个进程。
- 2 **进程状态**：指示进程当前的状态，如就绪、运行、等待等。
- 3 **程序计数器**：指向进程下一条要执行的指令的地址。
- 4 **CPU寄存器**：保存进程在执行时的寄存器状态，包括通用寄存器和特殊寄存器。
- 5 **内存管理信息**：包括基址寄存器和限界寄存器，或其他与进程地址空间相关的信息。
- 6 **进程调度信息**：如优先级、调度队列指针等，影响进程的调度策略。
- 7 **I/O状态信息**：描述进程所使用的I/O设备和文件等信息。

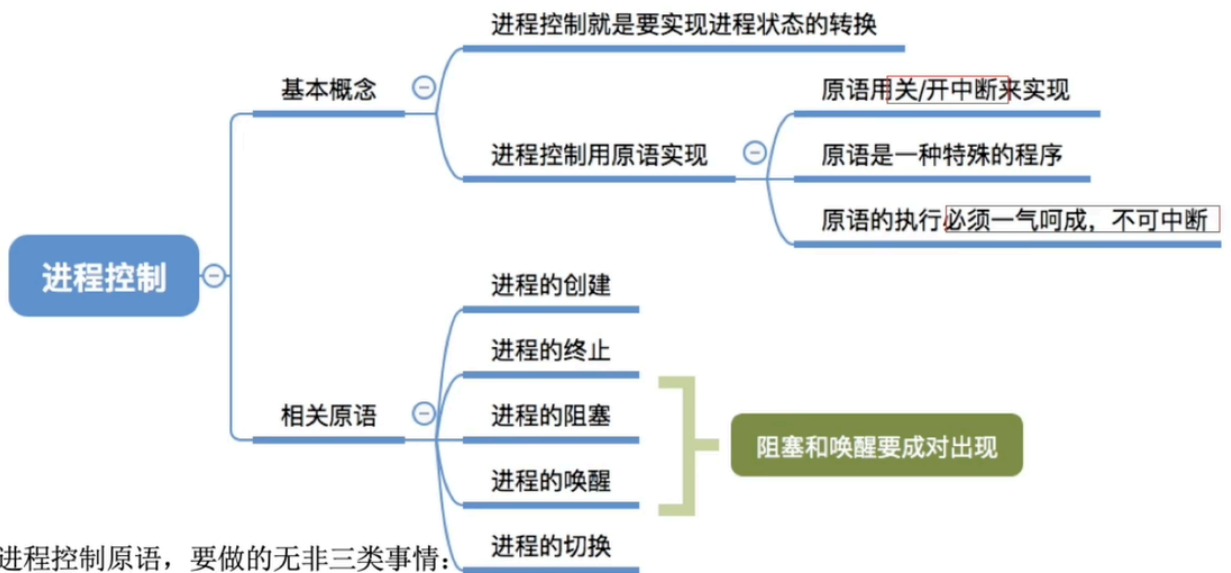
4.2 进程状态的转换



4.3 进程控制

进程控制主要是对系统中的所有进程实施有效的管理。它具有创建新进程、撤销已有进程、实现进程状态转换等功能。

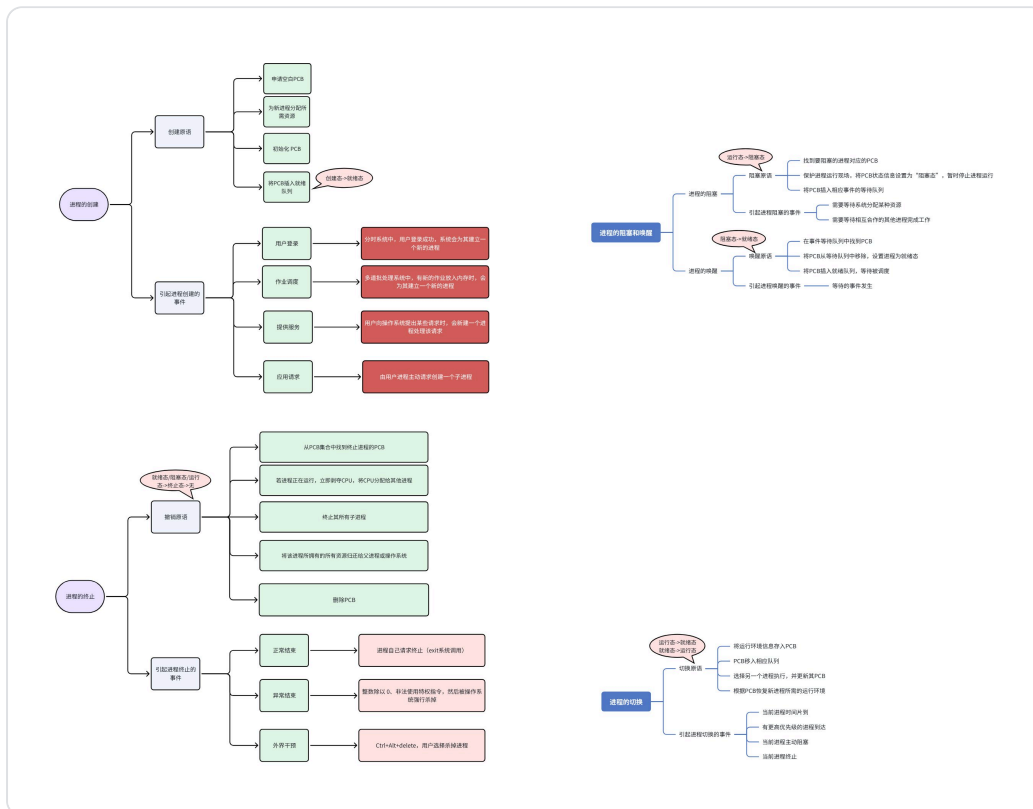
如何实现进程控制：



无论哪个进程控制原语，要做的无非三类事情：

1. 更新PCB中的信息
2. 将PCB插入合适的队列
3. 分配/回收资源

修改进程状态 (state)
保存/恢复运行环境

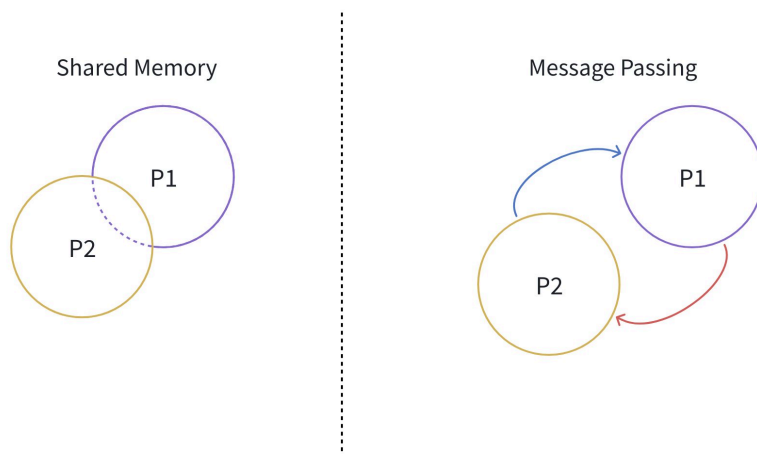


4.4 进程通信 IPC (重点)

inter-process communication(IPC), 指两个或多个进程之间产生数据交互。

IPC 共有两个模型：共享内存和消息传递

IPC



4.4.1 管道通信

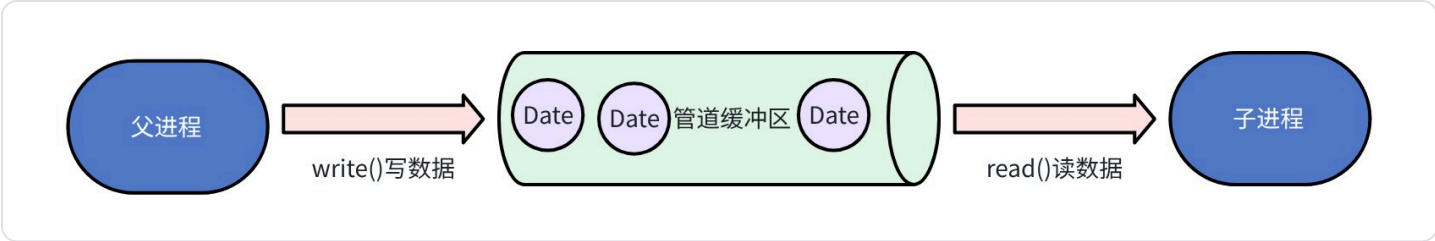
“管道”是一个特殊的共享文件，又名 pipe 文件。其实就是在内存中开辟一个大小固定的内存缓冲区。（循环队列）



- a. 管道只能采用**半双工通信**，某一段时间内**只能实现单项传输**。如果要想实现**双向同时通信**，则需要设置**两个管道**。
- b. 各个进程要**互斥**访问管道（由操作系统实现）
- c. 当管道写满时，写进程将阻塞，直到读进程将管道中的数据取走，即可唤醒写进程
- d. 当管道读空时，读进程将阻塞，直到写进程往管道中写入数据，即可唤醒读进程
- 与共享内存的区别：

共享内存的写和读能在任意位置，而管道通信必须 FIFO，写数据时先把数据写到头部，依次往后写，读数据时只能从头部开始依次读取，读写自由度相较于共享内存较低。

4.4.1.1 匿名管道



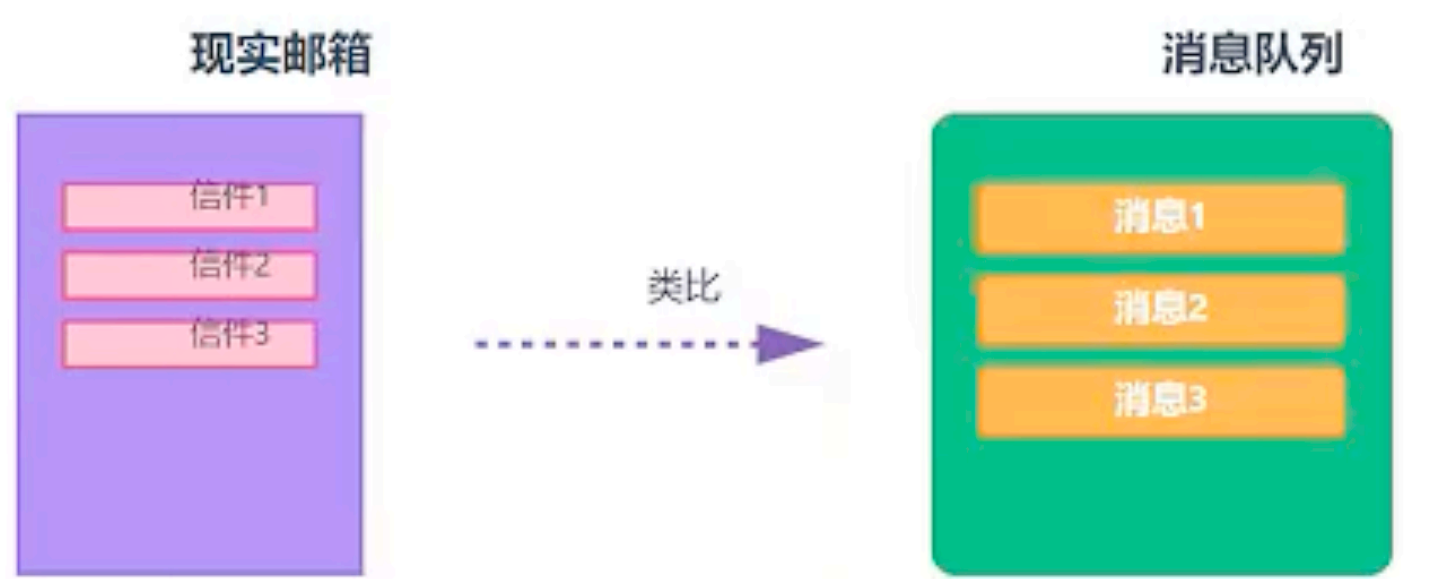
只能用于父子进程或有亲缘关系的进程。数据一旦被读出就会从缓冲区消失。pipe()创建管道，fork()创建子进程，父进程 write()往管道中写入数据，当子进程调用 read()时才从管道中读数据，否则数据会一直存放在管道缓冲区中。

4.4.1.2 命名管道



在文件系统中有一个“管道文件”作为入口，允许任意两个进程通信，遵循先进先出（FIFO）原则。不仅限于父子进程。

4.4.2 消息队列



消息队列可以理解为内核中的一个优先级邮箱

- **原理**：内核维护一个消息队列链表。每个消息都是一个独立的包，包含**类型、长度、数据**。发送方调用 `msgsnd` 将消息从用户态拷贝到内核；接收方调用 `msgrcv` 从内核拷贝到用户态
- **特点**：FIFO 特性；异步通信，发送进程不需要等待；两个进程解耦
- **实际应用**：日志系统

日志系统架构



- **劣势**：每条消息大小通常有限制；每次传输都有两次用户态与内核态之间的数据拷贝（发送方→内核→接收方）

4.4.3 信号量

信号量不传输数据，它是一个计数器，用于并发编程**同步/互斥（很重要）**，保护共享资源

- **原理**：它是一个内核维护的整数变量，只允许进行两种原子操作（不会被中断）：
 - P操作（申请资源）：如果计数器值 ≤ 0 ，则进程休眠等待；否则将计数器减1。
 - V操作（释放资源）：将计数器加1，并唤醒等待在该信号量上的进程。
- **例子**：一个公共电话亭（资源数为1）。信号量初始为1。第一个人进入（P，变0），第二人来看是0，就在外面排队（等待）。第一人出来（V，变1），内核立即唤醒排队的人进入。

信号量的 P/V 操作

P操作 (申请资源)

```
if (信号量 > 0) {  
    信号量--;  
} else { 阻塞等待; }
```

V操作 (释放资源)

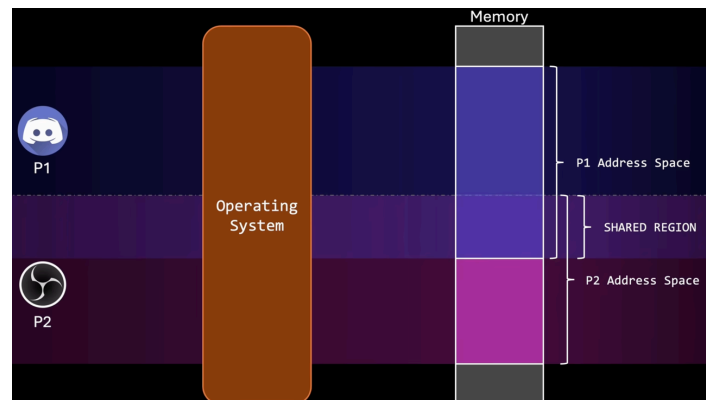
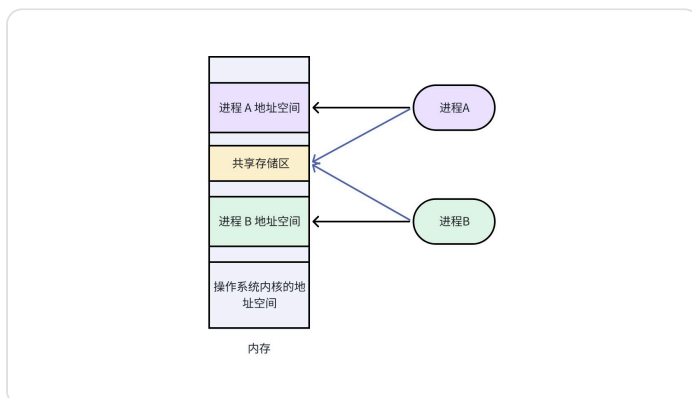
```
信号量++;  
if (有等待进程) {  
    唤醒一个进程; }
```

3

当前信号量值

4.4.4 共享内存

共享内存是最快的 IPC 方式



进程申请共享存储区，共享存储区可被多个进程访问。进程 A 把数据写入共享存储区，进程 B 从共享存储区读数据。

- **原理：**

- a. 在内核中申请一块物理内存。
- b. 进程A 的页表新增一项，把这块物理内存映射到自己的虚拟地址空间。
- c. 进程B 也同样映射这块物理内存。

- **效果：**进程A 直接往自己映射的地址写数据，进程B 立即就能看到——因为没有经过内核拷贝，数据始终停留在物理内存中。
- **致命问题：同步互斥。**当多个进程同时修改共享内存时，数据会混乱。因此它必须配合信号量、互斥锁等同步机制使用。

总结对比：

方式	核心原理	特点	典型场景
管道	内核环形缓冲区	简单、单向、流式、亲缘进程	Shell 命令连接，如 <code>ls grep .txt</code>
消息队列	内核链表消息包	有格式、可随机读、有拷贝开销	小数据量、离散消息
共享内存	同一块物理内存映射	最快、无拷贝、需同步	大数据量、高频通信（视频渲染）
信号量	内核整数计数器	用于同步/互斥，不传数据	保护共享内存、生产者-消费者模型
套接字	网络协议栈 / unix域	支持跨网络、通用、本机优化	所有网络通信、跨进程通信
信号	PCB 位掩码 + 中断	异步通知、异步安全要求高	杀死进程、定时器、异常处理

4.4.5 套接字 socket

套接字（Socket）本质上是一个**编程抽象**，它代表"通信的一端"。你可以把它理解为：

- **操作系统分配给进程的一个"网络门牌号"**：通过这个门牌号，进程可以收发网络数据。
- **对TCP/IP协议栈的封装**：你不用关心数据是如何拆包、路由、重传的，只需要调用 `send` / `recv` 之类的函数。

一个Socket由**三元组**（在原始TCP中）或**五元组**（在连接建立后）唯一标识：

- 原始识别： `{ 协议, 本地IP, 本地端口 }`
- 完整连接： `{ 协议, 本地IP, 本地端口, 目标IP, 目标端口 }`

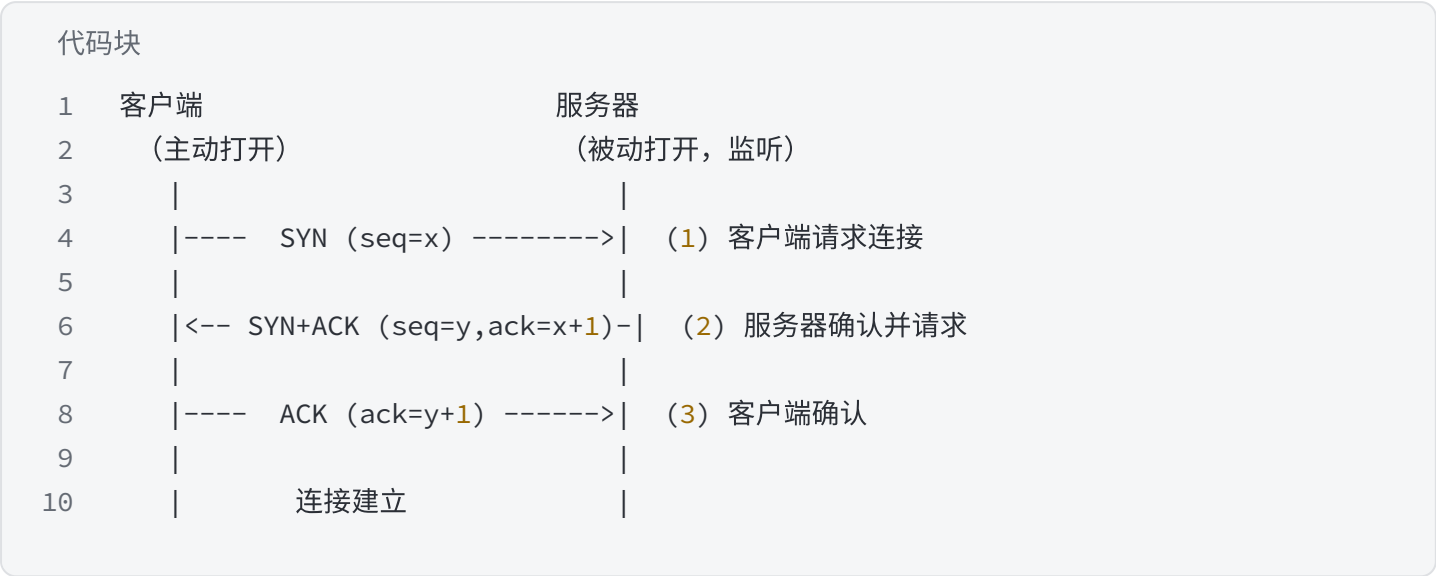
两种主要类型：TCP Socket 与 UDP Socket

特性	TCP Socket （流式）	UDP Socket （数据报式）
协议	面向连接、可靠、字节流	无连接、不可靠、数据报
数据边界	无边界（类似管道）	有边界（一次recv对应一次send）
保证送达	是（重传、确认）	否
顺序保证	是	否（可能乱序）
流量控制	有（滑动窗口）	无
典型场景	HTTP、SSH、文件传输	DNS查询、音视频流、游戏位置同步

1. TCP Socket 工作原理

1.1. 连接建立（三次握手）—— 内核自动完成

这一步不需要你写代码调用，是TCP协议栈在内核中自动完成的



1.2. 数据传输

这是TCP最复杂也最强大的部分。数据不是一个个字节直接发送的，而是：

a) 发送方的过程：

- 你的程序调用 `send(buffer, len)`。数据从用户态拷贝到内核中的发送缓冲区。
- TCP协议栈根据MSS（最大报文段长度，通常由MTU决定）将数据切分为多个报文段。
- 每个报文段加上TCP头部（包含seq序号等），封装成IP包，交给网络层发送。

b) 接收方的过程：

- 收到IP包后，内核取出TCP段，根据序号放入接收缓冲区的适当位置（可能乱序到达）。
- 立即回复ACK确认包（确认序号 = 期望收到的下一个字节序号）。
- 你的程序调用 `recv` 时，数据从内核接收缓冲区拷贝到用户态。

c) 可靠性是如何保证的？

机制	作用	举例
序号及确认	每个字节都有序号，ACK表明前面所有字节已收到	发送字节1-1000，收到ACK=1001，说明前1000字节已收
超时重传	发送包后启动定时器，未收到ACK则重传	RTO通常200ms~120s动态计算
快速重传	收到3个相同ACK，立即重传，不等超时	丢包后快速恢复
滑动窗口	流量控制，避免接收方被淹没	接收方通告"我还能收8192字节"

1.3. 连接关闭（四次挥手） —— 可半关闭



2. UDP Socket 工作原理

- 2.1. 套接字创建：不需要 connect （在客户端意义不同），也不需要 listen / accept。
- 2.2. 发送： sendto(数据, 目标IP+端口) —— 直接封装成UDP数据报，扔进网络就完事。
- 2.3. 接收： recvfrom —— 每次收到一个完整的数据报。

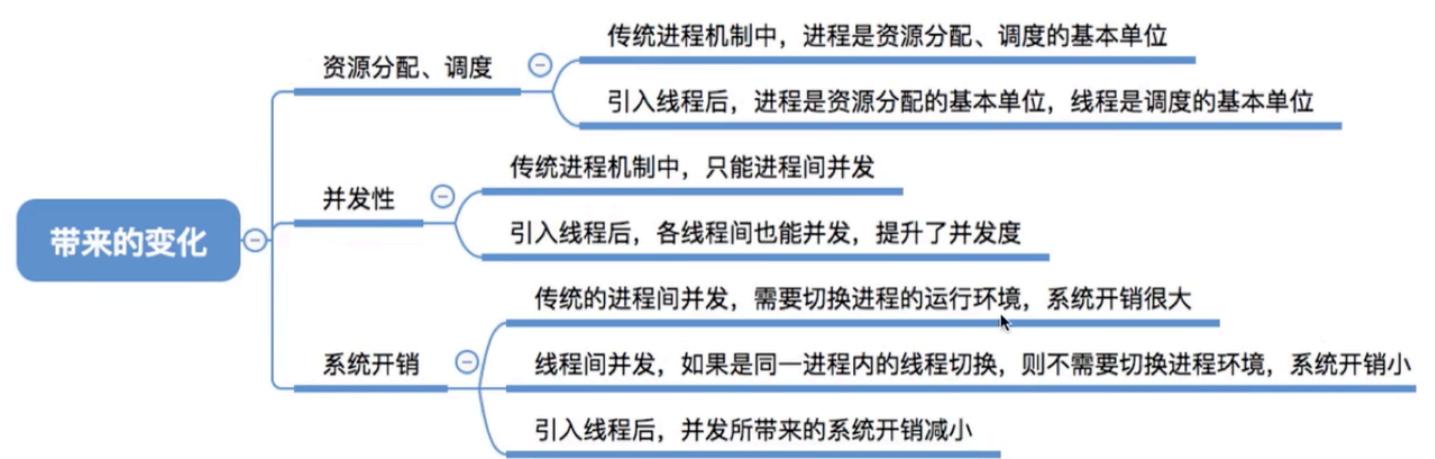
没有握手、没有重传、没有流量控制、没有序号。

这意味着：可能丢包、可能乱序、可能重复、可能超出发送缓冲区就丢弃。

但换来的是：极低延迟、更少的内存拷贝、无连接状态开销。

线程

线程带来的好处：



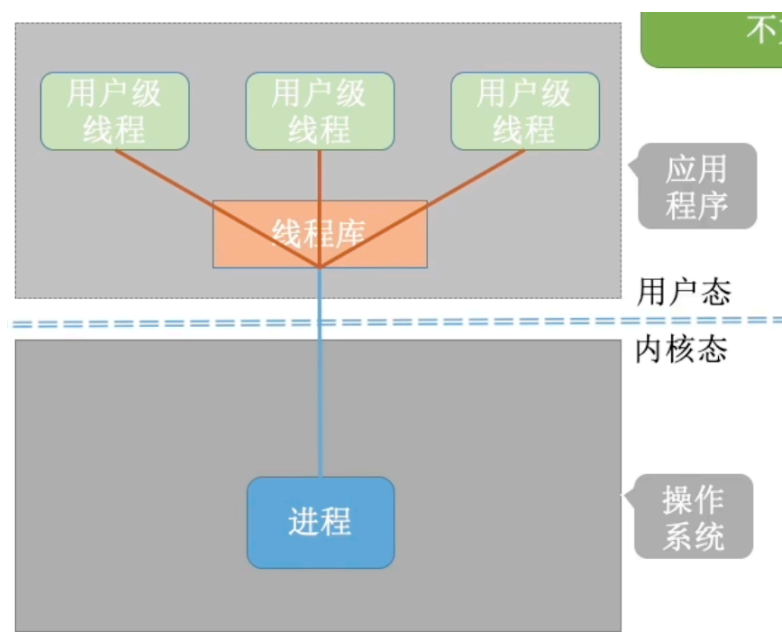
线程的属性：

线程是处理机调度的基本单位；多 CPU 计算机中，各个线程可占用不同的 CPU；**每个线程都有一个特线程 ID、线程控制块（TCB）**；线程也有**就绪、阻塞、运行**三种基本状态；线程几乎不拥有系统资源，因为系统资源都分配给进程；同一进程的不同线程间共享进程的资源；由于共享内存地址空间，同一进程中的线程间通信甚至无需系统干预；同一进程中的线程切换不会引起进程切换，不同进程中的线程切换会引起进程切换；切换同进程内的线程系统开销小，切换进程系统开销大。

线程的实现方式

用户级线程

早期的操作系统只支持进程，不支持线程，当时的“线程”是由线程库实现的。编程语言提供了强大的线程库，可以实现线程的创建、销毁、调度等功能。



优缺点：

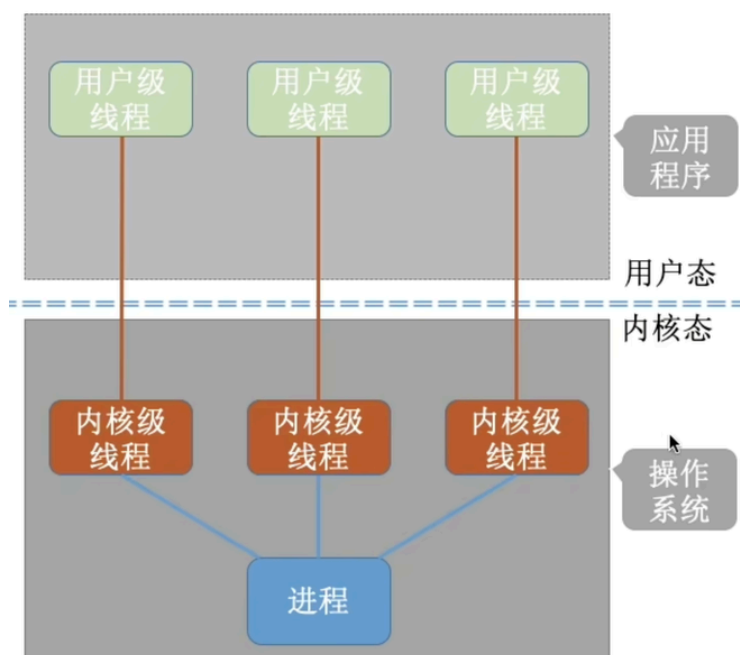
优点：因为用户级线程的切换在用户空间即可完成，不需要切换到内核态，而状态切换开销大，所以线程管理的系统开销小，效率高。

缺点：当一个用户级线程被阻塞后，整个进程 都会被阻塞，并发度不高。多个线程不可在多核处理机上并行运行。

内核级线程

内核级线程（Kernel-Level Thread, KLT, 又称“内核支持的线程”）

由操作系统支持的线程

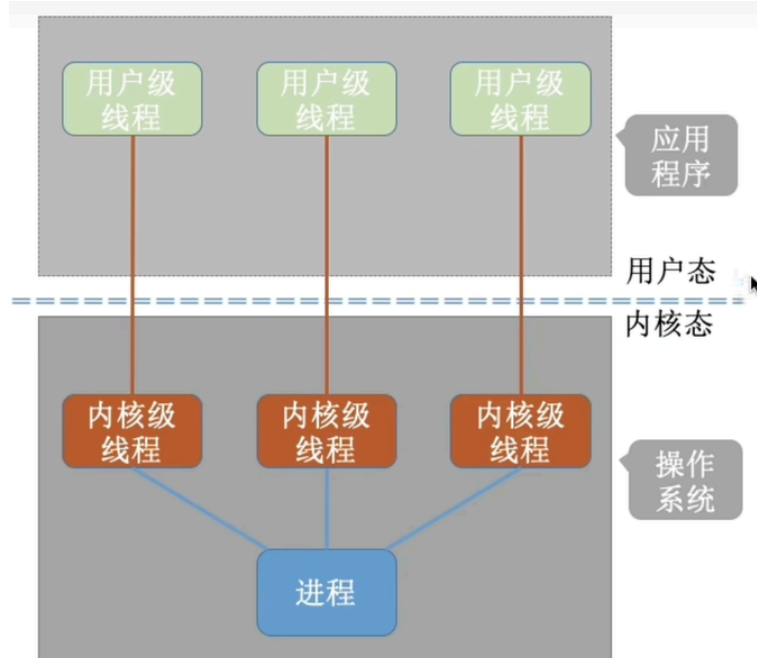


1. 内核级线程的管理工作由操作系统内核完成。
2. 线程调度、切换等工作都由内核负责，因此内核级线程的切换必然需要在核心态下才能完成。
3. 操作系统会为每个内核级线程建立相应的TCB（Thread Control Block，线程控制块），通过TCB对线程进行管理。“内核级线程”就是“从操作系统内核视角看能看到的线程”
4. 优缺点
优点：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。多线程可在多核处理机上并行执行。
缺点：一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，需要切换到核心态，因此线程管理的成本高，开销大。

多线程模型

在支持内核级线程的系统中，根据用户级线程和内核级线程的映射关系，可以划分为几种多线程模型

1. 一对一模型：一个用户级线程映射到一个内核级线程。每个用户进程有与用户级线程同数量的内核级线程。

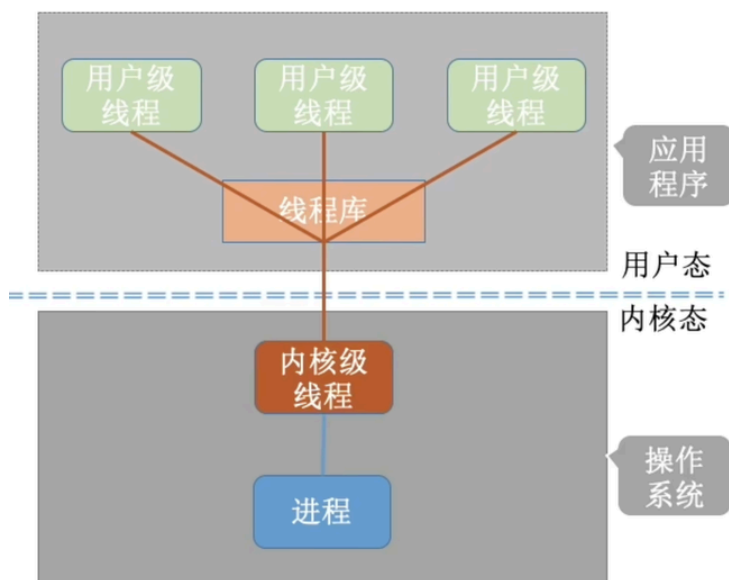


一对一模型：一个用户级线程映射到一个内核级线程。每个用户进程有与用户级线程同数量的内核级线程。

优点：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。多线程可在多核处理机上并行执行。

缺点：一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，需要切换到核心态，因此线程管理的成本高，开销大。

2. 多对一模型：多个用户级线程映射到一个内核级线程。且一个进程只被分配一个内核级线程



多对一模型：多个用户级线程映射到一个内核级线程。且一个进程只被分配一个内核级线程。

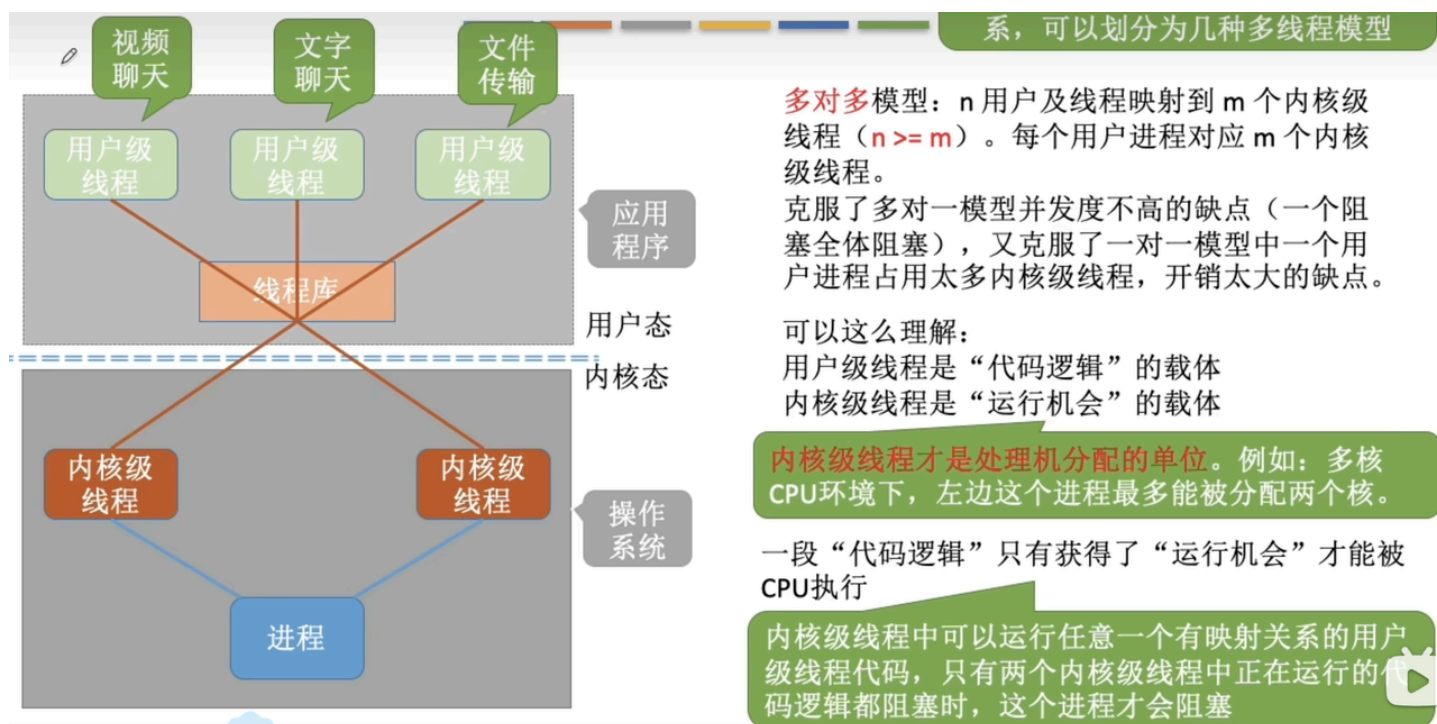
优点：用户级线程的切换在用户空间即可完成，不需要切换到核心态，线程管理的系统开销小，效率高

缺点：当一个用户级线程被阻塞后，整个进程都会被阻塞，并发度不高。多个线程不可在多核处理机上并行运行

重点重点重点：

操作系统只“看得见”内核级线程，因此只有**内核级线程才是处理机分配的单位**。

3. 多对多模型： n 个用户级线程映射到 m 个内核级线程 ($n \geq m$)，每个用户进程对应 m 个内核级线程。



系，可以划分为几种多线程模型

多对多模型： n 用户及线程映射到 m 个内核级线程 ($n \geq m$)。每个用户进程对应 m 个内核级线程。

克服了多对一模型并发度不高的缺点（一个阻塞全体阻塞），又克服了一对一模型中一个用户进程占用太多内核级线程，开销太大的缺点。

可以这么理解：

用户级线程是“代码逻辑”的载体

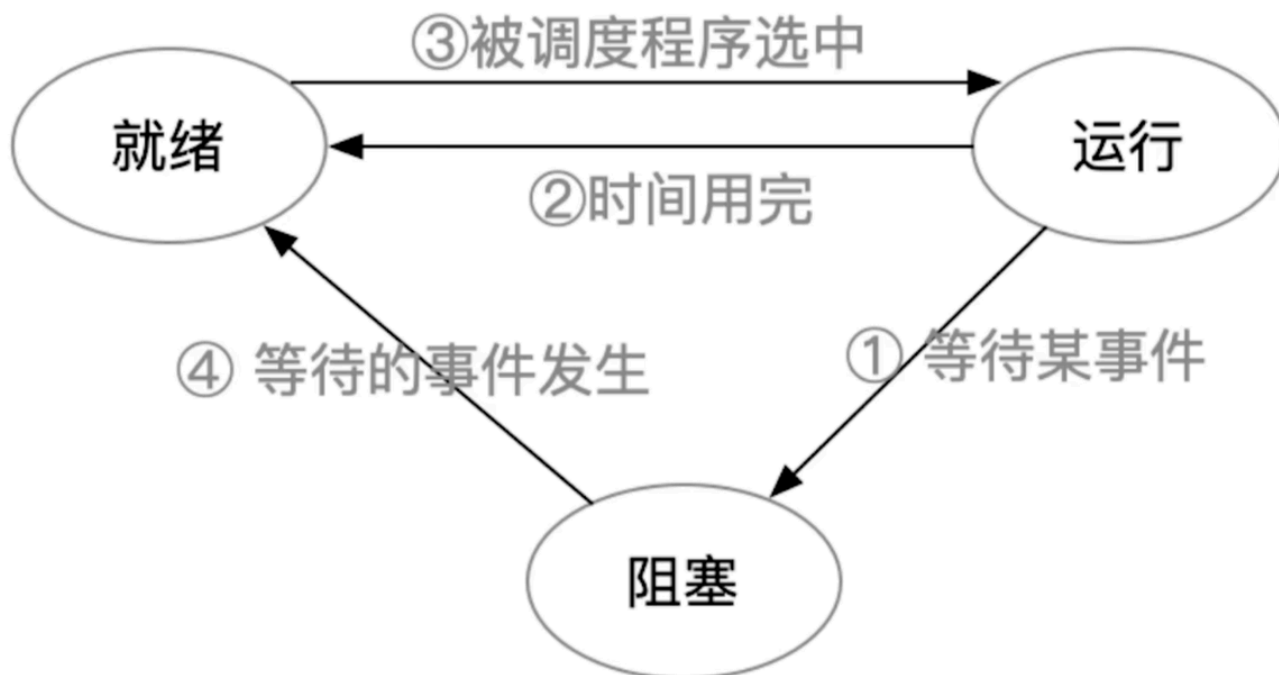
内核级线程是“运行机会”的载体

内核级线程才是处理机分配的单位。例如：多核 CPU 环境下，左边这个进程最多能被分配两个核。

一段“代码逻辑”只有获得了“运行机会”才能被 CPU 执行

内核级线程中可以运行任意一个有映射关系的用户级线程代码，只有两个内核级线程中正在运行的代码逻辑都阻塞时，这个进程才会阻塞

线程的状态与转换



线程组织与控制



处理机调度

查看进程内存泄漏

top/htop - 实时监控

代码块

```
1  # 按内存排序查看
2  top -o %MEM
3  # 或按RES排序
4  top -o RES
5
6  # 监控特定进程
7  top -p <pid>
8
9  # htop更直观（需安装）
10 htop
11 # 在htop中：F6→ 选择MEM%排序，F2设置显示列
```

ps 命令 - 进程快照

代码块

```
1  # 查看进程内存信息
2  ps -p <pid> -o pid,ppid,cmd,%mem,rss,vsz,time
3
4  # 定期监控内存变化
5  watch -n 1 "ps -p <pid> -o pid,rss,vsz,%mem,cmd"
```

如何使用tcmalloc 查看进程内存泄露

编译时链接tcmalloc

代码块

```
1  # 方式1: 编译时链接
2  g++ -o my_program my_program.cpp -ltcmalloc
3
4  # 方式2: 使用LD_PRELOAD（无需重新编译）
5  LD_PRELOAD="/usr/lib/libtcmalloc.so" ./my_program
```

使用Heap Checker（堆检查器）

代码块

```
1  # 设置环境变量启用堆检查
```

```
2 export HEAPCHECK=normal # 可选: minimal, normal, strict, draconian
3 ./my_program
4
5 # 或者直接运行
6 HEAPCHECK=normal ./my_program
```

使用Heap Profiler（堆分析器）

代码块

```
1 1. 生成堆分析文件
2 # 启用堆分析
3 export HEAPPROFILE=/tmp/my_program_heap_profile
4 ./my_program
5
6 # 设置采样间隔（单位：字节）
7 export HEAP_PROFILE_INTERVAL=104857600 # 每100MB采样一次
8 export HEAP_PROFILE_ALLOCATION_INTERVAL=10000 # 每10KB分配采样
9
10 2. 分析堆文件
11 # 使用pprof工具分析
12 pprof --text ./my_program /tmp/my_program_heap_profile.0001.heap
13
14 # 生成PDF可视化
15 pprof --pdf ./my_program /tmp/my_program_heap_profile.0001.heap > profile.pdf
16
17 # 交互模式
18 pprof --web ./my_program /tmp/my_program_heap_profile.0001.heap
```

如果一份新代码 coredump了, 怎么排查(gdb gef)

阶段	关键步骤	常用GDB/GEF命令	说明与技巧
1. 准备阶段	确保程序编译时包含调试信息 2。	<code>gcc -g -o myapp myapp.c</code>	使用 <code>-g</code> 选项编译，这是调试的基础。
	确保系统允许生成core dump 4。	<code>ulimit -c unlimited</code>	在当前终端执行。
2. 加载分析	启动GDB并加载程序和core文件。	<code>gdb myapp core.1234</code>	也可先 <code>gdb myapp</code> ，再 <code>(gdb) core-file core.1234</code> 1。
	在GDB中加载GEF插件（如果未自动加载）。	<code>source ~/gef/gef.py</code>	GEF提供更强大的上下文信息。
3. 定位问题	查看崩溃时的调用栈（最核心步骤）。	<code>bt</code> 或 <code>backtrace</code> 4 5	首先查看栈顶，通常是崩溃的直接原因。
	查看详细的寄存器、堆栈、内存信息（GEF优势）。	<code>context</code>	GEF的 <code>context</code> 命令会一次性展示寄存器、堆栈、反汇编等。
	查看崩溃的具体信号和地址。	<code>info registers</code> <code>x/i \$pc</code>	关注 <code>SIGSEGV</code> （段错误）等信号 4。
4. 深入分析	查看具体栈帧的局部变量和参数。	<code>frame N</code> <code>info locals</code> <code>info args</code> 5	N 为 <code>bt</code> 输出中的帧编号。
	查看崩溃点附近的源代码。	<code>list</code> 5	
	检查可疑内存地址或指针。	<code>x/wx 0x地址</code> <code>p *指针变量</code>	检查是否为空或非法地址。

关键点解析与实战建议

- 理解核心操作：排查的第一步也是最关键的一步，是在GDB中用 `bt` (backtrace) 命令查看崩溃时的函数调用堆栈。堆栈最顶层（#0帧）就是程序崩溃时正在执行的函数和代码位置。
- 发挥GEF插件优势：输入 `context` 命令，GEF会整合显示寄存器状态、堆栈、反汇编代码和源代码（如果有），信息比原生GDB更直观集中。
- 结合代码分析：使用 `list` 命令查看崩溃点附近的源代码，再结合 `info locals` 检查当时的变量值，能有效定位如空指针访问、数组越界等典型问题。

🔧 进阶：如果程序运行中崩溃但没有生成core文件

如果程序崩溃了但没有生成 `core` 文件，可以按以下步骤检查：

- 检查系统设置：通过 `ulimit -c` 命令，确认core文件大小限制是否为 `unlimited`。
- 检查core文件路径：有时core文件被系统重定向，可以通过 `cat /proc/sys/kernel/core_pattern` 命令查看存储路径。
- 主动用GDB运行程序：如果无法生成core，可以直接用GDB运行程序：`gdb ./myapp`，然后在GDB中执行 `run`，程序崩溃后会停在现场，此时同样可以使用 `bt` 和 `context` 命令进行分析

软链接和硬链接的区别

1. 定义

软链接又叫符号链接，这个文件包含了另一个文件的路径名。可以是任意文件或目录，可以链接不同文件系统的文件；

硬链接就是一个文件的一个或多个文件名。把文件名和计算机文件系统使用的节点号链接起来。因此我们可以用多个文件名与同一个文件进行链接，这些文件名可以在同一目录或不同目录

2. 限制不同

硬链接只能对已存在的文件进行创建，不能交叉文件系统进行硬链接的创建；

软链接可对不存在的文件或目录创建软链接；可交叉文件系统

3. 创建方式不同

硬链接不能对目录进行创建，只可对文件创建；

软链接可对文件或目录创建

4. 影响不同

删除一个硬链接文件并不影响其他有相同 inode 号的文件；

删除软链接并不影响被指向的文件，但若被指向的原文件被删除，则相关软连接被称为死链接（即 dangling link，若被指向路径文件被重新创建，死链接可恢复为正常的软链接）

静态库和动态库怎么制作及如何使用

静态库是在**编译时被链接到程序中的库**。生成的可执行文件中包含了所有需要的代码，因此部署时不需要额外的库文件。

代码块

```
1 // 创建
2 g++ -c mylib.cpp -o mylib.o // 编译源码并生成目标文件（.o 文件）
3 ar rcs libmylib.a mylib.o // 创建一个名为 libmylib.a 的静态库，将目标文件打包成静态库 libmylib.a
4 // 使用
5 g++ main.cpp -L. -lmylib -o myprogram // 编译链接，-L. 指定库的搜索路径为当前目录，-lmylib 指定要链接的库（去掉前缀 lib 和后缀 .a）
6 ./myprogram // 运行
```

动态库是在运行时被链接到程序中的库。只有在程序运行时，操作系统才会加载这些库

代码块

```
1 // 创建
2 g++ -fPIC -shared mylib.cpp -o libmylib.so // 编译为共享库。-fPIC 选项用于生成位置无关的代码，-shared 选项表示生成共享库
3 // 使用
4 g++ main.cpp -L. -lmylib -o myprogram // 编译链接
5 export LD_LIBRARY_PATH=./:$LD_LIBRARY_PATH // 如果库不在默认路径（如 /usr/lib 或 /usr/local/lib），需要设置 LD_LIBRARY_PATH 环境变量
6 ./myprogram // 运行
```

特性	静态库	动态库
链接时机	编译时	运行时
文件扩展名	.a 或 .lib	.so 或 .dll
可执行文件大小	较大（包含库代码）	较小（引用库）
更新和维护	需要重新编译所有程序	只需替换库文件
资源共享	不共享，每个程序一份	共享，多个程序可共用
运行时依赖性	无依赖于外部文件	依赖于外部动态库

动态库为什么能够动态链接

它通过生成地址无关的代码，并借助全局偏移表和过程链接表这两个跳板，由操作系统中的动态链接器在程序加载或运行时，将代码中对数据和函数的引用，动态地绑定到内存中实际的地址上。

1. 核心思想：地址无关代码

这是最根本的原因。编译动态库时，编译器会生成地址无关代码。

- 问题：如果不采用地址无关代码，动态库中的代码和数据需要被加载到一个固定的内存地址才能正确运行。但内存地址是有限的资源，多个动态库可能都想占用同一个地址，冲突就无法避免。
- 解决方案：地址无关代码让动态库中的指令不依赖绝对内存地址。它使用相对地址（比如相对于当前指令的偏移量）或间接寻址来访问自己的变量和函数。

这样一来，动态库的代码可以被加载到内存的任何空闲位置（这被称为“重定位”），都能正确执行。

2. 关键机制：全局偏移表和过程链接表

要实现地址无关，动态库需要两个关键的数据结构来处理不同类型的外部符号。

- 全局偏移表：用于访问数据（如全局变量）和调用其他模块的函数。
 - 动态库中的代码想访问一个全局变量时，不会直接写死它的地址，而是通过 GOT 这张表来间接访问。
 - 链接器/加载器在加载动态库时，会把变量真正的内存地址填入 GOT 的对应条目中。代码只需知道 GOT 在自己模块内的相对位置，就能找到真正的变量。
- 过程链接表：用于优化函数调用，特别是跨模块的函数调用（比如调用 printf）。
 - PLT 本身是一小段跳转代码。第一次调用某个函数时，PLT 会通过动态链接器解析出该函数的真实地址，并更新 GOT。之后再次调用，PLT 会直接通过更新后的 GOT 跳转到函数，实现“惰性绑定”，提高程序启动速度。

简单流程：

程序调用 `foo()` -> 跳转到 `PLT` 中的 `foo@plt` 代码 -> `foo@plt` 读取 `GOT` 中的地址 -> 跳转到真正的 `foo` 函数。

3. 关键角色：动态链接器/加载器

操作系统中的这个特殊组件是“动态链接”的导演。

当你在终端运行一个依赖动态库的程序时：

1. 加载：操作系统内核将程序的可执行文件加载到内存，并注意到它依赖一个动态链接器（例如 Linux 下的 `ld-linux.so`），于是把控制权交给它。
2. 解析依赖：动态链接器读取程序文件，找到它依赖的所有动态库列表。
3. 查找与加载：根据系统配置（如 `LD_LIBRARY_PATH` 环境变量、`/etc/ld.so.conf` 配置文件）在磁盘上找到这些动态库文件，并将它们也加载到内存中的空闲区域。
4. 符号解析与重定位：这是关键步骤。动态链接器会处理所有未决的符号。比如程序调用了 `printf`，链接器就在 `libc.so` 中找到 `printf` 的地址，然后去修改程序的 `GOT` 或进行其他重定位操作，把所有跨模块的引用都“绑定”到正确的内存地址。
5. 执行：所有重定位完成后，动态链接器将控制权交还给程序，程序从入口点开始执行。此时，所有对动态库的调用都已准备就绪。

进程调度算法

1. **先来先服务调度算法**：每次调度都是从后备作业（进程）队列中选择一个或多个**最先进入**该队列的作业（进程），将它们调入内存，为它们分配资源、创建进程，然后放入就绪队列
2. **短作业(进程)优先调度算法**：短作业优先(SJF)的调度算法是从后备队列中选择一个或若干个**估计运行时间最短**的作业（进程），将它们调入内存运行
3. **高优先级优先调度算法**：当把该算法用于作业调度时，系统将从后备队列中选择若干个**优先权最高**的作业装入内存。当用于进程调度时，该算法是把处理机分配给就绪队列中优先权最高的进程
4. **时间片轮转法**：每次调度时，把CPU 分配给队首进程，并令其执行一个时间片。时间片的大小从几ms 到几百ms。当执行的时间片用完时，由一个计时器发出时钟中断请求，调度程序便据此信号来停止该进程的执行，并将它送往就绪队列的末尾；然后，再把处理机分配给就绪队列中新的队首进程，同时也让它执行一个时间片
5. **多级反馈队列调度算法**：综合前面多种调度算法

LRU算法及其实现方式

1. **LRU算法**：LRU算法用于缓存淘汰。思路是**将缓存中最近最少使用（最久未访问）的对象删除掉**

2. 实现方式：利用链表和unordered_map。

- 当需要插入新的数据项的时候，如果新数据项在链表中存在（一般称为命中），则把该节点移到链表头部，如果不存在，则新建一个节点，放到链表头部，若缓存满了，则把链表最后一个节点删除即可。
- 在访问数据的时候，如果数据项在链表中存在，则把该节点移到链表头部，否则返回-1。这样一来在链表尾部的节点就是最近最久未访问的数据项

代码块

```
1  class LRUCache {
2      list<pair<int, int>> cache; //创建双向链表
3      unordered_map<int, list<pair<int, int>>::iterator> map; //创建哈希表
4      int cap;
5  public:
6      LRUCache(int capacity) {
7          cap = capacity;
8      }
9
10     int get(int key) {
11         if (map.count(key) > 0){
12             auto temp = *map[key];
13             cache.erase(map[key]);
14             map.erase(key);
15             cache.push_front(temp);
16             map[key] = cache.begin(); //映射头部
17             return temp.second;
18         }
19         return -1;
20     }
21
22     void put(int key, int value) {
23         if (map.count(key) > 0){
24             cache.erase(map[key]);
25             map.erase(key);
26         }
27         else if (cap == cache.size()){
28             auto temp = cache.back();
29             map.erase(temp.first);
30             cache.pop_back();
31         }
32         cache.push_front(pair<int, int>(key, value));
33         map[key] = cache.begin(); //映射头部
34     }
35 };
36
37 /**
```

```
38  * Your LRUCache object will be instantiated and called as such:
39  * LRUCache* obj = new LRUCache(capacity);
40  * int param_1 = obj->get(key);
41  * obj->put(key,value);
42  */
```

进程通信的方式有哪些

进程通信（Inter-Process Communication, IPC）是指在多个进程之间传递数据和信息的技术。由于进程是系统中独立执行的实体，它们之间通常需要某种机制来进行数据交换和同步。以下是一些常见的进程通信方式：

1. 管道（Pipes）

管道是一种最简单的进程间通信方式，允许一个进程将数据输出到另一个进程的输入。

- 无名管道：用于具有亲缘关系的进程（如父子进程）。创建后，数据通过管道从一个进程流向另一个进程。

代码块

```
1  c
2  int fd[2];
3  pipe(fd); // 创建无名管道
```

- 命名管道（FIFO）：可以用于没有亲缘关系的进程。命名管道在文件系统中有一个名字，允许任意进程打开。

代码块

```
1  c
2  mkfifo("myfifo", 0666); // 创建命名管道
```

2. 消息队列（Message Queues）

消息队列是一种更高级的通信方式，允许进程以消息的形式交换数据。它支持优先级，并且消息可以在队列中存储。

- 使用 `msgget` 创建消息队列，使用 `msgsnd` 和 `msgrcv` 发送和接收消息。

代码块

```
1  c
```

```
2  #include <sys/msg.h>struct msgbuf {long mtype; // 消息类型char mtext[100]; // 消息内容
3  };
```

3. 信号量 (Semaphores)

信号量主要用于进程间的同步，以控制对共享资源的访问。信号量可以是二进制的 (mutex) 或计数的，用于管理资源的数量。

- 使用 `sem_init` 初始化信号量，`sem_wait` 和 `sem_post` 用于加锁和解锁。

代码块

```
1  c
2  #include <semaphore.h>sem_t sem;
3  sem_init(&sem, 0, 1); // 初始化二进制信号量
```

4. 共享内存 (Shared Memory)

共享内存是一种高效的进程间通信方式，允许多个进程直接访问同一块内存区域。速度快，但需要使用其他机制 (如信号量) 来同步访问。

- 使用 `shmget` 创建共享内存段，使用 `shmat` 将其映射到进程的地址空间。

代码块

```
1  c
2  #include <sys/shm.h>int shmid = shmget(IPC_PRIVATE, size, IPC_CREAT | 0666);
3  char *ptr = shmat(shmid, NULL, 0); // 映射到地址空间
```

5. 套接字 (Sockets)

套接字不仅可以用于网络编程，也可以用于本地进程间通信。通过 UNIX 域套接字，可以在同一台机器上的进程之间进行通信。

- 使用 `socket` 创建套接字，使用 `bind`、`listen`、`accept` 和 `connect` 进行连接。

代码块

```
1  c
2  int sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
```

6. 文件映射 (Memory-Mapped Files)

文件映射允许多个进程将同一文件映射到各自的虚拟内存地址空间中，从而实现高效的进程间通信。

- 使用 `mmap` 创建内存映射。

代码块

```
1  c
2  #include <sys/mman.h>void *ptr = mmap(NULL, length, PROT_READ | PROT_WRITE,
    MAP_SHARED, fd, 0);
```

7. 信号 (Signals)

信号是一种用于通知进程发生特定事件的机制。进程可以通过信号进行简单的通信，例如终止、暂停或继续执行。

- 使用 `signal` 或 `sigaction` 设置信号处理函数。

代码块

```
1  c
2  signal(SIGINT, handler); // 注册信号处理函数
```

cpu访问内存流程

1. **生成地址**：当 CPU 需要访问内存数据时，首先会通过地址总线生成一个内存地址。这个地址通常是由程序计数器（PC）或某个指令中的操作数决定的。
2. **发送地址**：CPU 将生成的内存地址通过地址总线发送到内存控制器，指示要访问的内存位置。
3. **选择内存单元**：内存控制器接收到地址后，会解码该地址并选择相应的内存单元。如果是读取操作，它将准备从指定的内存单元中读取数据；如果是写入操作，它则会准备接收来自 CPU 的数据。
4. **数据传输**：
 - 读取数据：如果是读取操作，内存单元会将数据通过数据总线发送回 CPU。
 - 写入数据：如果是写入操作，CPU 会将数据发送到数据总线，然后内存单元会接收并存储这些数据。
5. **确认完成**：一旦数据传输完成，CPU 可以继续执行下一个指令或操作。
6. **缓存机制（可选）**：在现代计算机中，CPU 通常会利用缓存（如 L1、L2、L3 缓存）来加速内存访问。如果所需的数据已在缓存中，CPU 将直接从缓存中读取数据，而不必访问主内存，从而提高效率。

简述一下虚拟内存和物理内存，为什么要用虚拟内存，好处是什么？

物理内存：物理内存有四个层次，分别是寄存器、高速缓存、主存、磁盘。

1. 寄存器：速度最快、量少、价格贵。
2. 高速缓存：次之。
3. 主存：再次之。
4. 磁盘：速度最慢、量多、价格便宜。
5. 操作系统会对物理内存进行管理，有一个部分称为**内存管理器(memory manager)**，它的主要工作是有效的管理内存，记录哪些内存是正在使用的，在进程需要时分配内存以及在进程完成时回收内存。

虚拟内存：操作系统为每一个进程分配一个独立的地址空间，但是虚拟内存。虚拟内存与物理内存存在映射关系，通过页表寻址完成虚拟地址和物理地址的转换。

为什么要用虚拟内存：因为早期的内存分配方法存在以下问题：

- (1) 进程地址空间不隔离。会导致数据被随意修改。
- (2) 内存使用效率低。
- (3) 程序运行的地址不确定。操作系统随机为进程分配内存空间，所以程序运行的地址是不确定的。

使用虚拟内存的好处：

- (1) 扩大地址空间。每个进程独占一个4G空间，虽然真实物理内存没那么多。
- (2) 内存保护：防止不同进程对物理内存的争夺和践踏，可以对特定内存地址提供写保护，防止恶意篡改。
- (3) 可以实现内存共享，方便进程通信。
- (4) 可以避免内存碎片，虽然物理内存可能不连续，但映射到虚拟内存上可以连续。

使用虚拟内存的缺点：

- (1) 虚拟内存需要额外构建数据结构，占用空间。
- (2) 虚拟地址到物理地址的转换，增加了执行时间。
- (3) 页面换入换出耗时。
- (4) 一页如果只有一部分数据，浪费内存。

设计模式

说说什么是单例设计模式，如何实现

单例设计模式是一种创建型设计模式，它确保一个类只有一个实例，并提供一个全局访问点来访问该实例

CMAKE

代码块

```
1  cmake_minimum_required(VERSION 4.3.0)
2  # 指定工程名称
3  project(test)
4  # 设置 C++11 标准
5  set(CMAKE_CXX_STANDARD 11)
6  # 设置变量：源文件
7  # set(SRC_LIST add.cpp div.cpp main.cpp mult.cpp sub.cpp)
8  # 搜索源文件方法 1
9  # aux_source_directory(${PROJECT_SOURCE_DIR} SRC_LIST)
10 # 搜索源文件方法 2: CMAKE_CURRENT_SOURCE_DIR是 CMakeLists.txt 所在路径
11 file(GLOB SRC_LIST ${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp)
12 # 指定头文件
13 include_directories(${PROJECT_SOURCE_DIR}/include)
14 # 指定生成的可执行程序名字
15 add_executable(test ${SRC_LIST})
16 #指定可执行程序输出路径(不存在的路径会自动创建)
17 set(EXECUTABLE_OUTPUT_PATH /Users/czw/Desktop/coding/cmake_learn/build/aa)
```

编译：

代码块

```
1  cmake CMakeLists.txt 的路径 // 如：cmake ..
2  make
```

制作动态库或静态库：

代码块

```
1  # 指定库文件输出路径
2  set(LIBRARY_OUTPUT_PATH /Users/czw/Desktop/coding/cmake_learn_2/build/lib)
3  add_library(库名称 STATIC 源文件1 [源文件2] ...)
4  add_library(库名称 SHARED 源文件1 [源文件2] ...)
5  link_libraries(<static lib> [<static lib>...]) # 链接静态库
```

```
6  link_directories(<lib path>)    # 自己制作或者使用第三方提供的静态库路径
```

如：

代码块

```
1  cmake_minimum_required(VERSION 4.3.0)
2  project(test)
3  set(CMAKE_CXX_STANDARD 11)
4  file(GLOB SRC_LIST ${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp)
5  include_directories(${PROJECT_SOURCE_DIR}/include)
6  # 设置库的路径
7  set(LIBRARY_OUTPUT_PATH /Users/czw/Desktop/coding/cmake_learn_2/build/lib)
8  # 制作静态库
9  add_library(calc STATIC ${SRC_LIST})
10 # 制作动态库
11 add_library(calculate SHARED ${SRC_LIST})
```

其他

git

配置命令

- `git config`：配置 Git 的设置。

代码块

```
1  git config --global user.name "Your Name"    # 设置全局用户名
2  git config --global user.email "your_email@example.com" # 设置全局邮箱
```

创建和克隆仓库

- `git init`：初始化一个新的 Git 仓库。

代码块

```
1  git init myproject    # 在当前目录下创建一个名为 myproject 的新仓库
```

- `git clone <repository>`：克隆一个远程仓库到本地。


```
1 git clone https://github.com/user/repo.git # 从 GitHub 克隆仓库
```

查看状态和日志

- `git status`: 查看工作区和暂存区的状态，显示哪些文件被修改、添加或删除。

代码块

```
1 git status
```

- `git log`: 查看提交历史记录。

代码块

```
1 git log           # 显示提交历史
2 git log --oneline  # 以简洁模式显示提交历史
```

添加和提交

- `git add <file>`: 将文件添加到暂存区。

代码块

```
1 git add myfile.txt # 添加特定文件
2 git add .           # 添加当前目录下的所有文件
```

- `git commit -m "message"`: 提交暂存区的更改并附上提交信息。

代码块

```
1 git commit -m "Initial commit" # 提交变更，添加提交说明
```

分支管理

- `git branch`: 查看当前分支列表。

代码块

```
1 git branch # 列出所有本地分支
```

- `git branch <branch-name>`: 创建新分支。

```
1 git branch new-feature # 创建一个名为 new-feature 的新分支
```

- `git checkout <branch-name>`: 切换到指定分支。

代码块

```
1 git checkout new-feature # 切换到 new-feature 分支
```

- `git merge <branch-name>`: 将指定分支合并到当前分支。

代码块

```
1 git merge new-feature # 将 new-feature 分支合并到当前分支
```

- `git branch -d <branch-name>`: 删除指定分支。

代码块

```
1 git branch -d new-feature # 删除 new-feature 分支
```

远程操作

- `git remote`: 管理远程仓库。

代码块

```
1 git remote -v # 显示所有远程仓库的列表
```

- `git remote add <name> <url>`: 添加远程仓库。

代码块

```
1 git remote add origin https://github.com/user/repo.git # 添加名为 origin 的远程仓库
```

- `git push <remote> <branch>`: 将本地分支推送到远程仓库。

代码块

```
1 git push origin master # 将本地 master 分支推送到远程 origin
```

- `git pull <remote> <branch>`: 从远程仓库获取并合并更新。

代码块

```
1 git pull origin master    # 从远程 origin 获取 master 分支的更新
```

修改历史

- `git reset`: 撤销提交。可以使用不同的选项来控制撤销的程度。

代码块

```
1 git reset HEAD~1          # 撤销上一个提交，但保留更改在工作区
2 git reset --hard HEAD~1    # 撤销上一个提交，并丢弃更改
```

- `git revert <commit>`: 创建一个新的提交，以撤销指定的历史提交。

代码块

```
1 git revert abc123          # 撤销提交 abc123 生成一个新的提交
```

标签

- `git tag`: 创建标签，用于标记特定的提交。

代码块

```
1 git tag v1.0              # 创建一个名为 v1.0 的标签
```

- `git tag -d <tag>`: 删除本地标签。

代码块

```
1 git tag -d v1.0           # 删除 v1.0 标签
```

- `git push <remote> <tag>`: 将标签推送到远程仓库。

代码块

```
1 git push origin v1.0      # 将 v1.0 标签推送到远程
```

gcc 编译为可执行程序

`g++ example.cpp -o example -l pthread`

↑ ↑ ↑

源文件 可执行文件名 链接依赖库

1. 基本选项:

- `-o <file>`: 指定输出文件名。

代码块

```
1 gcc -o myprogram myprogram.c
```

- `<source files>`: 需要编译的源代码文件，通常以 `.c` 结尾。
- `<object files>`: 可选项，允许链接现有的目标文件。
- `<libraries>`: 可选项，用于指定链接库。

2. 编译选项:

- `-c`: 只编译源代码文件，不进行链接，生成目标文件（`.o`）。

代码块

```
1 gcc -c myprogram.c
```

- `-S`: 将源代码编译为汇编语言代码，生成 `.s` 文件。

代码块

```
1 gcc -S myprogram.c
```

3. 优化选项:

- `-O0`: 不进行优化（默认）。
- `-O1`, `-O2`, `-O3`: 分别表示不同级别的优化，`-O3` 是最高级别的优化。
- `-Os`: 优化用于减小生成的二进制文件的大小。

4. 调试选项:

- `-g` : 在编译时生成调试信息，以便使用调试工具（如GDB）。

代码块

```
1  bash
2  gcc -g myprogram.c
```

5. 警告选项:

- `-Wall` : 启用所有常见的警告信息。
- `-Wextra` : 启用额外的警告信息。
- `-Werror` : 将警告视为错误，编译失败。

6. 标准选项:

- `-std=<standard>` : 指定所用的C标准，如 `-std=c11`、`-std=c99` 等。

代码块

```
1  gcc -std=c11 myprogram.c
```

7. 链接选项:

- `-l<library>` : 链接动态库。

代码块

```
1  gcc myprogram.o -lm # 链接数学库libm.so
```

- `-L<directory>` : 指定库文件搜索路径。

代码块

```
1  gcc myprogram.o -L/usr/local/lib -lmylib
```

8. 预处理选项:

- `-E` : 仅进行预处理，输出预处理后的代码。

代码块

```
1  gcc -E myprogram.c
```

- `-I<directory>` : 指定头文件搜索路径。

代码块

```
1 gcc -I/usr/local/include myprogram.c
```

9. 其他有用选项:

- `-fPIC` : 生成位置无关的代码，通常用于创建共享库。
- `-pthread` : 启用POSIX线程支持。
- `-static` : 静态链接库，而不是使用动态链接。

简述GDB常见的调试命令

GDB调试: gdb调试的是可执行文件，在gcc编译时加入 `-g`，告诉gcc在编译时加入调试信息，这样gdb才能调试这个被编译的文件 `gcc -g test.cpp -o test`

代码块

- 1 **GDB命令格式:**
- 2 `quit`: 退出gdb, 结束调试
- 3 `list`: 查看程序源代码
- 4 `list 5, 10`: 显示5到10行的代码
- 5 `list test.cpp:5, 10`: 显示源文件5到10行的代码，在调试多个文件时使用
- 6 `list get_sum`: 显示`get_sum`函数周围的代码
- 7 `list test.c get_sum`: 显示源文件`get_sum`函数周围的代码，在调试多个文件时使用
- 8 `reverse-search`: 字符串用来从当前行向前查找第一个匹配的字符串
- 9 `run`: 程序开始执行
- 10 `help list/all`: 查看帮助信息
- 11 `break`: 设置断点
- 12 `break 7`: 在第七行设置断点
- 13 `break get_sum`: 以函数名设置断点
- 14 `break 行号或者函数名 if 条件`: 以条件表达式设置断点
- 15 `watch 条件表达式`: 条件表达式发生改变时程序就会停下来
- 16 `next`: 继续执行下一条语句，会把函数当作一条语句执行
- 17 `step`: 继续执行下一条语句，会跟踪进入函数，一次一条的执行函数内的代码